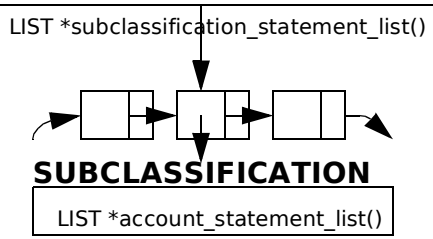
ELEMENT (1)

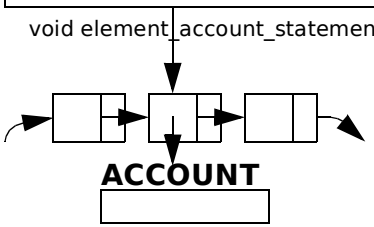
```
/* Usage */
LIST *element_statement_list(
	LIST *element_name_list,
	char *transaction_date_time_closing(),
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_latest,
	boolean fetch_transaction );

/* Process */
for char *element_name in element_name_list
{
	list_set(
		element_list,
		element_statement_fetch(
			element_name,
			transaction_date_time_closing,
			fetch_subclassification_list,
			fetch_account_list,
			fetch_journal_latest,
			fetch_transaction ) );
}

/* Usage */
ELEMENT *element_fetch( char *element_name );

/* Process */
static LIST *element_list();

ELEMENT *element_seek(
	element_name,
	element_list() );

/* Usage */
LIST *element_list( void );

/* Process */
char *element_system_string(
	ELEMENT_SELECT,
	ELEMENT_TABLE,
	“1 = 1” /* where */ );

LIST *element_system_list(
	element_system_string() );

/* Usage */
LIST *element_system_list(
	char *element_system_string() );

/* Process */
FILE *element_pipe( char *element_system_string() );

for input[] in string_input( element_pipe() )
{
	list_set(
		list,
		element_parse( input ) );
}

pclose( element_pipe() );

/* Usage */
ELEMENT *element_parse( char *input );

/* Process */
ELEMENT *element_new( element_name );

ELEMENT *element_calloc( void );

/* Usage */
ELEMENT *element_statement_fetch(
	char *element_name,
	char *transaction_date_time_closing(),
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_latest,
	boolean fetch_transaction );

/* Process */
char *element_primary_where(
	char *element_name );

char *element_system_string(
	char *ELEMENT_SELECT,
	char *ELEMENT_TABLE,
	char *element_primary_where() /* where */ );

FILE *element_pipe( element_system_string() );

ELEMENT *element_statement_parse(
	string_input( input[], element_pipe() ),
	transaction_date_time_closing,
	fetch_subclassification_list,
	fetch_account_list,
	fetch_journal_latest,
	fetch_transaction );

pclose( element_pipe() );

/* Usage */
ELEMENT *element_statement_parse(
	char *string_input( input[], element_pipe() ),
	char *transaction_date_time_closing,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_latest,
	boolean fetch_transaction );

/* Process */
ELEMENT *element = element_parse( input );

if ( fetch_subclassification_list )
{
	LIST *subclassification_statement_list(
		element_primary_where(
			element_name ),
		transaction_date_time_closing,
		fetch_account_list,
		fetch_journal_latest,
		fetch_transaction );
}
```



ELEMENT (2)

```
/* Usage */
void element_list_sum(
      LIST *element_statement_list );

/* Process */
for ELEMENT *element in element_statement_list
{
      element->sum =
            element_sum(
                  element );
}

/* Usage */
double element_sum( ELEMENT *element );

/* Process */
double subclassification_list_sum(
      /* Sets subclassification->sum */
      element->subclassification_statement_list );

/* Usage */
void element_list_account_statement_list_set(
   LIST *element_statement_list );

/* Process */
for ELEMENT *element in element_statement_list
{
      if ( element->sum )
      {
            void element_account_statement_list_set(
                  element );
      }
}

/* Usage */
void element_account_statement_list_set(
      ELEMENT *element );

/* Process */
element->account_statement_list =
      LIST *subclassification_account_statement_list(
            element->subclassification_statement_list );

if ( element->account_statement_list )
{
   element->account_statement_list =
         LIST *account_balance_sort_list(
               element->account_statement_list );
}

/* Usage */
void element_delta_prior_percent_set(
      ELEMENT *prior_element /* in/out */,
      ELEMENT *current_element );

/* Process */
int prior_element->delta_prior_percent =
      int float_delta_prior_percent(
            prior_element->sum,
            current_element->sum );

void subclassification_list_delta_prior_percent_set(
      prior_element->
            subclassification_statement_list
                  /* prior_subclassification_list in/out */,
      current_element->
            subclassification_statement_list
                  /* current_subclassification_list */ );

/* Usage */
void element_account_transaction_count_set(
   LIST *element_statement_list,
   char *transaction_begin_date_string(),
   char *transaction_date_time_closing() );

/* Process */
for ELEMENT *element in element_statement_list
{
   subclassification_account_transaction_count_set(
         element->subclassification_statement_list,
         transaction_begin_date_string,
         transaction_date_time_closing );
}

/* Usage */
void element_account_action_string_set(
   LIST *element_statement_list,
   char *application_name,
   char *session_key,
   char *login_name,
   char *role_name,
   char *transaction_begin_date_string(),
   char *transaction_date_time_closing() );

/* Process */
for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         void subclassification_account_action_string_set(
               element->subclassification_statement_list,
               application_name,
               session_key,
               login_name,
               role_name,
               transaction_begin_date_string,
               transaction_date_time_closing );
   }
}
```

void element_account_statement_list_set()

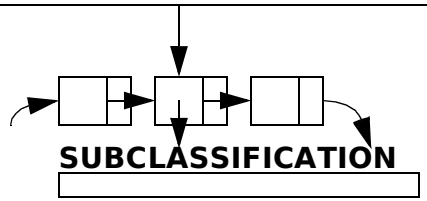


ELEMENT (3)

```
/* Usage */
void element_percent_of_asset_set(
   LIST *element_statement_list );

/* Process */
ELEMENT *element_asset =
   element_seek(
         ELEMENT_ASSET,
         element_statement_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum
   && !element_is_nominal( element->element_name ) )
   {
         int element->percent_of_asset =
               int float_percent_of_total(
                     /* Set by statement_fetch()->
                           element_list_sum() */
                     element->sum,
                     element_asset->sum );

         subclassification_percent_of_asset_set(
               element->subclassification_statement_list,
               element_asset->sum );
   }
}

/* Usage */
void element_percent_of_revenue_set(
   LIST *element_statement_list );

/* Process */
ELEMENT *element_revenue =
   element_seek(
         ELEMENT_REVENUE,
         element_statement_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum
   && element_is_nominal( element->element_name )
   {
         int element->percent_of_revenue =
               int float_percent_of_total(
                     /* Set by statement_fetch()->
                           element_list_sum() */
                     element->sum,
                     element_revenue->sum );

         subclassification_percent_of_revenue_set(
               element->subclassification_statement_list,
               element_revenue->sum );
   }
}

/* Usage */
double element_net_income(
   LIST *element_statement_list );

/* Process */
double revenue =
   element_seek_sum(
         ELEMENT_REVENUE,
         element_statement_list );

double gain =
   element_seek_sum(
         ELEMENT_GAIN,
         element_statement_list );

double expense =
   element_seek_sum(
         ELEMENT_EXPENSE,
         element_statement_list );

double loss =
   element_seek_sum(
         ELEMENT_LOSS,
         element_statement_list );

return (revenue + gain) - (expense + loss);

/* Attributes */
char *element_name;
boolean accumulate_debit;
LIST *subclassification_statement_list;
LIST *account_statement_list;
double sum;
int percent_of_asset;
int percent_of_revenue;

/* Public */
boolean element_is_nominal(
   char *element_name );

ELEMENT *element_seek(
   char *element_name,
   LIST *element_list );

double element_list_debit_sum(
   LIST *element_statement_list );

double element_list_credit_sum(
   LIST *element_statement_list );

double element_seek_sum(
   char *element_name,
   LIST *element_statement_list );
```

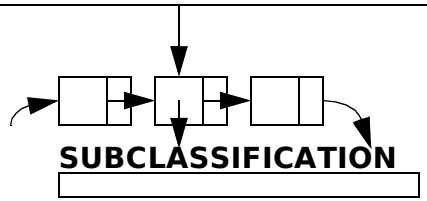
ELEMENT_FUND

```
/* Usage */
ELEMENT_FUND *element_fund_fetch(
	char *element_name,
	char *fund_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

/* Process */
char *element_primary_where(
	element_name );

char *element_system_string(
	ELEMENT_SELECT,
	ELEMENT_TABLE,
	element_primary_where()
		/* where */ );

ELEMENT_FUND *element_fund_parse(
	char *input,
	char *fund_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

ELEMENT_FUND *element_fund_new(
	char *element_name,
	char *fund_name );

ELEMENT_FUND *element_fund_calloc( void );

/* Input */
char *element_name;
char *fund_name;
boolean accumulate_debit;
```



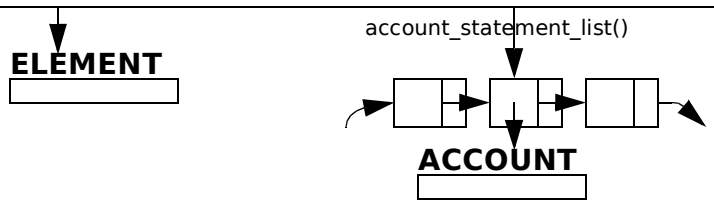
ELEMENT_PROGRAM

```
/* Usage */
ELEMENT_PROGRAM *element_program_fetch(
	char *element_name,
	char *program_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

/* Process */
char *element_primary_where(
	element_name );

char *element_system_string(
	ELEMENT_SELECT,
	ELEMENT_TABLE,
	element_primary_where()
		/* where */ );

ELEMENT_PROGRAM *element_program_parse(
	char *input,
	char *program_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

ELEMENT_PROGRAM *element_program_new(
	char *element_name,
	char *program_name );

ELEMENT_PROGRAM *element_program_calloc( void );

/* Input */
char *element_name;
char *program_name;
boolean accumulate_debit;
```



SUBCLASSIFICATION (1)

```
/* Usage */
LIST *subclassification_statement_list(
   char *element_primary_where(),
   char *transaction_date_time_closing(),
   boolean fetch_account_list,
   boolean fetch_journal_latest,
   boolean fetch_transaction );

/* Process */
char *subclassification_system_string(
   char *SUBCLASSIFICATION_SELECT,
   char *SUBCLASSIFICATION_TABLE,
   char *where );

FILE *subclassification_pipe(
   char *subclassification_system_string() );

for char *input in
   string_input( input[], subclassification_pipe() )
{
   list_set(
         list,
         subclassification_statement_parse(
               input,
               transaction_date_time_closing,
               fetch_account_list,
               fetch_journal_latest,
               fetch_transaction ) );
}

pclose( subclassification_pipe() );

/* Usage */
SUBCLASSIFICATION *
   subclassification_statement_parse(
         char *input,
         char *transaction_date_time_closing(),
         boolean fetch_account_list,
         boolean fetch_journal_latest,
         boolean fetch_transaction );

/* Process */
SUBCLASSIFICATION *subclassification =
   SUBCLASSIFICATION *subclassification_parse(
         input,
         0 /* not fetch_element */ );

if ( fetch_account_list )
{
   LIST *account_statement_list(
         subclassification_primary_where(
               subclassification_name ),
         transaction_date_time_closing,
         0 /* not fetch_subclassification */,
         0 /* not fetch_element */,
         fetch_journal_latest,
         fetch_transaction );

   LIST *account_statement_list =
         LIST *account_balance_sort_list(
               account_statement_list );
}

/* Usage */
SUBCLASSIFICATION *subclassification_fetch(
   char *subclassification_name,
   boolean fetch_element );

/* Process */
if ( fetch_element )
{
   static LIST *subclassification_list(
         “1 = 1” /* where */,
         1 /* fetch_element */ );

   SUBCLASSIFICATION *subclassification_seek(
         char *subclassification_name,
         LIST *subclassification_list() );
}
else
{
   char *subclassification_primary_where(
         char *subclassification_name );

   char *subclassification_system_string(
         SUBCLASSIFICATION_SELECT,
         SUBCLASSIFICATION_TABLE,
         subclassification_primary_where() /* where */ );

   FILE *subclassification_pipe(
         subcalssification_system_string() );

   SUBCLASSIFICATION *subclassification_parse(
         string_input(
               input[],
               subclassification_pipe() ),
         0 /* not fetch_element */ );

   pclose( subclassification_pipe() );
}

/* Usage */
SUBCLASSIFICATION *subclassification_parse(
   char *input,
   boolean fetch_element );

/* Process */
SUBCLASSIFICATION *subclassification_new(
   char *subclassification_name );

SUBCLASSIFICATION *subclassification_calloc( void );

if ( fetch_element )
{
   ELEMENT *element =
         element_fetch(
               subclassification->element_name );
}

/* Usage */
LIST *subclassification_account_statement_list(
   LIST *subclassification_statement_list() );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   list_set_list(
         account_statement_list,
         subclassification->account_statement_list );
}
```



SUBCLASSIFICATION (2)

```
/* Usage */
LIST *subclassification_list(
   char *where,
   boolean fetch_element );

/* Process */
char *subclassification_system_string(
   SUBCLASSIFICATION_SELECT,
   SUBCLASSIFICATION_TABLE,
   where );

LIST *subclassification_system_list(
   subclassification_system_string(),
   fetch_element );

/* Usage */
LIST *subclassification_system_list(
   char *subclassification_system_string(),
   boolean fetch_element );

/* Process */
FILE *subclassification_pipe(
   subclassification_system_string() );

for char input[] in string_input( subclassification_pipe() )
{
   list_set(
         list,
         subclassification_parse(
               input,
               fetch_element ) );
}

pclose( subclassification_pipe() );

/* Usage */
double subclassification_list_sum(
   /* Sets subclassification->sum */
   LIST *subclassification_statement_list() );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   subclassification->sum =
         account_list_sum(
               subclassification->
                     account_statement_list );
   list_sum += subclassification->sum;
}

/* Usage */
void subclassification_list_delta_prior_percent_set(
   LIST *prior_subclassification_list
         /* in/out */,
   LIST *current_subclassification_list );

/* Process */
for SUBCLASSIFICATION *current_subclassifiation in
   current_subclassification_list
{
   SUBCLASSIFICATION *prior_subclassification =
         subclassification_seek(
               current_subclassification->
                     subclassification_name,
               prior_subclassification_list
                     /* subclassification_list */ );

   void subclassification_delta_prior_percent_set(
         prior_subclassification /* in/out */,
         current_subclassification );
}

/* Usage */
void subclassification_delta_prior_percent_set(
   SUBCLASSIFICATION *prior_subclassification
         /* in/out */,
   SUBCLASSIFICATION *current_subclassification );

/* Process */
int prior_subclassification->delta_prior_percent =
      int float_delta_prior_percent(
            prior_subclassification->sum,
            current_subclassification->sum );

void account_list_delta_prior_percent_set(
   prior_subclassification->account_statement_list
         /* prior_account_list in/out */,
   current_subclassification->account_statement_list
         /* current_account_list */ );

/* Usage */
void subclassification_account_statement_list(
   LIST *subclassification_statement_list );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   list_set_list(
         list,
         subclassification->account_statement_list );
}

/* Usage */
void subclassification_account_transaction_count_set(
         LIST *subclassification_statement_list,
         char *transaction_begin_date_string(),
         char *transaction_date_time_closing() );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   void account_transaction_count_set(
         subclassification->account_statement_list,
         transaction_begin_date_string,
         transaction_date_time_closing );
}

/* Usage */
void subclassification_account_action_string_set(
   LIST *subclassification_statement_list,
   char *application_name,
   char *session_key,
   char *login_name,
   char *role_name,
   char *transaction_begin_date_string(),
   char *transaction_date_time_closing() );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   void account_list_action_string_set(
         subclassification->account_statement_list,
         application_name,
         session_key,
         login_name,
         role_name,
         transaction_begin_date_string,
         transaction_date_time_closing );
}
```

SUBCLASSIFICATION (3)

```
/* Usage */
void subclassification_percent_of_asset_set(
   LIST *subclassification_statement_list,
   double element_asset_sum );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   int subclassification->percent_of_asset =
         int float_percent_of_total(
               /* Set by statement_fetch()->
                     element_list_sum() */
               subclassification->sum,
               element_asset_sum );

   void account_percent_of_asset_set(
         subclassification->account_statement_list,
         element_asset_sum );
}

/* Usage */
void subclassification_percent_of_revenue_set(
   LIST *subclassification_statement_list,
   double element_revenue_sum );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   int subclassification->percent_of_revenue =
         int float_percent_of_total(
               /* Set by statement_fetch()->
                     element_list_sum() */
               subclassification->sum,
               element_revenue_sum );

   void account_percent_of_revenue_set(
         subclassification->account_statement_list,
         element_revenue_sum );
}

/* Usage */
SUBCLASSIFICATION *
   subclassification_element_list_seek(
         char *subclassification_name,
         LIST *element_statement_list );

/* Process */
for ELEMENT *element in element_statement_list
{
   SUBCLASSIFICATION *
         subclassification_seek(
               subclassification_name,
               element->subclassification_statement_list );
}

/* Attributes */
char *subclassification_name;
char *element_name;
int display_order;
ELEMENT *element;
LIST *account_statement_list;
double sum;
int percent_of_asset;
int percent_of_revenue;

/* Public */
double subclassification_list_debit_sum(
   LIST *subclassification_statement_list,
   boolean element_accumulate_debit );

double subclassification_list_credit_sum(
   LIST *subclassification_statement_list,
   boolean element_accumulate_debit );
```

ACCOUNT (1)

```
/* Usage */
LIST *account_statement_list(
    char *subclassification_primary_where(),
    char *transaction_date_time_closing(),
    boolean fetch_subclassification,
    boolean fetch_element,
    boolean fetch_journal_latest,
    boolean fetch_transaction );

/* Process */
char *account_system_string(
    char *ACCOUNT_SELECT,
    char *ACCOUNT_TABLE,
    char *subclassification_primary_where
        /* where */ );

FILE *account_pipe( char *account_system_string() );

for input[] in string_input( account_pipe() )
{
    list_set(
        statement_list,
        account_statement_parse(
            input,
            transaction_date_time_closing,
            fetch_subclassification,
            fetch_element,
            fetch_journal_latest,
            fetch_transaction ) );
}

pclose( account_pipe() );

/* Usage */
ACCOUNT *account_statement_parse(
    char *input,
    char *transaction_date_time_closing,
    boolean fetch_subclassification,
    boolean fetch_element,
    boolean fetch_journal_latest,
    boolean fetch_transaction );

/* Process */
ACCOUNT *account =
    account_parse(
        input,
        fetch_subclassification,
        fetch_element );

if ( fetch_journal_latest )
{
    ACCOUNT_JOURNAL *account_journal_latest(
        account_name,
        transaction_date_time_closing,
        fetch_transaction );
}

/* Usage */
ACCOUNT *account_key_fetch(
    char *hard_coded_account_key,
    boolean fetch_subclassification,
    boolean fetch_element );

/* Process */
char *account_hard_coded_key_where(
    char *hard_coded_account_key );

ACCOUNT *account_parse(
    string_pipe(
        account_system_string(
            ACCOUNT_SELECT,
            ACCOUNT_TABLE,
            account_hard_coded_key_where(
                hard_coded_account_key ) ) ),
    fetch_subclassification,
    fetch_element );
```

SUBCLASSIFICATION

```
ELEMENT *element;
```

```
account_journal_latest()
```

ACCOUNT_JOURNAL

```
/* Usage */
ACCOUNT_JOURNAL *
  account_journal_latest(
    char *account_name,
    char *transaction_date_time_closing,
    boolean fetch_transaction );

/* Process */
ACCOUNT_JOURNAL *
  account_journal_calloc(
    void );

JOURNAL *journal =
  journal_latest(
    account_name,
    transaction_date_time_closing,
    fetch_transaction );

/* Attributes */
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double previous_balance;
double debit_amount;
double credit_amount;
double balance;
TRANSACTION *transaction;
```

TRANSACTION

ACCOUNT (2)

```
/* Usage */
ACCOUNT *account_fetch(
   char *account_name,
   boolean fetch_subclassification,
   boolean fetch_element );

/* Process */
if ( fetch_subclassification && fetch_element )
{
   static LIST *account_list(
         “1 = 1” /* where */,
         1 /* fetch_subclassification */,
         1 /* fetch_element */ );

   ACCOUNT *account_seek(
         account_name,
         account_list() );
}
else
{
      char *account_primary_where(
            char *account_name );

      char *account_system_string(
            ACCOUNT_SELECT,
            ACCOUNT_TABLE,
            account_primary_where() /* where */ );

      ACCOUNT *account_parse(
            input,
            fetch_subclassification,
            fetch_element );
}

/* Usage */
ACCOUNT *account_parse(
      char *input,
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
if ( fetch_subclassification )
{
      account->subclassification =
            subclassification_fetch(
                  account->subclassification_name,
                  fetch_element );
}

/* Usage */
LIST *account_list(
      char *where,
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
char *account_system_string(
      ACCOUNT_SELECT,
      ACCOUNT_TABLE,
      where );

LIST *account_system_list(
      account_system_string(),
      fetch_subclassification,
      fetch_element );

/* Usage */
LIST *account_system_list(
      char *account_system_string(),
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
FILE *account_pipe( char *account_system_string() );

for input[] in string_input( account_pipe() )
{
      list_set(
            list,
            account_parse(
                  input,
                  fetch_subclassification,
                  fetch_element ) );
}

pclose( account_pipe() );

/* Usage */
LIST *account_balance_sort_list(
      LIST *account_statement_list() );

/* Process */
int account_balance_match_function(
      ACCOUNT *account_from_list,
      ACCOUNT *account_compare );

ACCOUNT_JOURNAL *from_list_latest_journal =
      account_from_list->account_journal_latest;

ACCOUNT_JOURNAL *compare_latest_journal =
      account_compare->account_journal_latest;

if ( !from_list_latest_journal
||   !from_list_latest_journal->balance )
{
      return 1;
}

if ( !compare_latest_journal
||   !compare_latest_journal->balance )
{
      return 1;
}

if (   from_list_latest_journal->balance <=
      compare_latest_journal->balance )
{
      return 1;
}
else
{
      return -1;
}
```

ACCOUNT (3)

```
/* Usage */
void account_list_delta_prior_percent_set(
   LIST *prior_account_list in/out */,
   LIST *current_account_list );

/* Process */
for ACCOUNT *current_account in
      current_account_list
{
      ACCOUNT *prior_account =
            ACCOUNT *account_seek(
                  current_account->account_name,
                  prior_account_list /* account_list */ );

      prior_account->delta_prior_percent =
            float_delta_prior_percent(
                  prior_account->
                        account_journal_latest->
                        balance,
                  current_account->
                        account_journal_latest->
                        balance );
}

/* Usage */
void account_percent_of_asset_set(
      LIST *subclassification_account_statement_list,
      double element_asset_sum );

/* Process */
for ACCOUNT *account in
      subclassifiication_account_statement_list
{
      int account->percent_of_asset =
            float_percent_of_total(
                  account->
                        account_journal_latest->
                        balance,
                  element_asset_sum );
}

/* Usage */
void account_percent_of_revenue_set(
      LIST *subclassification_account_statement_list,
      double element_revenue_sum );

/* Process */
for ACCOUNT *account in
      subclassification_account_statement_list
{
      int account->percent_of_revenue =
            float_percent_of_total(
                  account->
                        account_journal_latest->
                        balance,
                  element_revenue_sum );
}
```

ACCOUNT (4)

```
/* Usage */
double account_list_debit_sum(
      LIST *account_list,
      boolean element_accumulate_debit );

/* Process */
for ACCOUNT *account in account_list
{
      if ( !element_accumulate_debit
      && account_journal_latest->balance < 0.0 )
      {
            debit_sum -=
                  account_journal_latest->balance;
      }
      else
      if ( element_accumulate_debit
      && account_journal_latest->balance > 0.0 )
      {
            debit_sum +=
                  account_journal_latest->balance;
      }
}

/* Usage */
double account_list_credit_sum(
      LIST *account_list,
      boolean element_accumulate_debit );

/* Process */
for ACCOUNT *account in account_list
{
      if ( element_accumulate_debit
      && account_journal_latest->balance < 0.0 )
      {
            credit_sum -=
                  account_journal_latest->balance;
      }
      else
      if ( !element_accumulate_debit
      && account_journal_latest->balance > 0.0 )
      {
            credit_sum +=
                  account_journal_latest->balance;
      }
}

/* Usage */
double account_balance_sum(
      LIST *account_list );

/* Process */
for ACCOUNT *account in account_list
{
      balance_sum += account_journal_latest->balance;
}

/* Usage */
char *account_hard_coded_account_name(
      char *account_key,
      boolean warning_only,
      const char *calling_function_name );

/* Process */
ACCOUNT *account_key_seek(
      char *account_key,
      static LIST *account_list() );

/* Usage */
ACCOUNT *account_parse(
      char *input,
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
ACCOUNT *account_new( char *account_name );

ACCOUNT *account_calloc( void );

if ( fetch_subclassification )
{
      subclassification =
            subclassification_fetch(
                  account->subclassification_name,
                  fetch_element );
}

/* Usage */
double account_list_sum(
      LIST *account_statement_list );

/* Process */

/* Usage */
ACCOUNT *account_element_list_seek(
   char *account_name,
   LIST *element_statement_list );

/* Process */
for ELEMENT *element in element_statement_list
{
   ACCOUNT *account_subclassification_account_seek(
         account_name,
         element->subclassification_statement_list );
}

/* Usage */
ACCOUNT *account_subclassification_list_seek(
   char *account_name,
   LIST *subclassification_statement_list );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   ACCOUNT *account_seek(
         account_name,
         subclassification->account_statement_list );
}
```

ACCOUNT (5)

```
/* Usage */
void account_transaction_count_set(
   LIST *account_statement_list,
   char *transaction_begin_date_string(),
   char *transaction_date_time_closing() );

/* Process */
for ACCOUNT *account in account_statement_list
{
   if ( account->journal_latest->balance )
   {
         account->transaction_count =
               journal_transaction_count(
                     JOURNAL_TABLE,
                     account->account_name,
                     transaction_begin_date_string,
                     transaction_date_time_closing );
   }
}

/* Usage */
void account_list_action_string_set(
   LIST *account_statement_list,
   char *application_name,
   char *session_key,
   char *login_name,
   char *role_name,
   char *transaction_begin_date_string(),
   char *transaction_date_time_closing() );

/* Process */
for ACCOUNT *account in account_statement_list
{
   if ( account->transaction_count )
   {
         account->action_string =
               char *account_action_string(
                     char *application_name,
                     char *session_key,
                     char *login_name,
                     char *role_name,
                     char *transaction_begin_date_string,
                     char *transaction_date_time_closing,
                     char *account->account_name );
   }
}
```

```
/* Attributes */
char *account_name;
char *subclassification_name;
char *hard_coded_account_key;
int chart_account_number;
int annual_budget;
SUBCLASSIFICATION *subclassification;
ACCOUNT_JOURNAL *account_journal_latest;
int transaction_count;
double balance;
double payment_amount;
int percent_of_asset;
int percent_of_revenue;
int delta_prior_percent;
LIST *liability_journal_list;
double liability_due;
char *action_string;

/* Public */
char *account_name_format(
   char *account_name );

char *account_name_display(
   char *account_name );

ACCOUNT *account_seek(
   char *account_name,
   LIST *account_list );

char *account_depreciation_expense(
   char *ACCOUNT_DEPRECIATION_KEY );

char *account_accumulated_depreciation(
   char *ACCOUNT_ACCUMULATED_KEY );

char *account_sales_tax_payable(
   char *ACCOUNT_SALES_TAX_PAYABLE_KEY );

char *account_revenue(
   char *ACCOUNT_REVENUE_KEY );

char *account_receivable(
   char *ACCOUNT_RECEIVABLE_KEY );

char *account_payable(
   char *ACCOUNT_PAYABLE_KEY );
```

```
char *account_uncleared_checks(
   char *ACCOUNT_UNCLEARED_CHECKS_KEY );

char *account_shipping_revenue(
   char *ACCOUNT_SHIPPING_REVENUE_KEY );

char *account_cash(
   char *ACCOUNT_CASH_KEY );

char *account_gain(
   char *ACCOUNT_GAIN_KEY );

char *account_loss(
   char *ACCOUNT_LOSS_KEY );

char *account_fees_expense(
   char *ACCOUNT_FEES_EXPENSE_KEY );

char *account_sales_tax_expense(
   char *ACCOUNT_SALES_TAX_EXPENSE_KEY );

char *account_closing_entry(
   char *ACCOUNT_CLOSING_KEY );

char *account_drawing(
   char *ACCOUNT_DRAWING_KEY );

LIST *account_cash_name_list(
   char *ACCOUNT_TABLE,
   char *SUBCLASSIFICATION_CASH );

LIST *account_current_liability_name_list(
   char *ACCOUNT_TABLE,
   char *SUBCLASSIFICATION_CURRENT_LIABILITY,
   char *ACCOUNT_UNCLEARED_CHECKS );

LIST *account_receivable_name_list(
   char *ACCOUNT_TABLE,
   char *SUBCLASSIFICATION_RECEIVABLE );

ACCOUNT *account_getset(
   LIST *account_list,
   ACCOUNT *account );

boolean account_name_changed(
   char *preupdate_account_name );
```

JOURNAL (1)

```
/* Usage */
LIST *journal_system_list(
    char *journal_system_string(
        JOURNAL_SELECT,
        JOURNAL_TABLE,
        where ),
    boolean fetch_account,
    boolean fetch_subclassification,
    boolean fetch_element,
    boolean fetch_transaction );

/* Process */
FILE *journal_input_pipe( journal_system_string() );

for input in string_input( journal_input_pipe() )
{
    list_set(
        system_list,
        journal_parse(
            input,
            fetch_account,
            fetch_subclassification,
            fetch_element,
            fetch_transaction ) );
}

void pclose( journal_input_pipe() );

/* Usage */
JOURNAL *journal_parse(
    char *input,
    boolean fetch_account,
    boolean fetch_subclassification,
    boolean fetch_element,
    boolean fetch_transaction );

/* Process */
JOURNAL *journal =
    JOURNAL *journal_new(
        char *full_name,
        char *street_address,
        char *transaction_date_time,
        char *account_name );

JOURNAL *journal_calloc( void );

if ( fetch_transaction )
{
    TRANSACTION *transaction =
        transaction_fetch(
            journal->full_name,
            journal->street_address,
            journal->transaction_date_time,
            0 /* not fetch_journal_list */ );
}

if ( fetch_account )
{
    ACCOUNT *account =
        account_fetch(
            journal->account_name,
            fetch_subclassification,
            fetch_element );
}

/* Usage */
JOURNAL *journal_prior(
    char *transaction_date_time,
    char *account_name,
    boolean fetch_account,
    boolean fetch_subclassification,
    boolean fetch_element );

/* Process */
char *journal_max_prior_where(
    char *transaction_date_time,
    char *account_name );

char *journal_max_prior_transaction_date_time(
    char *journal_max_prior_where(),
    char *JOURNAL_TABLE );

if ( journal_max_prior_transaction_date_time() )
{
    transaction_date_time =
        journal_max_prior_transaction_date_time();
}

JOURNAL *journal =
    journal_account_fetch(
        transaction_date_time,
        account_name,
        fetch_account,
        fetch_subclassification,
        fetch_element,
        0 /* not fetch_transaction */ );

if (  journal_max_prior_transaction_date_time() ==
    transaction_date_time )
{
    journal->previous_balance = 0.0;
}
```

TRANSACTION

ACCOUNT

```
SUBCLASSIFICATION *subclassification;
```

JOURNAL (2)

```
/* Usage */
JOURNAL *journal_latest(
	char *account_name,
	char *transaction_date_time_closing(),
	boolean fetch_transaction );

/* Process */
char *journal_less_equal_where(
	char *transaction_date_time_closing,
	char *account_name );

char *journal_max_transaction_date_time(
	char *journal_less_equal_where(),
	char *JOURNAL_TABLE );

JOURNAL *journal =
	journal_account_fetch(
		journal_max_transaction_date_time()
			/* transaction_date_time */,
		account_name,
		0 /* not fetch_account */,
		0 /* not fetch_subclassification */,
		0 /* not fetch_element */,
		fetch_transaction );

/* Usage */
LIST *journal_year_list(
	int tax_year,
	char *tax_form_line_account->account_name,
	boolean fetch_transaction );

/* Process */
char *journal_year_where(
	int tax_year,
	char *account_name,
	char *TRANSACTION_PRECLOSE_TIME );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_year_where() ),
	0 /* not fetch_account */,
	0 /* not fetch_subclassification */,
	0 /* not fetch_element */,
	fetch_transaction );

/* Usage */
LIST *journal_minimum_list(
	char *minimum_transaction_date_time,
	char *account_name,
	double journal_prior_previous_balance );

/* Process */
char *journal_greater_equal_where(
	char *minimum_transaction_date_time,
	char *account_name );

LIST *journal_list =
	journal_system_list(
		journal_system_string(
			JOURNAL_SELECT,
			JOURNAL_TABLE,
			journal_greater_equal_where() ),
		0 /* not fetch_account */,
		0 /* not fetch_subclassification */,
		0 /* not fetch_element */,
		0 /* not fetch_transaction */ );

JOURNAL *first_journal = list_first( journal_list );

first_journal->previous_balance =
	journal_prior_previous_balance;

/* Usage */
JOURNAL *journal_account_fetch(
	char *transaction_date_time,
	char *account_name,
	boolean fetch_account,
	boolean fetch_subclassification,
	boolean fetch_element,
	boolean fetch_transaction );

/* Process */
char *journal_transaction_account_where(
	char *transaction_date_time,
	char *account_name );

JOURNAL *journal_parse(
	string_pipe(
		journal_system_string(
			JOURNAL_SELECT,
			JOURNAL_TABLE,
			journal_transaction_account_where() ),
		fetch_account,
		fetch_subclassification,
		fetch_element,
		fetch_transaction );

/* Usage */
void journal_account_list_propagate(
	char *transaction_date_time,
	LIST *journal_extract_account_list() );

/* Process */
for ACCOUNT *account in journal_extract_account_list
{
	JOURNAL *journal_prior(
		transaction_date_time,
		account->account_name,
		0 /* not fetch_account */,
		0 /* not fetch_subclassification */,
		0 /* not fetch_element */ );

	LIST *journal_list_prior(
		journal_prior(),
		account->account_name );

	LIST *journal_list_balance_set(
		LIST *journal_list_prior() /* in/out */,
		account->
			subclassification->
			element->
			accumulate_debit );

	void journal_list_update(
		journal_list_balance_set() );
}

/* Usage */
void journal_account_list_getset(
	LIST *account_list /* in/out */,
	LIST *journal_list );

/* Process */
for JOURNAL *journal in journal_list
{
	(void)account_getset(
		account_list /* in/out */,
		journal->account );
}
```

JOURNAL (3)

```
/* Usage */
void journal_propagate(
      char *transaction_date_time,
      char *account_name );

/* Process */
JOURNAL *journal_prior(
      transaction_date_time,
      account_name,
      1 /* fetch_subclassification */
      1 /* fetch_element */,
      1 /* fetch_account */ );

LIST *journal_list_prior(
      journal_prior(),
      account_name );

LIST *journal_list_balance_set(
      LIST *journal_list_prior() /* in/out */,
      boolean journal_prior()->
            account->
            subclassification->
            element->
            accumulate_debit
                  /* element_accumulate_debit */ );

void journal_list_update( journal_list_balance_set() );

/* Usage */
void journal_list_update(
      LIST *journal_list_balance_set() );

/* Process */
char *journal_update_system_string(
      char *JOURNAL_TABLE );

FILE *journal_update_pipe(
      char *journal_update_system_string() );

for JOURNAL *journal in journal_list_balance_set
{
      fprintf(   journal_update_pipe(),
                 journal->previous_balance );
      fprintf(   journal_update_pipe(),
                 journal->balance );
}

void pclose( journal_update_pipe() );

/* Usage */
LIST *journal_list_prior(
   JOURNAL *journal_prior(),
   char *account_name );

/* Process */
LIST *journal_minimum_list(
   journal_prior->transaction_date_time
         /* minimum_transaction_date_time */,
   account_name,
   journal_prior->previous_balance );

/* Usage */
void journal_list_insert(
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   LIST *journal_list );

/* Process */
FILE *journal_insert_pipe(
   char *JOURNAL_INSERT,
   char *JOURNAL_TABLE );

void journal_insert(
   FILE *journal_insert_pipe(),
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   char *account_name,
   double debit_amount,
   double credit_amount );

void pclose( journal_insert_pipe() );

/* Usage */
void journal_list_transaction_insert(
   FILE *journal_insert_pipe(),
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   LIST *journal_list );

/* Process */
for JOURNAL *journal in journal_list
{
   void journal_insert(
         journal_insert_pipe,
         full_name,
         street_address,
         transaction_date_time,
         journal->account_name,
         journal->debit_amount,
         journal->credit_amount );
}

/* Usage */
void journal_list_html_display(
   LIST *journal_list,
   char *transaction_date_time,
   char *transaction_memo );

/* Process */
void journal_list_pipe_display(
   FILE *output_pipe,
   char *transaction_date_time,
   char *transaction_memo,
   char *heading,
   LIST *journal_list );

/* Usage */
void journal_list_sum_html_display(
   LIST *journal_list,
   char *transaction_date_time,
   char *transaction_memo,
   double debit_sum,
   double credit_sum );

/* Process */
void journal_list_pipe_display(
   output_pipe,
   transaction_date_time,
   transaction_memo,
   heading,
   journal_list );
```

JOURNAL (4)

```
/* Usage */
double journal_list_transaction_amount(
   LIST *journal_list );

/* Process */
static LIST *account_cash_name_list(
      ACCOUNT_TABLE,
      SUBCLASSIFICATION_CASH );

static LIST *account_current_liability_name_list(
      ACCOUNT_TABLE,
      SUBCLASSIFICATION_CURRENT_LIABILITY,
      ACCOUNT_UNCLEARED_CHECKS );

double cash_sum =
   journal_account_list_debit_sum(
         journal_list,
         account_cash_name_list() );

double current_liability_sum =
   journal_account_list_credit_sum(
         journal_list,
         account_current_liability_name_list() );

if ( cash_sum )
{
   return cash_sum;
}
else
if ( current_liability_sum )
{
   return current_liability_sum;
}
else
{
   return journal_debit_sum( journal_list );
}

/* Usage */
double journal_account_list_debit_sum(
   LIST *journal_list,
   LIST *account_name_list );

/* Process */
for JOURNAL *journal in journal_list
{
   if ( list_string_exists(
         journal->account_name,
         account_name_list ) )
   {
         if ( journal->debit_amount )
               sum += journal->debit_amount;
         else
               sum -= journal->credit_amount;
   }
}

/* Usage */
double journal_account_list_credit_sum(
   LIST *journal_list,
   LIST *account_name_list );

/* Process */
for JOURNAL *journal in journal_list
{
   if ( list_string_exists(
         journal->account_name,
         account_name_list ) )
   {
         if ( journal->credit_amount )
               sum += journal->credit_amount;
         else
               sum -= journal->debit_amount;
   }
}

/* Usage */
double journal_prior_account_end_balance(
      char *feeder_load_file_minimum_date(),
      char *account_name );

/* Process */
if ( journal_prior(
         feeder_load_file_minimum_date,
         account_name,
         1 /* fetch_account */,
         1 /* fetch_subclassification */
         1 /* fetch_element */ ) )
{
   if (  journal_prior()->
         account->
         subclassification->
         element->
         accumulated_debit )
   {
         end_balance = journal_prior()->balance;
   }
   else
   {
         end_balance = -journal_prior()->balance;
   }
}
else
{
   end_balance = 0.0;
}

/* Usage */
double journal_balance_sum(
   LIST *journal_list );

/* Process */
for JOURNAL *journal in journal_list
{
         sum += journal->balance;
}

/* Usage */
LIST *journal_tax_form_list(
      char *tax_form_fiscal_begin_date(),
      char *tax_form_fiscal_end_date(),
      char *TRANSACTION_PRECLOSE_TIME,
      char *account_name );

/* Process */
char *journal_tax_form_where(
      char *tax_form_fiscal_begin_date(),
      char *tax_form_fiscal_end_date(),
      char *TRANSACTION_PRECLOSE_TIME,
      char *account_name );

char *journal_system_string(
      JOURNAL_SELECT,
      JOURNAL_TABLE,
      journal_tax_form_where() );

LIST *journal_system_list(
      journal_system_string(),
      0 /* not fetch_account */,
      0 /* not fetch_subclassification */,
      0 /* not fetch_element */,
      0 /* not fetch_transaction */ );
```

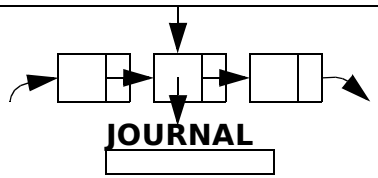
JOURNAL (5)

```
/* Usage */
double journal_debit_sum( LIST *journal_list );

/* Process */
for JOURNAL *Journal in journal_list
{
   if ( journal->debit_amount )
         sum += journal->debit_amount;
}

/* Usage */
double journal_credit_sum( LIST *journal_list );

/* Process */
for JOURNAL *journal in journal_list
{
      if ( journal->credit_amount )
            sum += journal->credit_amount;
}

/* Usage */
JOURNAL *journal_debit_new(
      char *debit_account_name,
      double debit_amount );

/* Process */
JOURNAL *journal_calloc();

journal->account =
      ACCOUNT *account_fetch(
            debit_account_name,
            1 /* fetch_subclassification */,
            1 /* fetch_element */ );

/* Usage */
JOURNAL *journal_credit_new(
      char *credit_account_name,
      double credit_amount );

/* Process */
JOURNAL *journal_calloc();

journal->account =
      ACCOUNT *account_fetch(
            credit_account_name,
            1 /* fetch_subclassification */,
            1 /* fetch_element */ );

/* Public */
char *journal_primary_where(
      char *full_name,
      char *street_address,
      char *transaction_date_time,
      char *account_name );

char *journal_system_string(
      char *JOURNAL_SELECT,
      char *JOURNAL_TABLE,
      char *where );

double journal_credit_debit_difference_sum(
   LIST *journal_list );

double journal_debit_credit_difference_sum(
   LIST *journal_list );

LIST *journal_account_distinct_entity_list(
   char *JOURNAL_TABLE,
   LIST *account_name_list );

LIST *journal_extract_account_list(
   LIST *journal_list );

char *journal_delete_system_string(
   char *JOURNAL_TABLE,
   char *transaction_primary_where()
         /* where */ );

LIST *journal_binary_list(
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   double float_abs( transaction_amount ),
   ACCOUNT *debit_account,
   ACCOUNT *credit_account );

int journal_transaction_count(
   char *JOURNAL_TABLE,
   char *account->account_name,
   char *transaction_begin_date_string,
   char *transaction_date_time_closing );

LIST *journal_date_time_account_name_list(
   char *JOURNAL_TABLE,
   char *transaction_date_time );

JOURNAL *journal_seek(
   char *transaction_date_time,
   char *feeder_account,
   char *journal_system_list() );

char *journal_minimum_transaction_date_time(
   char *account_name );

double journal_balance(
   double balance,
   boolean journal->
         account->
         subclassification->
         element->
         accumulate_debit );

LIST *journal_account_name_list(
   char *full_name,
   char *street_address,
   char *transaction_date_time );

double journal_amount(
   double debit_amount,
   double credit_amount,
   boolean journal->
         account->
         subclassification->
         element->
         accumulate_debit );
```

JOURNAL_PROPERTY

TRANSACTION (1)

```
/* Usage */
LIST *transaction_list(
	char *where,
	boolean fetch_journal_ledger );

/* Process */
char *transaction_system_string(
	TRANSACTION_SELECT,
	TRANSACTION_TABLE,
	where );

FILE *transaction_input_pipe(
	char *transaction_system_string() );

for input[] in string_input( transaction_input_pipe() )
{
	list_set(
		list,
		transaction_parse(
			input,
			fetch_journal_list ) );
}

void pclose( transaction_input_pipe() );

/* Usage */
TRANSACTION *transaction_fetch(
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	boolean fetch_journal_list );

/* Process */
char *transaction_primary_where(
	full_name,
	street_address,
	transaction_date_time );

char *transaction_system_string(
	TRANSACTION_SELECT,
	TRANSACTION_TABLE,
	transaction_primary_where()
		/* where */ );

TRANSACTION *transaction_parse(
	string_pipe_fetch(
		transaction_system_string() )
			/* input */,
	fetch_journal_list );

/* Usage */
TRANSACTION *transaction_parse(
	char *input,
	boolean fetch_journal_list );

/* Process */
TRANSACTION *transaction_new(
	char *full_name,
	char *street_address,
	char *transaction_date_time );

TRANSACTION *transaction_calloc( void );

if ( fetch_journal_list )
{
	LIST *journal_list =
		journal_system_list(
			journal_system_string(
				JOURNAL_SELECT,
				JOURNAL_TABLE,
				transaction_primary_where() ),
			1 /* fetch_account */,
			1 /* fetch_subclassification */,
			1 /* fetch_element */,
			0 /* not fetch_transaction */ );
}

/* Usage
TRANSACTION *transaction_entity_new(
	ENTITY *entity,
	char *transaction_date_time,
	double transaction_amount,
	int check_number,
	char *memo,
	LIST *journal_list );

/* Process */
TRANSACTION *transaction_calloc();

/* Usage */
TRANSACTION *transaction_binary(
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	double transaction_amount,
	char *memo,
	char *debit_account_name,
	char *credit_account_name );

/* Process */
TRANSACTION *transaction_new(
	full_name,
	street_address,
	transaction_date_time );

LIST *journal_list =
	journal_binary_list(
		full_name,
		street_address,
		transaction_date_time,
		float_abs( transaction_amount ),
		account_fetch(
			debit_account_name,
			1 /* fetch_subclassification */,
			1 /* fetch_element */ ),
		account_fetch(
			credit_account_name,
			1 /* fetch_subclassification */,
			1 /* fetch_element */ ) );
```



TRANSACTION (2)

```
/* Usage */
char *transaction_list_insert(
      LIST *transaction_list,
      boolean insert_journal_list_boolean );

/* Process */
for TRANSACTION *transaction in transaction_list
{
      transaction->transaction_date_time =
            transaction_insert(
                  transaction->full_name,
                  transaction->street_address,
                  transaction->transaction_date_time,
                  transaction->transaction_amount,
                  transaction->check_number,
                  transaction->memo,
                  TRANSACTION_LOCK_Y,
                  transaction->journal_list,
                  insert_journal_list_boolean );

      if ( !first_transaction_date_time )
      {
            first_transaction_date_time =
                  transaction->transaction_date_time;
      }
}

return first_transaction_date_time;

/* Usage */
/* Returns inserted transaction_date_time */
char *transaction_insert(
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   double transaction_amount,
   int check_number,
   char *memo,
   char lock_transaction_yn,
   LIST *journal_list,
   boolean insert_journal_list );

/* Process */
FILE *transaction_insert_pipe_open(
   char *TRANSACTION_SELECT,
   char *TRANSACTION_TABLE );

char *transaction_race_free_date_time(
         char *transaction_date_time );

void transaction_insert_pipe(
   FILE *transaction_insert_pipe_open(),
   char *full_name,
   char *street_address,
   char *transaction_race_free_date_time(),
   double transaction_amount,
   char *transaction_check_number(
         int check_number ),
   char *transaction_memo(
         char *memo ),
   char *transaction_lock(
         char transaction_lock_yn ) );

pclose( transaction_insert_pipe_open() );

if ( insert_journal_list_boolean )
{
   void journal_list_insert(
         full_name,
         street_address,
         race_free_date_time,
         journal_list );

   LIST *journal_extract_account_list( journal_list );

   void journal_account_list_propagate(
         transaction_race_free_date_time(),
         journal_extract_account_list() );
}

/* Usage */
char *transaction_stamp_insert(
   TRANSACTION *transaction,
   boolean insert_journal_list_boolean );

/* Process */
char *transaction_date_time =
      char *transaction_insert(
            transaction->full_name,
            transaction->street_address,
            transaction->transaction_date_time,
            transaction->transaction_amount,
            transaction->check_number,
            transaction->memo,
            TRANSACTION_LOCK_Y,
            transaction->journal_list,
            insert_journal_list_boolean );

/* Usage */
void transaction_delete(
   full_name,
   street_address,
   transaction_date_time );

/* Process */
TRANSACTION *transaction =
      transaction_fetch(
            char *full_name,
            char *street_address,
            char *transaction_date_time,
            1 /*  fetch_journal_list */ );

char *transaction_primary_where(
   full_name,
   street_address,
   transaction_date_time );

char *transaction_delete_system_string(
   char *TRANSACTION_TABLE,
   char *transaction_primary_where() );

system( transaction_delete_system_string() );

char *journal_delete_system_string(
   JOURNAL_TABLE,
   transaction_primary_where(
         full_name,
         street_address,
         transaction_date_time ) );

system( journal_delete_system_string() );

void journal_account_list_propagate(
   transaction_date_time,
   journal_extract_account_list(
         transaction->journal_list ) );
```

TRANSACTION (3)

```
/* Usage */
boolean transaction_closing_entry_exists(
   char *TRANSACTION_TABLE,
   char *TRANSACTION_CLOSE_TIME,
   char *transaction_as_of_date );

/* Process */
char *transaction_date_time_where(
   TRANSACTION_DATE_TIME_COLUMN,
   transaction_date,
   TRANSACTION_CLOSE_TIME );

char *transaction_closing_memo_where(
   char *TRANSACTION_CLOSING_ENTRY_MEMO );

char *transaction_closing_entry_where(
   char *transaction_date_time_where(),
   char *transaction_closing_memo_where() );

char *transaction_closing_entry_system_string(
   char *TRANSACTION_TABLE,
   char *transaction_closing_entry_where() );

( atoi(
   pipe2string(
         transaction_closing_entry_system_string() )
 == 1 );

/* Usage */
void transaction_fetch_update(
   char *full_name,
   char *street_address,
   char *transaction_date_time );

/* Process */
TRANSACTION *transaction =
   transaction_fetch(
         full_name,
         street_address,
         transaction_date_time,
         1 /* fetch_journal_list */ );

double journal_list_transaction_amount(
   transaction->journal_list );

char *transaction_primary_where(
   full_name,
   street_address,
   transaction_date_time );

void transaction_update(
   journal_list_transaction_amount(),
   transaction_primary_where() );

/* Usage */
void transaction_update(
   double journal_list_transaction_amount(),
   char *transaction_primary_where() );

/* Process */
char *transaction_set_clause(
   char *TRANSACTION_AMOUNT_COLUMN,
   double journal_list_transaction_amount );

char *transaction_update_statement_system_string(
   char *TRANSACTION_TABLE,
   transaction_set_clause(),
   transaction_primary_where );

system(
   transaction_update_statement_system_string() );

/* Usage */
char *transaction_date_time_where(
   char *TRANSACTION_DATE_TIME_COLUMN,
   char *transaction_date,
   char *transaction_time );

/* Process */
char *transaction_date_time(
   char *transaction_date,
   char *transaction_time );

/* Usage */
char *transaction_as_of_date(
   char *TRANSACTION_TABLE,
   char *as_of_date_string );

/* Process */
boolean transaction_as_of_date_populated(
   char *as_of_date_string );

if ( transaction_as_of_date_populated() )
{
   return as_of_date_string;
}
else
{
   return
   char *transaction_date_max(
         TRANSACTION_TABLE );
}

/* Usage */
char *transaction_begin_date_string(
   char *TRANSACTION_TABLE,
   char *transaction_as_of_date() );

/* Process */
DATE *transaction_prior_closing_transaction_date(
   TRANSACTION_CLOSE_TIME,
   TRANSACTION_CLOSING_ENTRY_MEMO,
   TRANSACTION_TABLE,
   transaction_as_of_date );

if ( transaction_prior_closing_transaction_date() )
{
   date_increment_days(
         transaction_prior_closing_transaction_date(),
         1.0 );

   return
   date_yyyy_mm_dd(
         transaction_prior_closing_transaction_date() );
}
else
{
   return
   transaction_minimum_transaction_date_string(
         TRANSACTION_TABLE );
}
```

TRANSACTION (4)

```
/* Usage */
TRANSACTION *transaction_date_time_fetch(
      char *transaction_date_time,
      boolean fetch_journal_list );

/* Process */
char *transaction_date_time_fetch_where(
      transaction_date_time );

char *transaction_system_string(
      TRANSACTION_SELECT,
      TRANSACTION_TABLE,
      transaction_date_time_fetch_where()
            /* where */ );

TRANSACTION *transaction_parse(
      string_pipe_fetch(
            transaction_system_string() )
                  /* input */,
      fetch_journal_list );

/* Usage */
char *transaction_date_max(
   char *TRANSACTION_TABLE );

/* Process */
char *transaction_date_time_max(
   char *TRANSACTION_TABLE );

/* Usage */
DATE *transaction_prior_closing_transaction_date(
   char *TRANSACTION_CLOSE_TIME,
   char *TRANSACTION_CLOSING_ENTRY_MEMO,
   char *TRANSACTION_TABLE,
   char *transaction_as_of_date() );

/* Process */
```

```
/* Usage */
void transaction_journal_list_insert(
   LIST *transaction_list,
   boolean with_propagate  );

/* Process */
FILE *journal_insert_pipe(
   JOURNAL_INSERT,
   JOURNAL_TABLE );

for TRANSACTION *transaction in transaction_list
{
   if ( !first_transaction_date_time )
   {
         first_transaction_date_time =
               transaction->transaction_date_time;
   }

   void journal_list_transaction_insert(
         journal_insert_pipe(),
         transaction->full_name,
         transaction->street_address,
         transaction->transaction_date_time,
         transaction->journal_list );
}

pclose( journal_insert_pipe() );

if ( with_propagate )
{
   LIST *transaction_list_extract_account_list(
         transaction_list );

   void journal_account_list_propagate(
         first_transaction_date_time,
         transaction_list_extract_account_list() );
```

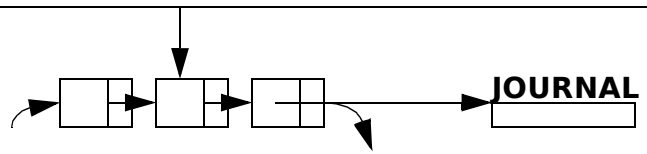
```
}

/* Usage */
LIST *transaction_list_extract_account_list(
   LIST *transaction_list );

/* Process */
LIST *account_list = list_new();

for TRANSACTION *transaction in transaction_list
{
   void journal_account_list_getset(
         account_list /* in/out */,
         transaction->journal_list );
}

/* Usage */
void transaction_list_html_display(
   LIST *transaction_list );

/* Process */
for TRANSACTION *transaction in transaction_list
{
   void transaction_html_display( transaction );
}

/* Usage */
void transaction_html_display(
   TRANSACTION *transaction );

/* Process */
void journal_list_html_display(
   transaction->journal_list,
   transaction->transaction_date_time,
   transaction->transaction_memo );
```

TRANSACTION (5)

```
/* Usage */
char *transaction_refresh(
   char *full_name,
   char *street_address,
   char *transaction_date_time,
   double transaction_amount,
   int check_number,
   char *memo,
   char lock_transaction_yn,
   LIST *journal_list );

/* Process */
void transaction_delete(
   full_name,
   street_address,
   transaction_date_time );

char *transaction_date_time =
   char *transaction_insert(
         full_name,
         street_address,
         transaction_date_time,
         transaction_amount,
         check_number,
         memo,
         lock_transaction_yn,
         journal_list,
         1 /* insert_journal_list */ );

/* Attributes */
char *full_name;
char *street_address;
char *transaction_date_time;
double transaction_amount;
char *memo;
int check_number;
boolean lock_transaction;
LIST *journal_list;

/* Public */
char *transaction_primary_where(
   char *full_name,
   char *street_address,
   char *transaction_date_time );

char *transaction_system_string(
   char *TRANSACTION_SELECT,
   char *TRANSACTION_TABLE,
   char *where );

char *transaction_date_time_closing(
   char *TRANSACTION_PRECLOSE_TIME,
   char *TRANSACTION_CLOSE_TIME,
   char *transaction_as_of_date(),
   boolean preclose_time_boolean );

boolean transaction_date_time_changed(
   char *preupdate_transaction_date_time );

char *transaction_date_time_earlier(
   char *transaction_date_time,
   char *preupdate_transaction_date_time );

boolean transaction_date_time_exists(
   char *TRANSACTION_TABLE,
   char *TRANSACTION_DATE_TIME_COLUMN,
   char *transaction_date_time );

char *transaction_minimum_transaction_date_string(
   char *TRANSACTION_TABLE );

/* Increments second each invocation. */
char *transaction_increment_date_time(
   char *transaction_date );
```

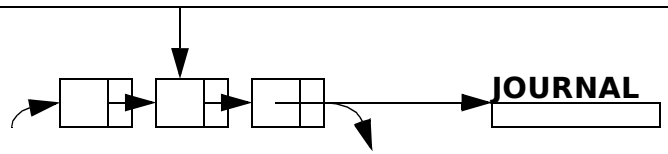
TRANSACTION_PROGRAM

```
/* Usage */
TRANSACTION *transaction_program_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *program_name,
        boolean fetch_journal_ledger );

/* Process */
```



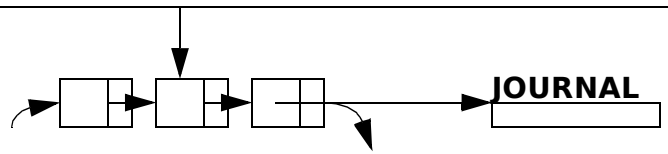
TRANSACTION_FUND

```
/* Usage */
TRANSACTION *transaction_program_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *fund_name,
        boolean fetch_journal_ledger );

/* Process */
```



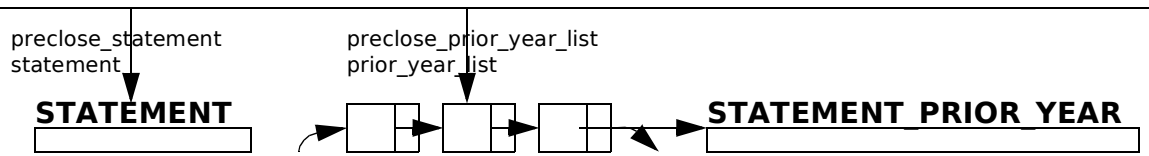
TRANSACTION_PROPERTY

```
/* Usage */
TRANSACTION *transaction_property_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *property_street_address,
        boolean fetch_journal_ledger );

/* Process */
```



TRIAL_BALANCE (1)

```
/* Usage */
TRIAL_BALANCE *trial_balance_fetch(
   char *application_name,
   char *session_key,
   char *login_name,
   char *role_name,
   char *process_name,
   char *document_root_directory,
   char *as_of_date_string,
   int prior_year_count,
   char *subclassification_option_string,
   char *output_medium_string );

/* Process */
enum statement_subclassification_option =
   statement_resolve_subclassification_option(
         subclassification_option_string );

enum statement_output_medium =
   statement_resolve_output_medium(
         output_medium_string );

char *transaction_as_of_date(
   TRANSACTION_TABLE,
   as_of_date_string );

if ( !transaction_as_of_date() )
{
   return (TRIAL_BALANCE *)0;
}

char *transaction_begin_date_string(
   TRANSACTION_TABLE,
   transaction_as_of_date() );

if ( !transaction_begin_date_string() )
{
   exit( 1 );
}

LIST *trial_balance_element_name_list(
   char *ELEMENT_ASSET,
   char *ELEMENT_LIABILITY,
   char *ELEMENT_REVENUE,
   char *ELEMENT_EXPENSE,
   char *ELEMENT_GAIN,
   char *ELEMENT_LOSS,
   char *ELEMENT_EQUITY );

boolean transaction_closing_entry_exists(
   TRANSACTION_TABLE,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date() );

if ( transaction_closing_entry_exists() )
{
   char *transaction_date_time_closing(
         TRANSACTION_PRECLOSE_TIME,
         TRANSACTION_CLOSE_TIME,
         transaction_as_of_date(),
         1 /* preclose_time_boolean */ );

   char *trial_balance_preclose_process_name(
         char *process_name );

   STATEMENT *preclose_statement =
         statement_fetch(
               application_name,
               trial_balance_preclose_process_name(),
               trial_balance_element_name_list(),
               transaction_begin_date_string(),
               transaction_as_of_date(),
               transaction_date_time_closing(),
               1 /* fetch_transaction */ );

   void element_account_transaction_count_set(
         preclose_statement->element_statement_list,
         transaction_begin_date_string(),
         transaction_date_time_closing() );

   if ( statement_output_medium == output_table )
   {
         void element_account_action_string_set(
               preclose_statement->
                     element_statement_list,
                     application_name,
                     session_key,
                     login_name,
                     role_name,
                     transaction_begin_date_string(),
                     transaction_date_time_closing() );
   }

   if ( prior_year_count )
   {
         LIST *preclose_statement_prior_year_list =
               LIST *statement_prior_year_list(
                     trial_balance_element_name_list(),
                     transaction_date_time_closing(),
                     prior_year_count,
                     preclose_statement );
   }
}

char *transaction_date_time_closing(
   TRANSACTION_PRECLOSE_TIME,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date(),
   0 /* not preclose_time_boolean */ );

STATEMENT *statement =
   statement_fetch(
         application_name,
         process_name,
         trial_balance_element_name_list(),
         transaction_begin_date_string(),
         transaction_as_of_date(),
         transaction_date_time_closing(),
         1 /* fetch_transaction */ );

void element_account_transaction_count_set(
   statement->element_statement_list,
   transaction_begin_date_string(),
   transaction_date_time_closing() );
```



TRIAL_BALANCE (2)

```
/* Process (continued) */
if ( statement_output_medium ==
        statement_output_table )
{
   void element_account_action_string_set(
        statement->element_statement_list,
        application_name,
        session_key,
        login_name,
        role_name,
        transaction_begin_date_string(),
        transaction_date_time_closing() );
}

if ( prior_year_count )
{
   LIST *statement_prior_year_list(
        trial_balance_element_name_list(),
        transaction_date_time_closing(),
        prior_year_count,
        statement );
}

if ( transaction_closing_entry_exists() )
{
   double preclose_debit_sum =
        double element_list_debit_sum(
             preclose_statement->
                  element_statement_list );

   double preclose_credit_sum =
        double element_list_credit_sum(
             preclose_statement->
                  element_statement_list );
}

double debit_sum =
   double element_list_debit_sum(
        statement->
             element_statement_list );

double credit_sum =
   double element_list_credit_sum(
        statement->
             element_statement_list );

if ( transactionclosing_entry_exists()
&& statement_subclassification_option ==
        subclassification_omit )
{
   element_list_account_statement_list_set(
        preclose_statement->
             element_statement_list );
}

if (statement_subclassification_option ==
   subclassification_omit )
{
   element_list_account_statement_list_set(
        statement->
             element_statement_list );
}

if ( statement_output_medium ==
        statement_output_PDF )
{
   TRIAL_BALANCE_PDF *trial_balance_pdf =
        trial_balance_pdf_new(
             application_name,
             process_name,
             document_root_directory,
             statement_subclassification_option,
             preclose_statement,
             preclose_statement_prior_year_list,
             preclose_debit_sum,
             preclose_credit_sum,
             statement,
             statement_prior_year_list,
             debit_sum,
             credit_sum,
             getpid() /* process_id */ );
}
else
if ( statement_output_medium ==
        statement_output_table )
{
   TRIAL_BALANCE_TABLE *trial_balance_table =
        trial_balance_table_new(
             statement_subclassification_option,
             preclose_statement,
             preclose_statement_prior_year_list,
             preclose_debit_sum,
             preclose_credit_sum,
             statement,
             statement_prior_year_list,
             debit_sum,
             credit_sum );
}

/* Public */
char *trial_balance_transaction_count_string(
   int transaction_count );
```

TRIAL_BALANCE_PDF **TRIAL_BALANCE_TABLE**

TRIAL_BALANCE (3)

```
/* Driver */
printf( “%s\n”,
        statement->
              statement_caption->
              frame_title );

if ( trial_balance_pdf )
{
   if ( trial_balance_pdf->
        preclose_trial_balance_latex )
   {
        void statement_latex_output(
              trial_balance_pdf->
                    preclose_trial_balance_latex->
                    latex,
              trial_balance_pdf->
                    preclose_statement_link->
                    ftp_output_filename,
              TRIAL_BALANCE_PRECLOSE_PROMPT,
              process_name,
              statement->
                    statement_caption->
                    date_time_string );
   }

   void statement_latex_output(
        trial_balance_pdf->
              trial_balance_latex->
              latex,
        trial_balance_pdf->
              statement_link->
              ftp_output_filename,
        TRIAL_BALANCE_PROMPT,
        process_name,
        statement->
              statement_caption->
              date_time_string );
}
else
if ( trial_balance_table )
{
   if ( trial_balance_table->
        preclose_trial_balance_html )
   {
        void statement_html_output(
              trial_balance_table->
                    preclose_trial_balance_html->
                    html_table,
              TRIAL_BALANCE_PRECLOSE_TITLE
                    /* secondary_title */ );
   }

   void statement_html_output(
        trial_balance_table->
              trial_balance_html->
              html_table,
        (char *)0 /* secondary_title */ );
}
```

TRIAL_BALANCE_TABLE

```
/* Usage */
TRIAL_BALANCE_TABLE *trial_balance_table_new(
   enum statement_subclassification_option
         statement_subclassification_option,
   STATEMENT *preclose_statement,
   LIST *preclose_statement_prior_year_list,
   double preclose_debit_sum,
   double preclose_credit_sum,
   STATEMENT *statement,
   LIST *statement_prior_year_list,
   double debit_sum,
   double credit_sum );
```

```
/* Process */
TRIAL_BALANCE_TABLE *trial_balance_table_calloc(
   void );

if ( preclose_statement )
{
   TRIAL_BALANCE_HTML *
         preclose_trial_balance_html =
               trial_balance_html_new(
                     statement_subclassification_option,
                     preclose_statement,
                     preclose_statement_prior_year_list,
                     preclose_debit_sum,
                     preclose_credit_sum );
}

TRIAL_BALANCE_HTML *trial_balance_html =
   trial_balance_html_new(
         statement_subclassification_option,
         statement,
         statement_prior_year_list,
         debit_sum,
         credit_sum );
```

preclose_trial_balance_html
trial_balance_html

TRIAL_BALANCE_HTML

TRIAL_BALANCE_HTML

```
/* Usage */
TRIAL_BALANCE_HTML *trial_balance_html_new(
   enum statement_subclassification_option
         statement_subclassification_option,
   STATEMENT *statement,
   LIST *statement_prior_year_list,
   double debit_sum,
   double credit_sum );

/* Process */
TRIAL_BALANCE_HTML *trial_balance_html_calloc(
   void );

if (statement_subclassification_option ==
   subclassification_display )
{
   TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML *
         trial_balance_subclass_display_html =
               trial_balance_subclass_display_html_new(
                     statement,
                     statement_prior_year_list,
                     debit_sum,
                     credit_sum );

   HTML_TABLE *trial_balance_html_table(
         statement->caption_subtitle,
         trial_balance_subclass_display_html->
               heading_list,
         trial_balance_subclass_display_html->
               row_list );
}
else
{
   TRIAL_BALANCE_SUBCLASS_OMIT_HTML *
         trial_balance_subclass_omit_html =
               trial_balance_subclass_omit_html_new(
                     statement,
                     statement_prior_year_list,
                     debit_sum,
                     credit_sum );

   HTML_TABLE *trial_balance_html_table(
         statement->caption_subtitle,
         trial_balance_subclass_omit_html->
               heading_list,
         trial_balance_subclass_omit_html->
               row_list );
}

/* Usage */
HTML_TABLE *trial_balance_html_table(
   char *statement_caption_subtitle,
   LIST *heading_list,
   LIST *row_list );

/* Process */
HTML_TABLE *html_table =
   html_table_new(
         (char *)0 /* title */,
         statement_caption_subtitle,
         (char *)0 /* sub_sub_title */ );

html_table->heading_list = heading_list;
html_table->row_list = row_list;

/* Public */
char *trial_balance_html_account_title(
   char *account->account_name,
   char *account->
         account_journal_latest->
         full_name,
   double account->
         account_journal_latest->
         debit_amount,
   double account->
         account_journal_latest->
         credit_amount,
   char *statement_date_american(
         account->
               account_journal_latest->
               transaction_date_time )
                     /* transaction_date_american */,
   char *account->
         account_journal_latest->
         transaction->
         memo );
```

TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML

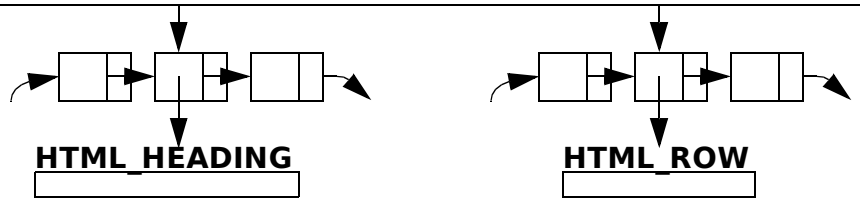
TRIAL_BALANCE_SUBCLASS_OMIT_HTML

HTML_TABLE

```
LIST *heading_list;
LIST *row_list;
```

TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML (1)

```
/* Usage */
TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML *
   trial_balance_subclass_display_html_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double debit_sum,
         double credit_sum );

/* Process */
TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML *
   trial_balance_subclass_display_html_calloc(
         void );

LIST *trial_balance_subclass_display_html_heading_list(
   statement_prior_year_list );

LIST *trial_balance_subclass_display_html_row_list(
   statement->transaction_as_of_date,
   statement->element_statement_list,
   statement_prior_year_list,
   debit_sum,
   credit_sum );

/* Usage */
LIST *trial_balance_subclass_display_html_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   html_heading_new(
         “Element”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Subclassification”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Account”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Count”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Debit”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Credit”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “percent_of_asset”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “percent_of_revenue”,
         1 /* right_justify_boolean */ ) );

void html_heading_list_right_justify_set(
   heading_list,
   LIST *statement_prior_year_heading_list(
         statement_prior_year_list )
               /* label_list */ );
```



TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML (2)

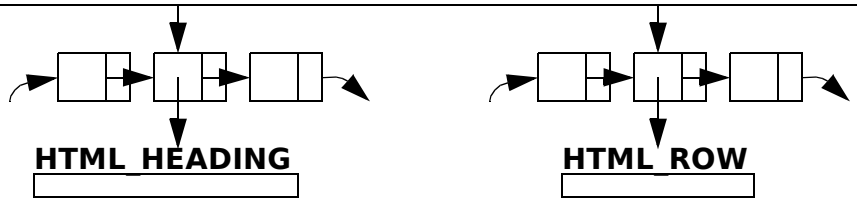
```
/* Usage */
LIST *trial_balance_subclass_display_html_row_list(
   char *statement->transaction_as_of_date,
   LIST *statement->element_statement_list,
   LIST *statement_prior_year_list(),
   double debit_sum,
   double credit_sum );

/* Process */
for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set_list(
               row_list,
               trial_balance_subclass_display_html_element_row_list(
                     transaction_as_of_date,
                     element->element_name,
                     element->accumulate_debit,
                     element->subclassification_statement_list,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   HTML_ROW *trial_balance_subclass_display_html_sum_row(
         double debit_sum,
         double credit_sum ) );

/* Usage */
LIST *trial_balance_subclass_display_html_element_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   LIST *subclassification_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   list_set_list(
         row_list,
               trial_balance_subclass_display_html_account_row_list(
                     transaction_as_of_date,
                     element_name,
                     element_accumulate_debit,
                     subclassification->subclassification_name,
                     subclassification->account_statement_list,
                     statement_prior_year_list ) );

   element_name = (char *)0;
}

/* Usage */
LIST *trial_balance_subclass_display_html_account_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   char *subclassification_name,
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               transaction_as_of_date,
               element_accumulate_debit,
               account->account_journal_latest,
               account->action_string,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         HTML_ROW *trial_balance_subclass_display_html_account_row(
               element_name,
               subclassification_name,
               statement_account,
               statement_prior_year_list ) );

   element_name = (char *)0;
   subclassification_name = (char *)0;
}
```

TRIAL_BALANCE_SUBCLASS_DISPLAY_HTML (3)

```
/* Usage */
HTML_ROW *trial_balance_subclass_display_html_account_row(
   char *element_name,
   char *subclassification_name,
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *statement_cell_data_label(
   subclassification_name /* name */ );

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *transaction_date_american =
   char *statement_date_american(
         statement_account->
               account->
               account_journal_latest->
               transaction_date_time );

char *trial_balance_html_account_title(
   statement_account->account->account_name,
   statement_account->account->account_journal_latest->full_name,
   statement_account->account->account_journal_latest->debit_amount,
   statement_account->account->account_journal_latest->credit_amount,
   transaction_date_american,
   statement_account->
         account->
         account_journal_latest->
         transaction->memo );

html_cell_data_set(
   html_row->cell_list,
   trial_balance_html_account_title(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *trial_balance_transaction_count_string(
   statement_account->account->transaction_count );

html_cell_data_set(
   html_row->cell_list,
   trial_balance_transaction_count_string(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->debit_string,
   statement_account->within_days_between_boolean /* large_bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->credit_string,
   statement_account->within_days_between_boolean /* large_bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->asset_percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */);

html_cell_data_set(
   html_row->cell_list,
   statement_account->revenue_percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   html_cell_data_list_set(
         html_row->cell_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->account->account_name,
               statement_prior_year_list ) );
}
```

TRIAL_BALANCE_SUBCLASS_OMIT_HTML (1)

```
/* Usage */
TRIAL_BALANCE_SUBCLASS_OMIT_HTML *
   trial_balance_subclass_omit_html_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double debit_sum,
         double credit_sum );

/* Process */
TRIAL_BALANCE_SUBCLASS_OMIT_HTML *
   trial_balance_subclass_omit_html_calloc(
         void );

LIST *trial_balance_subclass_omit_html_heading_list(
   statement_prior_year_list );

LIST *trial_balance_subclass_omit_html_row_list(
   statement->transaction_as_of_date,
   statement->element_statement_list,
   statement_prior_year_list,
   debit_sum,
   credit_sum );

/* Usage */
LIST *trial_balance_subclass_omit_html_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   html_heading_new(
         “Element”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Account”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Count”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Debit”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “Credit”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “percent_of_asset”,
         1 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   html_heading_new(
         “percent_of_revenue”,
         1 /* right_justify_boolean */ ) );

void html_heading_list_right_justify_set(
   heading_list,
   LIST *statement_prior_year_heading_list(
         statement_prior_year_list )
               /* label_list */ );
```



TRIAL_BALANCE_SUBCLASS_OMIT_HTML (2)

```
/* Usage */
LIST *trial_balance_subclass_omit_html_row_list(
   char *statement->transaction_as_of_date,
   LIST *statement->element_statement_list,
   LIST *statement_prior_year_list(),
   double debit_sum,
   double credit_sum );

/* Process */
for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set_list(
               row_list,
               trial_balance_subclass_omit_html_element_row_list(
                     transaction_as_of_date,
                     element->element_name,
                     element->accumulate_debit,
                     element->account_statement_list,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   HTML_ROW *trial_balance_subclass_omit_html_sum_row(
         double debit_sum,
         double credit_sum ) );
```

```
/* Usage */
LIST *trial_balance_subclass_omit_html_element_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               transaction_as_of_date,
               element_accumulate_debit,
               account->account_journal_latest,
               account->action_string,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         HTML_ROW *trial_balance_subclass_omit_html_account_row(
               element_name,
               statement_account,
               statement_prior_year_list ) );

   element_name = (char *)0;
}
```

TRIAL_BALANCE_SUBCLASS_OMIT_HTML (3)

```
/* Usage */
HTML_ROW *trial_balance_subclass_omit_html_account_row(
   char *element_name,
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *transaction_date_american =
   char *statement_date_american(
         statement_account->
               account->
               account_journal_latest->
               transaction_date_time );

char *trial_balance_html_account_title(
   statement_account->account->account_name,
   statement_account->account->account_journal_latest->full_name,
   statement_account->account->account_journal_latest->debit_amount,
   statement_account->account->account_journal_latest->credit_amount,
   transaction_date_american,
   statement_account->
         account->
         account_journal_latest->
         transaction->memo );

html_cell_data_set(
   html_row->cell_list,
   trial_balance_html_account_title(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *trial_balance_transaction_count_string(
   statement_account->account->transaction_count );

html_cell_data_set(
   html_row->cell_list,
   trial_balance_transaction_count_string(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->debit_string,
   0 /* not large_boolean */,
   statement_account->within_days_between_boolean /* bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->credit_string,
   0 /* not large_boolean */,
   statement_account->within_days_between_boolean /* bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->asset_percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   statement_account->revenue_percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   html_cell_data_list_set(
         html_row->cell_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->account->account_name,
               statement_prior_year_list ) );
}
```

TRIAL_BALANCE_PDF

```
/* Usage */
TRIAL_BALANCE_PDF *trial_balance_pdf_new(
   char *application_name,
   char *process_name,
   char *document_root_directory,
   enum statement_subclassification_option
         statement_subclassification_option,
   STATEMENT *preclose_statement,
   LIST *preclose_statement_prior_year_list,
   double preclose_debit_sum,
   double preclose_credit_sum,
   STATEMENT *statement,
   LIST *statement_prior_year_list,
   double debit_sum,
   double credit_sum,
   pid_t process_id );

/* Process */
TRIAL_BALANCE_PDF *trial_balance_pdf_calloc( void );

boolean statement_pdf_landscape_boolean(
   int list_length( statement_prior_year_list )
         /* statement_prior_year_list_length */ );

if ( preclose_statement )
{
   char *trial_balance_pdf_preclose_key(
         char *TRIAL_BALANCE_PDF_PRECLOSE_KEY );

   STATEMENT_LINK *preclose_statement_link =
         statement_link_new(
               application_name,
               process_name,
               document_root_directory,
               preclose_statement->
                     transaction_begin_date_string,
               preclose_statement->
                     transaction_as_of_date,
               trial_balance_pdf_preclose_key(),
               process_id );

   TRIAL_BALANCE_LATEX *
         preclose_trial_balance_latex =
               trial_balance_latex_new(
                     statement_subclassification_option,
                     preclose_statement->
                           transaction_as_of_date,
                     preclose_statement_link->
                           tex_filename,
                     preclose_statement_link->
                           dvi_filename,
                     preclose_statement_link->
                           working_directory,
                     statement_pdf_landscape_boolean(),
                     preclose_statement->
                           statement_caption->
                           logo_filename,
                     preclose_statement,
                     preclose_statement_prior_year_list,
                     preclose_debit_sum,
                     preclose_credit_sum );
}

STATEMENT_LINK *statement_link =
   statement_link_new(
         application_name,
         process_name,
         document_root_directory,
         statement->transaction_begin_date_string,
         statement->transaction_as_of_date,
         (char *)0 /* trial_balance_pdf_preclose_key */,
         process_id );

TRIAL_BALANCE_LATEX *trial_balance_latex =
   trial_balance_latex_new(
         statement_subclassification_option,
         statement->transaction_as_of_date,
         statement_link->tex_filename,
         statement_link->dvi_filename,
         statement_link->working_directory,
         statement_pdf_landscape_boolean(),
         statement->
               statement_caption->
               logo_filename,
         statement,
         statement_prior_year_list,
         debit_sum,
         credit_sum );
```

preclose_statement_link
statement_link

preclose_trial_balance_latex
trial_balance_latex

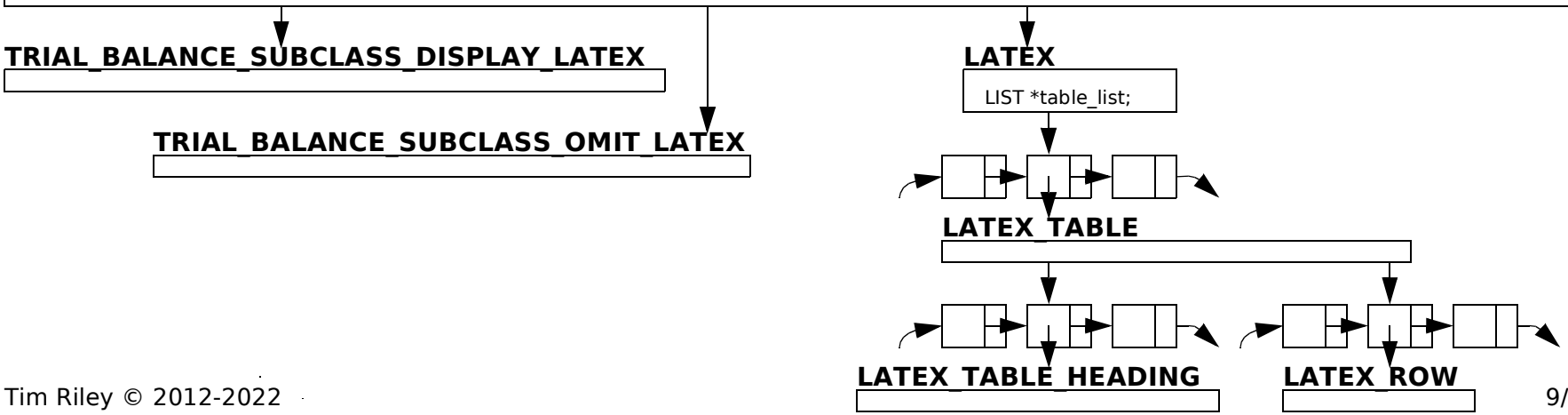
STATEMENT_LINK

```
char *tex_filename;
char *dvi_filename;
char *working_directory;
```

TRIAL_BALANCE_LATEX

TRIAL_BALANCE_LATEX

```
/* Usage */
TRIAL_BALANCE_LATEX *trial_balance_latex_new(
   enum statement_subclassification_option
         statement_subclassification_option,
   char *transaction_as_of_date(),
   char *statement_link->tex_filename,
   char *statement_link->dvi_filename,
   char *statement_link->working_directory,
   boolean statement_pdf_landscape_boolean(),
   char *statement_logo_filename(),
   STATEMENT *statement,
   LIST *statement_prior_year_list,
   double debit_sum,
   double credit_sum );

/* Process */
TRIAL_BALANCE_LATEX *trial_balance_latex_calloc(
   void );

LATEX *latex =
   latex_new(
         tex_filename,
         dvi_filename,
         working_directory,
         statement_pdf_landscape_boolean,
         statement_logo_filename );

if (statement_subclassification_option ==
   subclassification_display )
{
   TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX *
         trial_balance_subclass_display_latex =
               trial_balance_subclass_display_latex_new(
                     transaction_as_of_date,
                     statement,
                     statement_prior_year_list,
                     debit_sum,
                     credit_sum );

   void list_set(
         latex->table_list,
         LATEX_TABLE *trial_balance_latex_table(
               statement->statement_caption->combined,
               trial_balance_subclass_display_latex->
                     heading_list,
               trial_balance_subclass_display_latex->
                     row_list ) );
}
else
{
   TRIAL_BALANCE_SUBCLASS_OMIT_LATEX *
         trial_balance_subclass_omit_latex =
               trial_balance_subclass_omit_latex_new(
                     transaction_as_of_date,
                     statement,
                     statement_prior_year_list,
                     debit_sum,
                     credit_sum );

   void list_set(
         latex->table_list,
         LATEX_TABLE *trial_balance_latex_table(
               statement->statement_caption->string,
               trial_balance_subclass_omit_latex->
                     heading_list,
               trial_balance_subclass_omit_latex->
                     row_list ) );
}

/* Usage */
LATEX_TABLE *trial_balance_latex_table(
   char *statement_caption_combined,
   LIST *heading_list,
   LIST *row_list );

/* Process */
LATEX_TABLE *latex_table_new(
   statement_caption_combined );

latex_table->heading_list = heading_list;
latex_table->row_list = row_list;

/* Public */
char *trial_balance_latex_account_title(
   char *account->account_name,
   char *account->
         account_journal_latest->
         full_name,
   double account->
         account_journal_latest->
         debit_amount,
   double account->
         account_journal_latest->
         credit_amount,
   char *statement_date_american(
         account->
               account_journal_latest->
               transaction_date_time )
                     /* transaction_date_american */,
   char *account->
         account_journal_latest->
         memo );
```

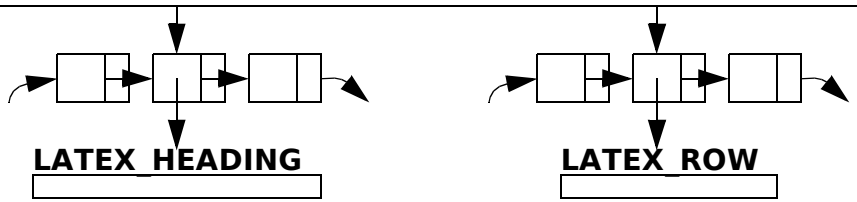


TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX (1)

```
/* Usage */
TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX *
   trial_balance_subclass_display_latex_new(
         char *transaction_as_of_date(),
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double debit_sum,
         double credit_sum );

/* Process */
TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX *
   trial_balance_subclass_display_latex_calloc(
         void );

LIST *trial_balance_subclass_display_latex_heading_list(
   statement_prior_year_list );

LIST *trial_balance_subclass_display_latex_row_list(
   transaction_as_of_date,
   statement->element_statement_list,
   statement_prior_year_list,
   debit_sum,
   credit_sum );

/* Usage */
LIST *trial_balance_subclass_display_latex_heading_list(
   LIST *statement_prior_year_list );
```

```
/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “Element”,
         0 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Subclassification”,
         0 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Account”,
         0 /* right_justified_flag */,
         “5.5cm” /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Count”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );
```

```
list_set(
   heading_list,
   latex_table_heading_new(
         “Debit”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Credit”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Standardized”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set_list(
   heading_list,
   LIST *latex_table_right_heading_list(
         LIST *statement_prior_year_heading_list(
               statement_prior_year_list ) ) );
```



TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX (2)

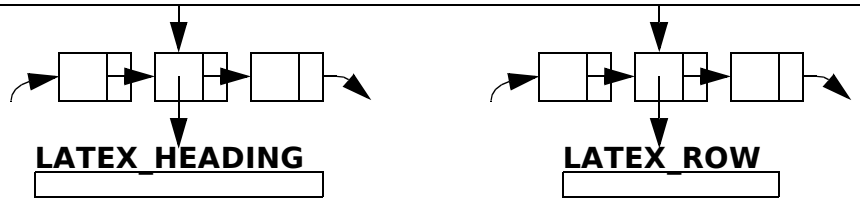
```
/* Usage */
LIST *trial_balance_subclass_display_latex_row_list(
   char *transaction_as_of_date(),
   LIST *statement->element_statement_list,
   LIST *statement_prior_year_list(),
   double debit_sum,
   double credit_sum );

/* Process */
for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set_list(
               row_list,
               LIST *trial_balance_subclass_display_latex_element_row_list(
                     transaction_as_of_date,
                     element->element_name,
                     element->accumulate_debit,
                     element->subclassification_statement_list,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   LATEX_ROW *trial_balance_subclass_display_latex_sum_row(
         double debit_sum,
         double credit_sum ) );

/* Usage */
LIST *trial_balance_subclass_display_latex_element_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   LIST *subclassification_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for SUBCLASSIFICATION *subclassification in
   subclassification_statement_list
{
   list_set_list(
         row_list,
               LIST *trial_balance_subclass_display_latex_account_row_list(
                     transaction_as_of_date,
                     element_name,
                     element_accumulate_debit,
                     subclassification->subclassification_name,
                     subclassification->account_statement_list,
                     statement_prior_year_list ) );

   element_name = (char *)0;
}

/* Usage */
LIST *trial_balance_subclass_display_latex_account_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   char *subclassification_name,
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               transaction_as_of_date,
               element_accumulate_debit,
               account->account_journal_latest,
               (char *)0 /* account_action_string */,
               1 /* round_dollar_boolean */,
               account );

   list_set(
         row_list,
         LATEX_ROW *trial_balance_subclass_display_latex_account_row(
               element_name,
               subclassification_name,
               statement_account,
               statement_prior_year_list ) );

   element_name = (char *)0;
   subclassification_name = (char *)0;
}
```

TRIAL_BALANCE_SUBCLASS_DISPLAY_LATEX (3)

```
/* Usage */
LATEX_ROW *trial_balance_subclass_display_latex_account_row(
   char *element_name,
   char *subclassification_name,
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *statement_cell_data_label(
   subclassification_name /* name */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *transaction_date_american =
   char *statement_date_american(
         statement_account->
               account->
               account_journal_latest->
               transaction_date_time );

char *trial_balance_latex_account_title(
   statement_account->account->account_name,
   statement_account->account->account_journal_latest->full_name,
   statement_account->account->account_journal_latest->debit_amount,
   statement_account->account->account_journal_latest->credit_amount,
   transaction_date_american,
   statement_account->account->account_journal_latest->transaction->memo );

void latex_column_data_set(
   latex_row->column_data_list,
   trial_balance_latex_account_title(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *trial_balance_transaction_count_string(
   statement_account->account->transaction_count );

void latex_column_data_set(
   latex_row->column_data_list,
   trial_balance_transaction_count_string(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->debit_string,
   statement_account->within_days_between_boolean /* large_bold */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->credit_string,
   statement_account->within_days_between_boolean /* large_bold */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->account->account_name,
               statement_prior_year_list ) );
}
```

TRIAL_BALANCE_SUBCLASS_OMIT_LATEX (1)

```
/* Usage */
TRIAL_BALANCE_SUBCLASS_OMIT_LATEX *
   trial_balance_subclass_omit_latex_new(
         char *transaction_as_of_date(),
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double debit_sum,
         double credit_sum );

/* Process */
TRIAL_BALANCE_SUBCLASS_OMIT_LATEX *
   trial_balance_subclass_omit_latex_calloc(
         void );

LIST *trial_balance_subclass_omit_latex_heading_list(
   statement_prior_year_list );

LIST *trial_balance_subclass_omit_latex_row_list(
   transaction_as_of_date,
   statement->element_statement_list,
   statement_prior_year_list,
   debit_sum,
   credit_sum );

/* Usage */
LIST *trial_balance_subclass_omit_latex_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “Element”,
         0 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Account”,
         0 /* right_justified_flag */,
         “8.0cm” /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Count”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Debit”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Credit”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Standardized”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set_list(
   heading_list,
   LIST *latex_table_right_heading_list(
         LIST *statement_prior_year_heading_list(
               statement_prior_year_list ) ) );
```



TRIAL_BALANCE_SUBCLASS_OMIT_LATEX (2)

```
/* Usage */
LIST *trial_balance_subclass_omit_latex_row_list(
   char *transaction_as_of_date(),
   LIST *statement->element_statement_list,
   LIST *statement_prior_year_list(),
   double debit_sum,
   double credit_sum );

/* Process */
for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set_list(
               row_list,
               LIST *trial_balance_subclass_omit_latex_element_row_list(
                     transaction_as_of_date,
                     element->element_name,
                     element->accumulate_debit,
                     element->account_statement_list,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   LATEX_ROW *trial_balance_subclass_omit_latex_sum_row(
         double debit_sum,
         double credit_sum ) );
```

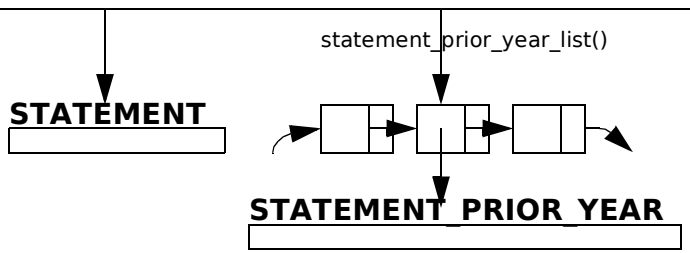
```
/* Usage */
LIST *trial_balance_subclass_omit_latex_element_row_list(
   char *transaction_as_of_date,
   char *element_name,
   boolean element_accumulate_debit,
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               transaction_as_of_date,
               element_accumulate_debit,
               account->account_journal_latest,
               (char *)0 /* account_action_string */,
               1 /* round_dollar_boolean */,
               account );

   list_set(
         row_list,
         LATEX_ROW *trial_balance_subclass_omit_latex_account_row(
               element_name,
               statement_account,
               statement_prior_year_list ) );

   element_name = (char *)0;
}
```

TRIAL_BALANCE_SUBCLASS_OMIT_LATEX (3)

```
/* Usage */
LATEX_ROW *trial_balance_subclass_omit_latex_account_row(
   char *element_name,
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *transaction_date_american =
   char *statement_date_american(
         statement_account->
               account->
               account_journal_latest->
               transaction_date_time );

char *trial_balance_latex_account_title(
   statement_account->account->account_name,
   statement_account->account->account_journal_latest->full_name,
   statement_account->account->account_journal_latest->debit_amount,
   statement_account->account->account_journal_latest->credit_amount,
   transaction_date_american,
   statement_account->account->account_journal_latest->transaction->memo );

void latex_column_data_set(
   latex_row->column_data_list,
   trial_balance_latex_account_title(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *trial_balance_transaction_count_string(
   statement_account->account->transaction_count );

void latex_column_data_set(
   latex_row->column_data_list,
   trial_balance_transaction_count_string(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->debit_string,
   statement_account->within_days_between_boolean /* large_boolen */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->credit_string,
   statement_account->within_days_between_boolean /* large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->account->account_name,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT (1)

```
/* Usage */
INCOME_STATEMENT *income_statement_fetch(
   char *argv_0,
   char *application_name,
   char *session_key,
   char *login_name,
   char *role_name,
   char *process_name,
   char *document_root_directory,
   char *as_of_date_string,
   int prior_year_count,
   char *subclassification_option_string,
   char *output_medium_string );

/* Process */
INCOME_STATEMENT *income_statement_calloc( void );

enum statement_subclassification_option =
   statement_resolve_subclassification_option(
         subclassification_option_string );

enum statement_output_medium =
   statement_resolve_output_medium(
         output_medium_string );

char *transaction_as_of_date(
   TRANSACTION_TABLE,
   as_of_date_string );

if ( !transaction_as_of_date() )
{
   return (INCOME_STATEMENT *)0;
}

char *transaction_begin_date_string(
   TRANSACTION_TABLE,
   transaction_as_of_date() );

if ( !transaction_begin_date_string() )
{
   exit( 1 );
}

LIST *income_statement_element_name_list(
   char *ELEMENT_REVENUE,
   char *ELEMENT_EXPENSE,
   char *ELEMENT_GAIN,
   char *ELEMENT_LOSS );

boolean transaction_closing_entry_exists(
   TRANSACTION_TABLE,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date() );

char *transaction_date_time_closing(
   TRANSACTION_PRECLOSE_TIME,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date(),
   transaction_closing_entry_exists()
         /* preclose_time_boolean */ );

STATEMENT *statement =
   statement_fetch(
         application_name,
         process_name,
         income_statement_element_name_list(),
         transaction_begin_date_string(),
         transaction_as_of_date(),
         transaction_date_time_closing(),
         0 /* not fetch_transaction */ );

if ( statement_output_medium == output_table )
{
   void element_account_transaction_count_set(
         statement->element_statement_list,
         transaction_begin_date_string(),
         transaction_date_time_closing() );

   element_account_action_string_set(
         statement->element_statement_list,
         application_name,
         session_key,
         login_name,
         role_name,
         transaction_begin_date_string(),
         transaction_date_time_closing() );
}

if ( prior_year_count )
{
   LIST *statement_prior_year_list(
         income_statement_element_name_list(),
         transaction_date_time_closing(),
         prior_year_count,
         statement );
}
```



INCOME_STATEMENT (2)

```
/* Process (continued) */
if (statement_subclassification_option ==
   subclassification_omit )
{
   element_list_account_statement_list_set(
         statement->
               element_statement_list );
}

double element_net_income(
   statement->element_statement_list );

char *income_statement_net_income_percent_of_revenue_display(
   char *ELEMENT_REVENUE,
   double element_net_income(),
   LIST *statement->element_statement_list );

char *income_statement_net_income_label(
   char *argv_0 );

if ( statement_output_medium == statement_output_PDF )
{
   INCOME_STATEMENT_PDF *
         income_statement_pdf =
               income_statement_pdf_new(
                     application_name,
                     process_name,
                     document_root_directory,
                     statement_subclassification_option,
                     statement,
                     statement_prior_year_list,
                     element_net_income(),
                     income_statement_net_income_percent_of_revenue_display(),
                     income_statement_net_income_label(),
                     getpid() /* process_id */ );
}
else
if ( statement_output_medium == statement_output_table )
{
   INCOME_STATEMENT_HTML *
         income_statement_html =
               income_statement_html_new(
                     statement_subclassification_option,
                     statement,
                     statement_prior_year_list,
                     element_net_income(),
                     income_statement_net_income_percent_of_revenue_display(),
                     income_statement_net_income_label() );
}
```

INCOME_STATEMENT_PDF **INCOME_STATEMENT_HTML**

INCOME_STATEMENT (3)

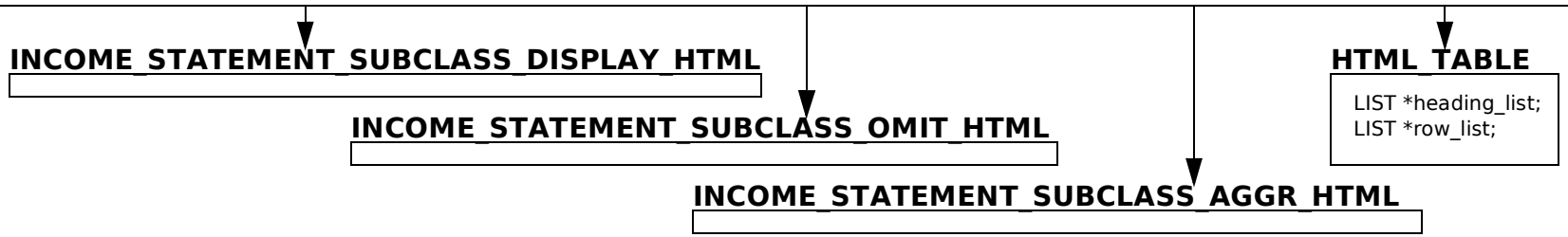
```
/* Usage */
LIST *income_statement_prior_net_income_data_list(
   double current_net_income,
   LIST *statement_prior_year_list );

/* Process */
for STATEMENT_PRIOR_YEAR *statement_prior_year in
   statement_prior_year_list
{
   double prior_net_income =
         element_net_income(
               statement_prior_year->
                     element_statement_list );

   int delta_prior_percent =
            float_delta_prior_percent(
                  prior_net_income,
                  current_net_income );

   list_set(
         data_list,
         char *statement_prior_year_cell_display(
            0 /* not cell_empty */,
            delta_prior_percent,
            prior_net_income /* prior_year_amount */ ) );
}
```

```
/* Usage */
double income_statement_fetch_net_income(
   char *transaction_date_time_closing() );

/* Process */
LIST *income_statement_element_name_list(
   ELEMENT_REVENUE,
   ELEMENT_EXPENSE,
   ELEMENT_GAIN,
   ELEMENT_LOSS );

STATEMENT *statement =
   statement_fetch(
         (char *)0 /* application_name */,
         (char *)0 /* process_name */,
         income_statement_element_name_list(),
         (char *)0 /* transaction_begin_date_string */,
         (char *)0 /* transaction_as_of_date */,
         transaction_date_time_closing(),
         0 /* not fetch_transaction */ );

double element_net_income(
   statement->element_statement_list );

/* Driver */
printf( “%s\n”,
         statement->
```

```
         statement_caption->
         frame_title );

if ( income_statement_pdf )
{
   void statement_latex_output(
         income_statement_pdf->
               income_statement_latex->
               latex,
         income_statement_pdf->
               statement_link->
               ftp_output_filename,
         statement_pdf_prompt( process_name ),
         process_name,
         statement->
               statement_caption->
               date_time_string );
}
else
if ( income_statement_html )
{
   void statement_html_output(
         income_statement_html->
               html_table,
               (char *)0 /* secondary_title */ );
}
```

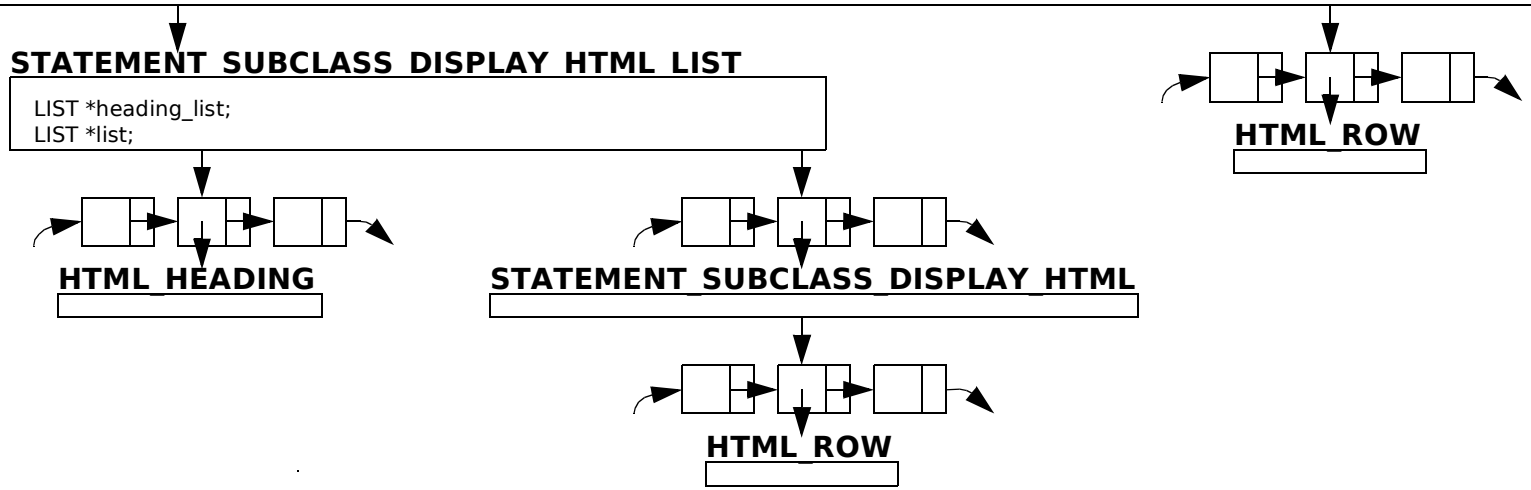
INCOME_STATEMENT_HTML

```
/* Usage */
INCOME_STATEMENT_HTML *
   income_statement_html_new(
         enum statement_subclassification_option
               statement_subclassification_option,
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_HTML *income_statement_html_calloc( void );

if (statement_subclassification_option == subclassification_display )
{
   INCOME_STATEMENT_SUBCLASS_DISPLAY_HTML *
         income_statement_subclass_display_html =
               income_statement_subclass_display_html_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     income_statement_net_income_percent_of_revenue_display,
                     income_statement_net_income_label );

   HTML_TABLE *income_statement_html_table(
         statement->caption_subtitle,
         income_statement_subclass_display_html->heading_list,
         income_statement_subclass_display_html->row_list );
}
else
if ( statement_subclassification_option == subclassification_omit )
{
   INCOME_STATEMENT_SUBCLASS_OMIT_HTML *
         income_statement_subclass_omit_html =
               income_statement_subclass_omit_html_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     income_statement_net_income_percent_of_revenue_display,
                     income_statement_net_income_label );

   HTML_TABLE *income_statement_html_table(
         statement->caption_subtitle,
         income_statement_subclass_omit_html->heading_list,
         income_statement_subclass_omit_html->row_list );
}
else
if ( statement_subclassification_option == subclassification_aggregate )
{
   INCOME_STATEMENT_SUBCLASS_AGGR_HTML *
         income_statement_subclass_aggr_html =
               income_statement_subclass_aggr_html_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     income_statement_net_income_percent_of_revenue_display,
                     income_statement_net_income_label );

   HTML_TABLE *income_statement_html_table(
         statement->caption_subtitle,
         income_statement_subclass_aggr_html->heading_list,
         income_statement_subclass_aggr_html->row_list );
}

/* Usage */
HTML_TABLE *income_statement_html_table(
   char *statement_caption_subtitle,
   LIST *heading_list,
   LIST *row_list );

/* Process */
HTML_TABLE *html_table =
   html_table_new(
         (char *)0 /* title */,
         statement_caption_subtitle,
         (char *)0 /* sub_sub_title */ );

   html_table->heading_list = heading_list;
   html_table->row_list = row_list;
```



INCOME_STATEMENT_SUBCLASS_DISPLAY_HTML (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_DISPLAY_HTML *
   income_statement_subclass_display_html_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_DISPLAY_HTML *
   income_statement_subclass_display_html_calloc(
         void );

STATEMENT_SUBCLASS_DISPLAY_HTML_LIST *
   statement_subclass_display_html_list =
         statement_subclass_display_html_list_new(
               statement->element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   statement_subclass_display_html_list_extract_row_list(
         statement_subclass_display_html_list );

list_set(
   row_list,
   HTML_ROW *income_statement_subclass_display_html_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label ) );
```

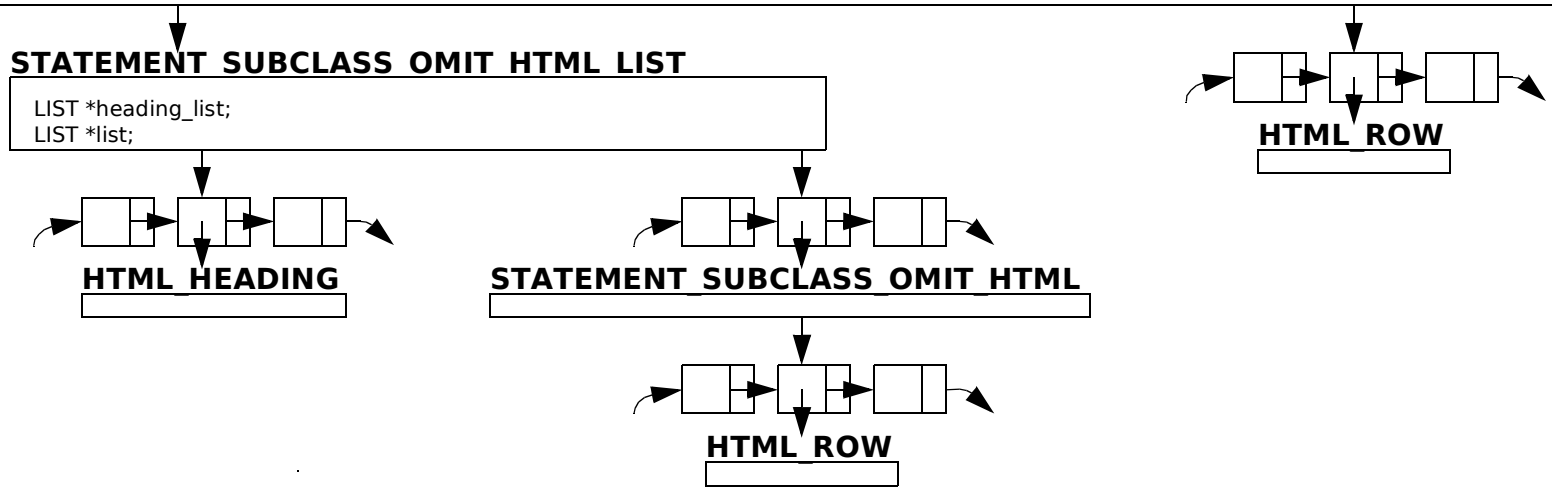


INCOME_STATEMENT_SUBCLASS_DISPLAY_HTML (2)

```
/* Usage */
HTML_ROW *income_statement_subclass_display_html_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
HTML_ROW *html_row = html_row_new();

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT_SUBCLASS_OMIT_HTML (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_OMIT_HTML *
   income_statement_subclass_omit_html_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_OMIT_HTML *
   income_statement_subclass_omit_html_calloc(
         void );

STATEMENT_SUBCLASS_OMIT_HTML_LIST *
   statement_subclass_omit_html_list =
         statement_subclass_omit_html_list_new(
               statement->element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   statement_subclass_omit_html_list_extract_row_list(
         statement_subclass_omit_html_list );

list_set(
   row_list,
   HTML_ROW *income_statement_subclass_omit_html_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label ) );
```

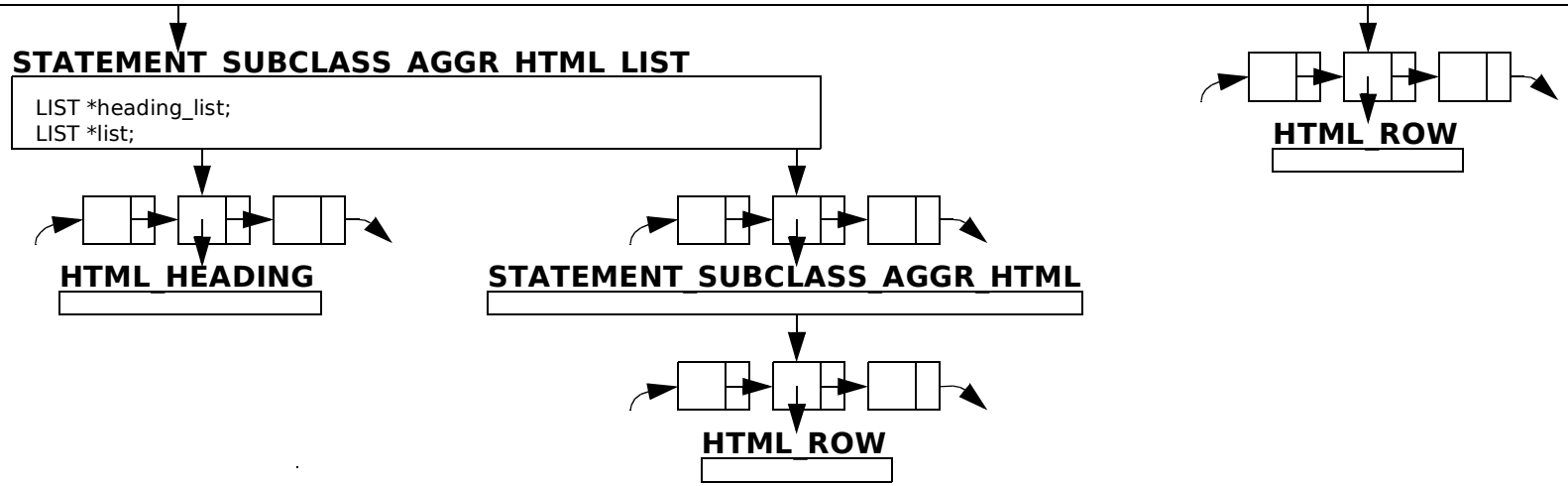


INCOME_STATEMENT_SUBCLASS_OMIT_HTML (2)

```
/* Usage */
HTML_ROW *income_statement_subclass_omit_html_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
HTML_ROW *html_row = html_row_new();

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT_SUBCLASS_AGGR_HTML (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_AGGR_HTML *
   income_statement_subclass_aggr_html_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_AGGR_HTML *
   income_statement_subclass_aggr_html_calloc(
         void );

STATEMENT_SUBCLASS_AGGR_HTML_LIST *
   statement_subclass_aggr_html_list =
         statement_subclass_aggr_html_list_new(
               statement->element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   statement_subclass_aggr_html_list_extract_row_list(
         statement_subclass_aggr_html_list );

list_set(
   row_list,
   HTML_ROW *income_statement_subclass_aggr_html_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label ) );
```



INCOME_STATEMENT_SUBCLASS_AGGR_HTML (2)

```
/* Usage */
HTML_ROW *income_statement_subclass_aggr_html_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
HTML_ROW *html_row = html_row_new();

html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_data_set(
   html_row->cell_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT_PDF

```
/* Usage */
INCOME_STATEMENT_PDF *
   income_statement_pdf_new(
         char *application_name,
         char *process_name,
         char *document_root_directory,
         enum statement_subclassification_option
               statement_subclassification_option,
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label(),
         pid_t process_id );

/* Process */
INCOME_STATEMENT_PDF *
   income_statement_pdf_calloc(
         void );

boolean statement_pdf_landscape_boolean(
   list_length( statement_prior_year_list )
         /* statement_prior_year_list_length */ );

STATEMENT_LINK *statement_link =
   statement_link_new(
         application_name,
         process_name,
         document_root_directory,
         statement->transaction_begin_date_string,
         statement->transaction_as_of_date,
         (char *)0 /* statement_pdf_preclose_key */,
         process_id );

INCOME_STATEMENT_LATEX *
   income_statement_latex =
         income_statement_latex_new(
               statement_subclassification_option,
               statement_link->tex_filename,
               statement_link->dvi_filename,
               statement_link->working_directory,
               statement_pdf_landscape_boolean(),
               statement->statement_logo_filename,
               statement,
               statement_prior_year_list,
               element_net_income,
               income_statement_net_income_percent_of_revenue_display,
               income_statement_net_income_label );
```

statement_link

STATEMENT_LINK

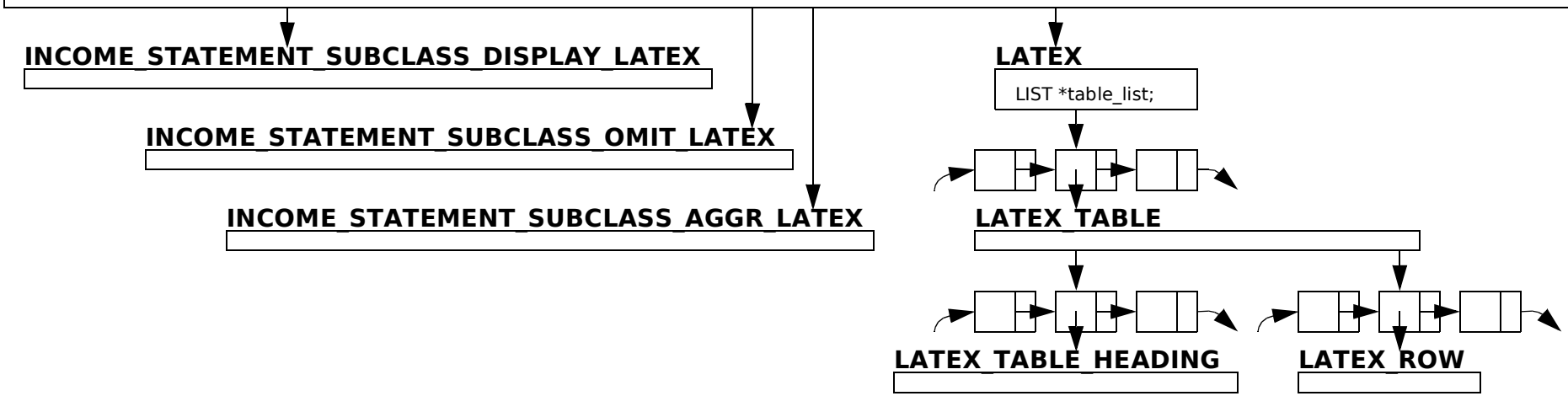
```
char *tex_filename;
char *dvi_filename;
char *working_directory;
```

income_statement_latex

INCOME_STATEMENT_LATEX

INCOME_STATEMENT_LATEX (1)

```
/* Usage */
INCOME_STATEMENT_LATEX *
   income_statement_latex_new(
         enum statement_subclassification_option
               statement_subclassification_option,
         char *statement_link->tex_filename,
         char *statement_link->dvi_filename,
         char *statement_link->working_directory,
         boolean statement_pdf_landscape_boolean(),
         char *statement_logo_filename(),
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         double element_net_income(),
         char *net_income_percent_of_revenue_display,
         char *net_income_label );

/* Process */
INCOME_STATEMENT_LATEX *income_statement_latex_calloc( void );

LATEX *latex =
   latex_new(
         tex_filename,
         dvi_filename,
         working_directory,
         statement_pdf_landscape_boolean,
         statement_logo_filename );

if (statement_subclassification_option == subclassification_display )
{
   INCOME_STATEMENT_SUBCLASS_DISPLAY_LATEX *
         income_statement_subclass_display_latex =
               income_statement_subclass_display_latex_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     net_income_percent_of_revenue_display,
                     net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *income_statement_latex_table(
               statement->statement_caption->combined,
               income_statement_subclass_display_latex_list->heading_list,
               income_statement_subclass_display_latex->row_list ) );
}
```

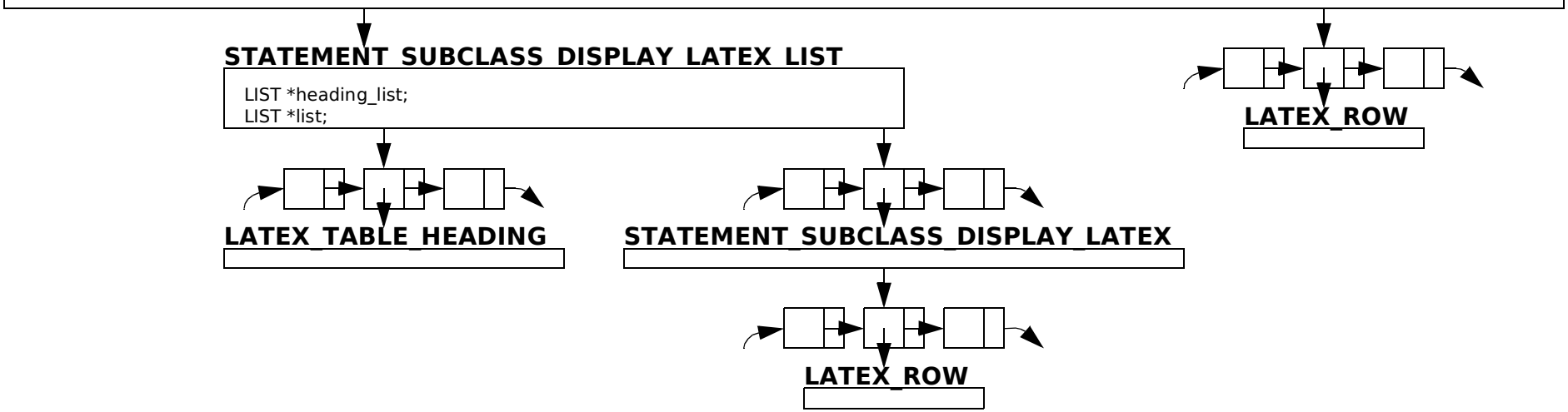


INCOME_STATEMENT_LATEX (2)

```
/* Process (continued) */
else
if ( statement_subclassification_option == subclassification_omit )
{
   INCOME_STATEMEMENT_SUBCLASS_OMIT_LATEX *
         income_statement_subclass_omit_latex =
               income_statement_subclass_omit_latex_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     net_income_percent_of_revenue_display,
                     net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *income_statement_latex_table(
               statement->statement_caption->combined,
               income_statement_subclass_omit_latex_list->heading_list,
               income_statement_subclass_omit_latex->row_list ) );
}
else
if ( statement_subclassification_option == subclassification_aggregate )
{
   INCOME_STATEMEMENT_SUBCLASS_AGGR_LATEX *
         income_statement_subclass_aggr_latex =
               income_statement_subclass_aggr_latex_new(
                     statement,
                     statement_prior_year_list,
                     element_net_income,
                     net_income_percent_of_revenue_display,
                     net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *income_statement_latex_table(
               statement->statement_caption->combined,
               income_statement_subclass_aggr_latex_list->heading_list,
               income_statement_subclass_aggr_latex->row_list ) );
}

/* Usage */
LATEX_TABLE *income_statement_latex_table(
   char *statement_caption_combined,
   LIST *heading_list,
   LIST *row_list );

/* Process */
LATEX_TABLE *latex_table = latex_table_new( statement_caption_combined );
latex_table->heading_list = heading_list;
latex_table->row_list = row_list;
```

INCOME_STATEMENT_SUBCLASS_DISPLAY_LATEX (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_DISPLAY_LATEX *
   income_statement_subclass_display_latex_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_DISPLAY_LATEX *
   income_statement_subclass_display_latex_calloc(
         void );

STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST *
   statement_subclass_display_latex_list =
         statement_subclass_display_latex_list_new(
               statement->element_statement_list,
               statement->statement_prior_year_list );

LIST *row_list =
   statement_subclass_display_latex_list_extract_row_list(
         statement_subclass_display_latex_list );

list_set(
   row_list,
   LATEX_ROW *income_statement_subclass_display_latex_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label );
```

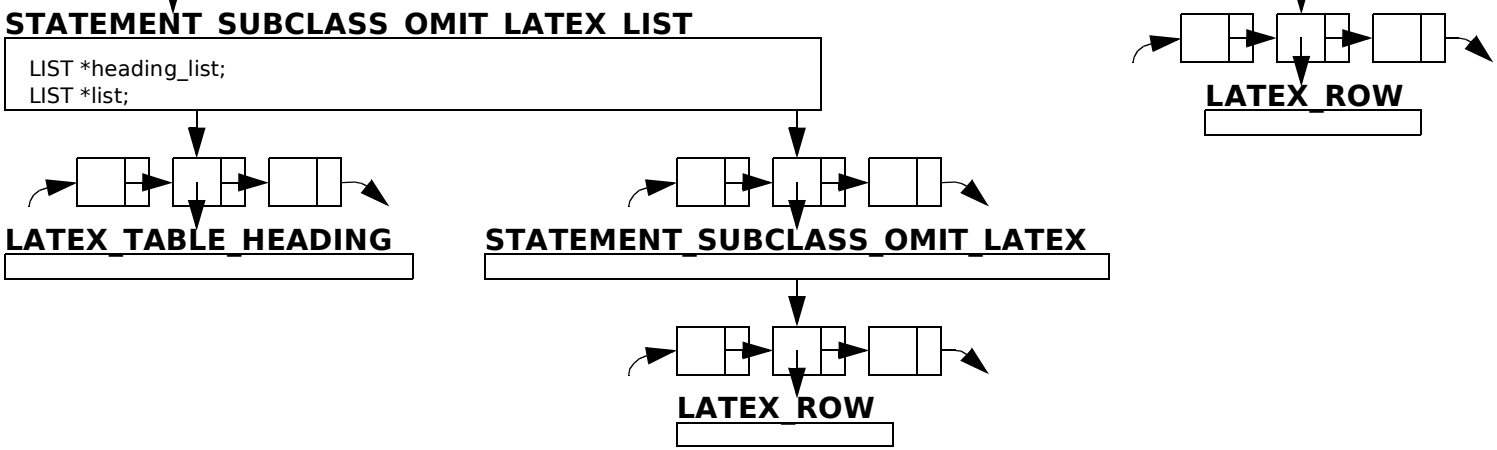


INCOME_STATEMENT_SUBCLASS_DISPLAY_LATEX (2)

```
/* Usage */
LATEX_ROW *income_statement_subclass_display_latex_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */);

latex_column_data_set(
   latex_row->column_data_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */);

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT_SUBCLASS_OMIT_LATEX (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_OMIT_LATEX *
   income_statement_subclass_omit_latex_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_OMIT_LATEX *
   income_statement_subclass_omit_latex_calloc(
         void );

STATEMENT_SUBCLASS_OMIT_LATEX_LIST *
   statement_subclass_omit_latex_list =
         statement_subclass_omit_latex_list_new(
               statement->element_statement_list,
               statement->statement_prior_year_list );

LIST *row_list =
   statement_subclass_omit_latex_list_extract_row_list(
         statement_subclass_omit_latex_list );

list_set(
   row_list,
   LATEX_ROW *income_statement_subclass_omit_latex_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label );
```

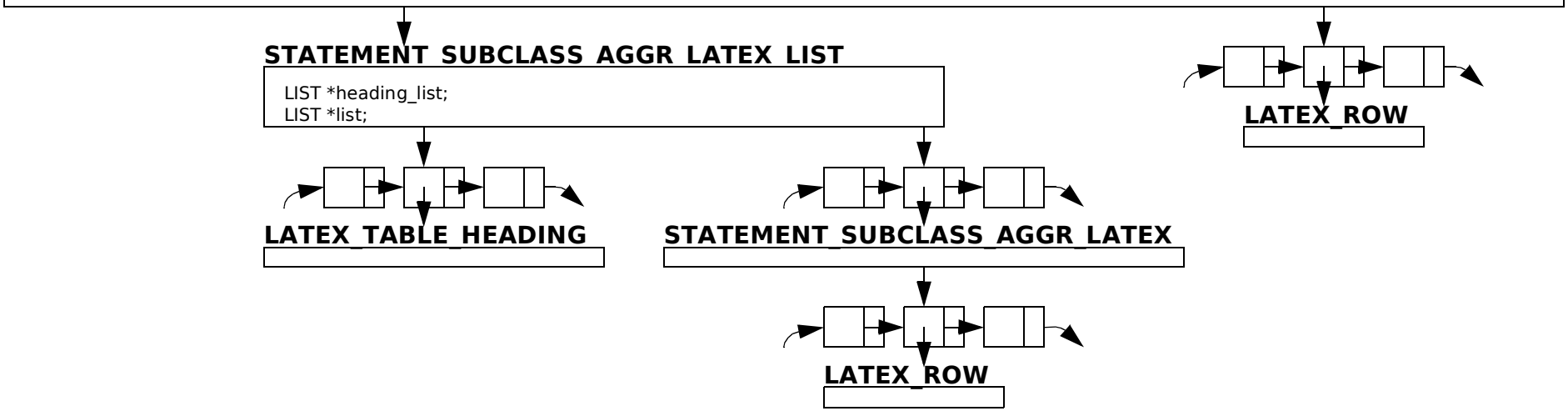


INCOME_STATEMENT_SUBCLASS_OMIT_LATEX (2)

```
/* Usage */
LATEX_ROW *income_statement_subclass_omit_latex_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */);

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

INCOME_STATEMENT_SUBCLASS_AGGR_LATEX (1)

```
/* Usage */
INCOME_STATEMENT_SUBCLASS_AGGR_LATEX *
   income_statement_subclass_aggr_latex_new(
         STATEMENT *statement,
         LIST *statement_prior_year_list(),
         double element_net_income(),
         char *income_statement_net_income_percent_of_revenue_display(),
         char *income_statement_net_income_label() );

/* Process */
INCOME_STATEMENT_SUBCLASS_AGGR_LATEX *
   income_statement_subclass_aggr_latex_calloc(
         void );

STATEMENT_SUBCLASS_AGGR_LATEX_LIST *
   statement_subclass_aggr_latex_list =
         statement_subclass_aggr_latex_list_new(
               statement->element_statement_list,
               statement->statement_prior_year_list );

LIST *row_list =
   statement_subclass_aggr_latex_list_extract_row_list(
         statement_subclass_aggr_latex_list );

list_set(
   row_list,
   LATEX_ROW *income_statement_subclass_aggr_latex_net_income_row(
         statement_prior_year_list,
         element_net_income,
         income_statement_net_income_percent_of_revenue_display,
         income_statement_net_income_label );
```

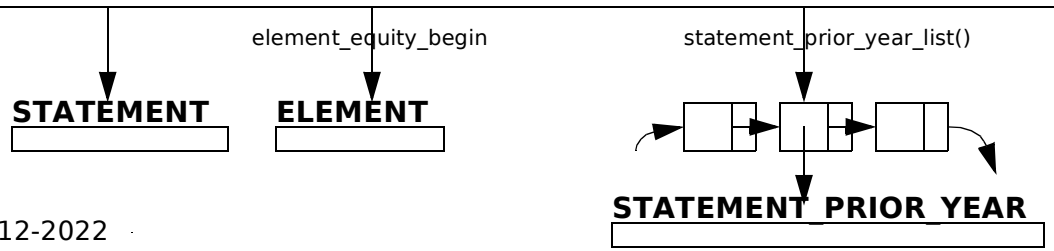


INCOME_STATEMENT_SUBCLASS_AGGR_LATEX (2)

```
/* Usage */
LATEX_ROW *income_statement_subclass_aggr_latex_net_income_row(
   LIST *statement_prior_year_list,
   double element_net_income(),
   char *income_statement_net_income_percent_of_revenue_display(),
   char *income_statement_net_income_label() );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label( income_statement_net_income_label ),
   1 /* large_boolean */,
   1 /* bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   (char *)0,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup( timlib_place_commas_in_money( element_net_income ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   income_statement_net_income_percent_of_revenue_display,
   0 /* not large_boolean */,
   0 /* not bold_boolean */);

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *income_statement_prior_net_income_data_list(
               element_net_income /* current_net_income */,
               statement_prior_year_list ) );
}
```

BALANCE_SHEET (1)

```
/* Usage */
BALANCE_SHEET *balance_sheet_fetch(
      char *argv_0,
      char *application_name,
      char *session_key,
      char *role_name,
      char *process_name,
      char *document_root_directory,
      char *as_of_date_string,
      int prior_year_count,
      char *subclassification_option_string,
      char *output_medium_string );

/* Process */
BALANCE_SHEET *balance_sheet_calloc( void );

enum statement_subclassification_option =
      statement_resolve_subclassification_option(
            subclassification_option_string );

enum statement_output_medium =
      statement_resolve_output_medium(
            output_medium_string );

char *transaction_as_of_date(
      TRANSACTION_TABLE,
      as_of_date_string );

if ( !transaction_as_of_date() )
{
      return (BALANCE_SHEET *)0;
}

char *transaction_begin_date_string(
      TRANSACTION_TABLE,
      transaction_as_of_date() );

if ( !transaction_begin_date_string() )
{
      exit( 1 );
}

LIST *balance_sheet_element_name_list(
      char *ELEMENT_ASSET,
      char *ELEMENT_LIABILITY );

boolean transaction_closing_entry_exists(
      TRANSACTION_TABLE,
      TRANSACTION_CLOSE_TIME,
      transaction_as_of_date() );

char *transaction_date_time_closing(
      TRANSACTION_PRECLOSE_TIME,
      TRANSACTION_CLOSE_TIME,
      transaction_as_of_date(),
      transaction_closing_entry_exists()
            /* preclose_time_boolean */ );

STATEMENT *statement =
      statement_fetch(
            application_name,
            process_name,
            balance_sheet_element_name_list(),
            transaction_begin_date_string(),
            transaction_as_of_date(),
            transaction_date_time_closing(),
            0 /* not fetch_transaction */ );

ELEMENT *element_equity_current =
      element_statement_fetch(
            ELEMENT_EQUITY,
            transaction_date_time_closing(),
            1 /* fetch_subclassification_list */,
            1 /* fetch_account_list */,
            1 /* fetch_journal_latest */,
            0 /* not fetch_transaction */ );

double element_equity_current->sum =
      element_sum(
            element_equity_current );

if ( statement_output_medium == output_table )
{
   void element_account_transaction_count_set(
            statement->element_statement_list,
            transaction_begin_date_string(),
            transaction_date_time_closing() );

   element_account_action_string_set(
            statement->element_statement_list,
            application_name,
            session_key,
            login_name,
            role_name,
            transaction_begin_date_string(),
            transaction_date_time_closing() );
}

if ( prior_year_count )
{
   LIST *statement_prior_year_list(
            balance_sheet_element_name_list(),
            transaction_date_time_closing(),
            prior_year_count,
            statement );
}

char *balance_sheet_prior_date_time_closing(
      transaction_begin_date_string() );

if ( balance_sheet_prior_date_time_closing() )
{
      ELEMENT *element_equity_begin =
            element_statement_fetch(
                  ELEMENT_EQUITY,
                  balance_sheet_prior_date_time_closing(),
                  1 /* fetch_subclassification_list */,
                  1 /* fetch_account_list */,
                  1 /* fetch_journal_latest */,
                  0 /* not fetch_transaction */ );

      double element_equity_begin->sum =
            element_sum(
                  element_equity_begin );
}
```



BALANCE_SHEET (2)

```
/* Process (continued) */
double income_statement_fetch_net_income(
      transaction_date_time_closing() );

char *income_statement_net_income_label(
      char *argv_0 );

if (statement_subclassification_option ==
   subclassification_omit )
{
   element_list_account_statement_list_set(
         statement->
               element_statement_list );
}

if ( statement_output_medium ==
         statement_output_PDF )
{
   BALANCE_SHEET_PDF *
         balance_sheet_pdf =
               balance_sheet_pdf_new(
                     application_name,
                     process_name,
                     document_root_directory,
                     statement_subclassification_option,
                     statement,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     income_statement_fetch_net_income(),
                     income_statement_net_income_label(),
                     getpid() /* process_id */ );
}
else
if ( statement_output_medium ==
         statement_output_table )
{
   BALANCE_SHEET_HTML *
         balance_sheet_html =
               balance_sheet_html_new(
                     statement_subclassification_option,
                     statement,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     income_statement_fetch_net_income(),
                     income_statement_net_income_label() );
}

/* Usage */
char *balance_sheet_prior_date_time_closing(
      char *transaction_begin_date_string() );

/* Process */
DATE *prior = date_yyyy_mm_dd_new(
      transaction_begin_date_string );

void date_decrement_second( prior, 1 );
char *date_display19( prior );

/* Public */
double balance_sheet_equity_transaction_amount(
      double element_equity_current_sum,
      double element_equity_begin_sum );

double balance_sheet_equity_ending_balance(
   double element_equity_current_sum,
   double income_statement_fetch_net_income() );

double balance_sheet_liability_plus_equity(
   double element_liability_sum,
   double balance_sheet_equity_ending_balance() );

/* Driver */
printf( "%s\n",
         statement->
         statement_caption->
         frame_title );

if ( balance_sheet_pdf )
{
   void statement_latex_output(
         balance_sheet_pdf->
               balance_sheet_latex->
               latex,
         balance_sheet_pdf->
               statement_link->
               ftp_output_filename,
         statement_pdf_prompt( process_name ),
         process_name,
         statement->
               statement_caption->
               date_time_string );
}
else
if ( balance_sheet_html )
{
   void statement_html_output(
         balance_sheet_html->
               html_table,
               (char *)0 /* secondary_title */ );
}
```

BALANCE_SHEET_PDF **BALANCE_SHEET_HTML**

BALANCE_SHEET_EQUITY

```
/* Usage */
BALANCE_SHEET_EQUITY *
    balance_sheet_equity_new(
        ELEMENT *element_equity_begin,
        ELEMENT *element_equity_current,
        ELEMENT *element_liability,
        double income_statement_fetch_net_income() );

/* Process */
BALANCE_SHEET_EQUITY *balance_sheet_equity_calloc(
    void );

double balance_sheet_equity_begin_balance(
    ELEMENT *element_equity_begin );

double balance_sheet_equity_current_balance(
    ELEMENT *element_equity_current );

double balance_sheet_equity_transaction_amount(
    double balance_sheet_equity_current_balance(),
    double balance_sheet_equity_begin_balance() );

double balance_sheet_equity_end_balance(
    double element_equity_current_balance(),
    double income_statement_fetch_net_income );

double balance_sheet_equity_liability_balance(
    ELEMENT *element_liability );

double balance_sheet_equity_liability_plus_equity(
    double balance_sheet_equity_liability_balance(),
    double balance_sheet_equity_end_balance() );
```

BALANCE_SHEET_HTML

```
/* Usage */
BALANCE_SHEET_HTML *
   balance_sheet_html_new(
         enum statement_subclassification_option
               statement_subclassification_option,
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_HTML *balance_sheet_html_calloc(
   void );

ELEMENT *element_liability =
   element_seek(
         ELEMENT_LIABILITY,
         statement->element_statement_list );

if (     statement_subclassification_option ==
         subclassification_display )
{
   BALANCE_SHEET_SUBCLASS_DISPLAY_HTML *
         balance_sheet_subclass_display_html =
               balance_sheet_subclass_display_html_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_currrent,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   HTML_TABLE *balance_sheet_html_table(
         statement->caption_subtitle,
         balance_sheet_subclass_display_html->
               statement_subclass_display_html->
               heading_list,
         balance_sheet_subclass_display_html->row_list );
}
else
if (     statement_subclassification_option ==
         subclassification_omit )
{
   BALANCE_SHEET_SUBCLASS_OMIT_HTML *
         balance_sheet_subclass_omit_html =
               balance_sheet_subclass_omit_html_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   HTML_TABLE *balance_sheet_html_table(
         statement->caption_subtitle,
         balance_sheet_subclass_omit_html->
               statement_subclass_omit_html->
               heading_list,
         balance_sheet_subclass_omit_html->row_list );
}
else
if (     statement_subclassification_option ==
         subclassification_aggregate )
{
   BALANCE_SHEET_SUBCLASS_AGGR_HTML *
         balance_sheet_subclass_aggr_html =
               balance_sheet_subclass_aggr_html_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   HTML_TABLE *balance_sheet_html_table(
         statement->caption_subtitle,
         balance_sheet_subclass_aggr_html->
               statement_subclass_aggr_html->
               heading_list,
         balance_sheet_subclass_aggr_html->row_list );
}

/* Usage */
HTML_TABLE *balance_sheet_html_table(
   char *statement_caption_subtitle,
   LIST *heading_list,
   LIST *row_list );

/* Process */
HTML_TABLE *html_table =
   html_table_new(
         (char *)0 /* title */,
         statement_caption_subtitle,
         (char *)0 /* sub_sub_title */ );

html_table->heading_list = heading_list;
html_table->row_list = row_list;
```

BALANCE_SHEET_SUBCLASS_DISPLAY_HTML

BALANCE_SHEET_SUBCLASS_OMIT_HTML

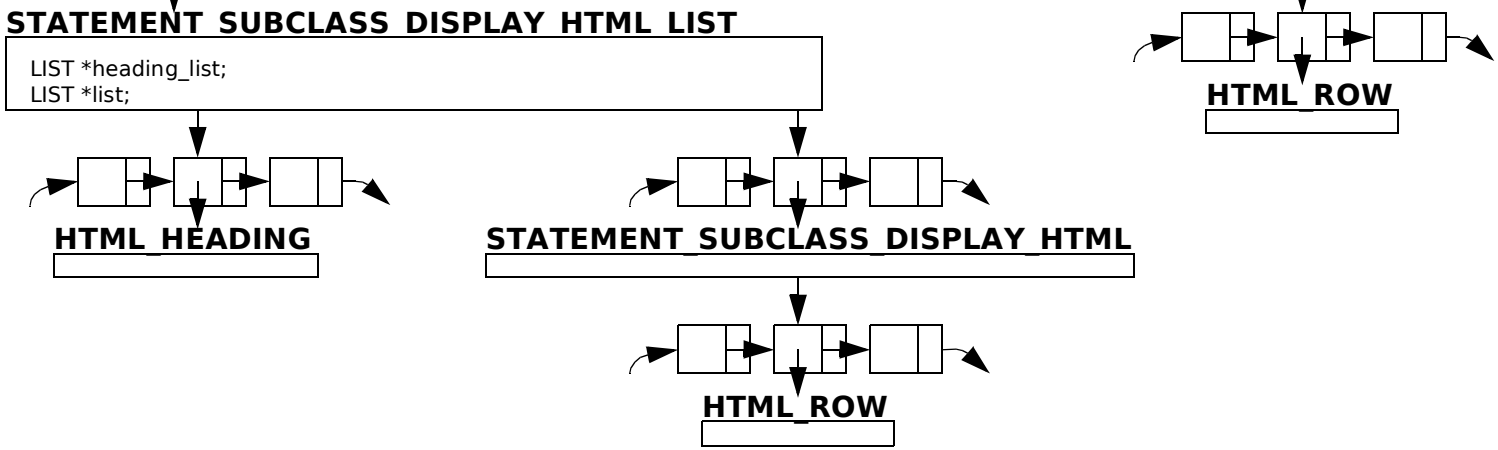
BALANCE_SHEET_SUBCLASS_AGGR_HTML

HTML_TABLE

```
LIST *heading_list;
LIST *row_list;
```

BALANCE_SHEET_SUBCLASS_DISPLAY_HTML (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_DISPLAY_HTML *
   balance_sheet_subclass_display_html_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_DISPLAY_HTML *
   balance_sheet_subclass_display_html_calloc(
         void );

STATEMENT_SUBCLASS_DISPLAY_HTML_LIST *
   statement_subclass_display_html_list =
         statement_subclass_display_html_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_display_html_list_extract_row_list(
         statement_subclass_display_html_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_display_html_equity_row_list(
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```

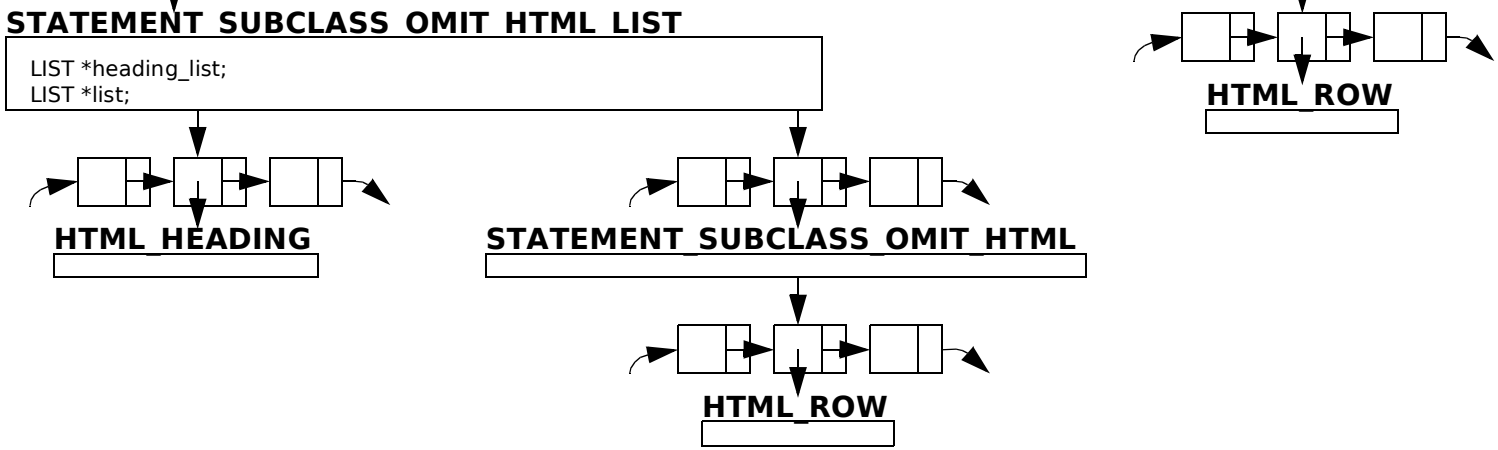


BALANCE_SHEET_SUBCLASS_DISPLAY_HTML (2)

```
/* Usage */
LIST *balance_sheet_subclass_display_html_equity_row_list(
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *balance_sheet_equity =
   balance_sheet_equity_new(
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_display_html_equity_begin_row(
         char *BALANCE_SHEET_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_display_html_transaction_amount_row(
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_display_html_net_income_row(
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_display_html_equity_end_row(
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_display_html_liability_plus_equity_row(
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

BALANCE_SHEET_SUBCLASS_OMIT_HTML (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_OMIT_HTML *
   balance_sheet_subclass_omit_html_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_OMIT_HTML *
   balance_sheet_subclass_display_html_calloc(
         void );

STATEMENT_SUBCLASS_OMIT_HTML_LIST *
   statement_subclass_omit_html_list =
         statement_subclass_omit_html_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_omit_html_list_extract_row_list(
         statement_subclass_omit_html_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_omit_html_equity_row_list(
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```

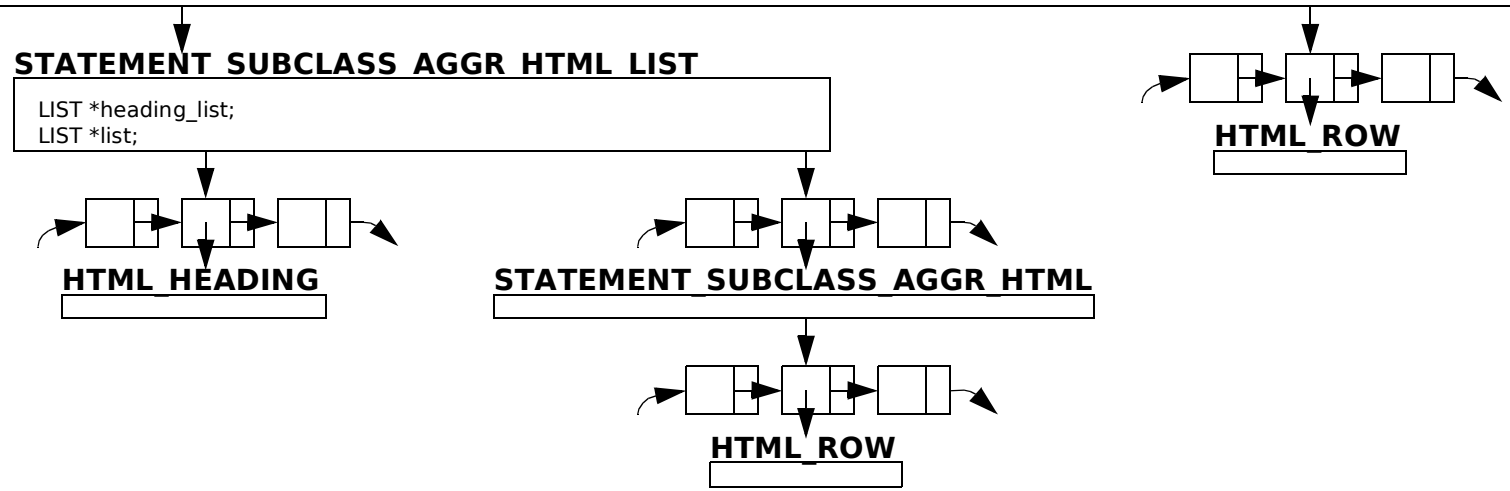


BALANCE_SHEET_SUBCLASS_OMIT_HTML (2)

```
/* Usage */
LIST *balance_sheet_subclass_omit_html_equity_row_list(
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *
   balance_sheet_equity =
         balance_sheet_equity_new(
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_omit_html_equity_begin_row(
         char *BALANCE_SHEET_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_omit_html_transaction_amount_row(
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_omit_html_net_income_row(
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_omit_html_equity_end_row(
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_omit_html_liability_plus_equity_row(
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

BALANCE_SHEET_SUBCLASS_AGGR_HTML (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_AGGR_HTML *
   balance_sheet_subclass_aggr_html_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_OMIT_HTML *
   balance_sheet_subclass_display_html_calloc(
         void );

STATEMENT_SUBCLASS_OMIT_HTML_LIST *
   statement_subclass_aggr_html_list =
         statement_subclass_aggr_html_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_aggr_html_list_extract_row_list(
         statement_subclass_aggr_html_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_aggr_html_equity_row_list(
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```



BALANCE_SHEET_SUBCLASS_AGGR_HTML (2)

```
/* Usage */
LIST *balance_sheet_subclass_aggr_html_equity_row_list(
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *
   balance_sheet_equity =
         balance_sheet_equity_new(
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_aggr_html_equity_begin_row(
         char *BALANCE_SHEET_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_aggr_html_transaction_amount_row(
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_aggr_html_net_income_row(
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_aggr_html_equity_end_row(
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   HTML_ROW *balance_sheet_subclass_aggr_html_liability_plus_equity_row(
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

BALANCE_SHEET_PDF

```
/* Usage */
BALANCE_SHEET_PDF *
   balance_sheet_pdf_new(
         char *application_name,
         char *process_name,
         char *document_root_directory,
         enum statement_subclassification_option
               statement_subclassification_option,
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label(),
         pid_t process_id );

/* Process */
BALANCE_SHEET_PDF *balance_sheet_pdf_calloc( void );

ELEMENT *element_liability =
   element_seek(
         ELEMENT_LIABILITY,
         statement->element_statement_list );

boolean statement_pdf_landscape_boolean(
   list_length( statement_prior_year_list )
         /* statement_prior_year_list_length */ );

STATEMENT_LINK *statement_link =
   statement_link_new(
         application_name,
         process_name,
         document_root_directory,
         statement->transaction_begin_date_string,
         statement->transaction_as_of_date,
         (char *)0 /* statement_pdf_preclose_key */,
         process_id );

BALANCE_SHEET_LATEX *
   balance_sheet_latex =
         balance_sheet_latex_new(
               statement_subclassification_option,
               statement_link->tex_filename,
               statement_link->dvi_filename,
               statement_link->working_directory,
               statement_pdf_landscape_boolean(),
               statement->statement_logo_filename,
               statement,
               statement_prior_year_list,
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income,
               income_statement_net_income_label() );
```

statement_link

balance_sheet_latex

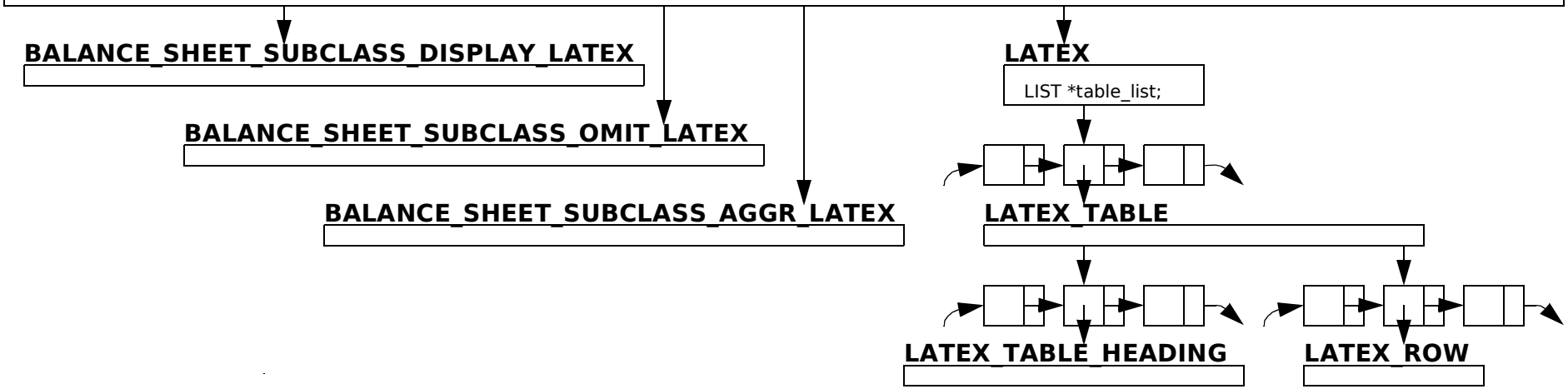
STATEMENT_LINK

```
char *tex_filename;
char *dvi_filename;
char *working_directory;
```

BALANCE_SHEET_LATEX

BALANCE_SHEET_LATEX (1)

```
/* Usage */
BALANCE_SHEET_LATEX *
   balance_sheet_latex_new(
         enum statement_subclassification_option
               statement_subclassification_option,
         char *statement_link->tex_filename,
         char *statement_link->dvi_filename,
         char *statement_link->working_directory,
         boolean statement_pdf_landscape_boolean(),
         char *statement_logo_filename(),
         STATEMENT *statement,
         LIST *statement_prior_year_list,
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_LATEX *balance_sheet_latex_calloc( void );

LATEX *latex =
   latex_new(
         tex_filename,
         dvi_filename,
         working_directory,
         statement_pdf_landscape_boolean,
         statement_logo_filename );


if (statement_subclassification_option == subclassification_display )
{
   BALANCE_SHEET_SUBCLASS_DISPLAY_LATEX *
         balance_sheet_subclass_display_latex =
               balance_sheet_subclass_display_latex_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *balance_sheet_latex_table(
               statement->statement_caption->combined,
               balance_sheet_subclass_display_latex->
                     statement_subclass_display_latex_list->
                     heading_list,
               balance_sheet_subclass_display_latex->
                     row_list ) );
}
```

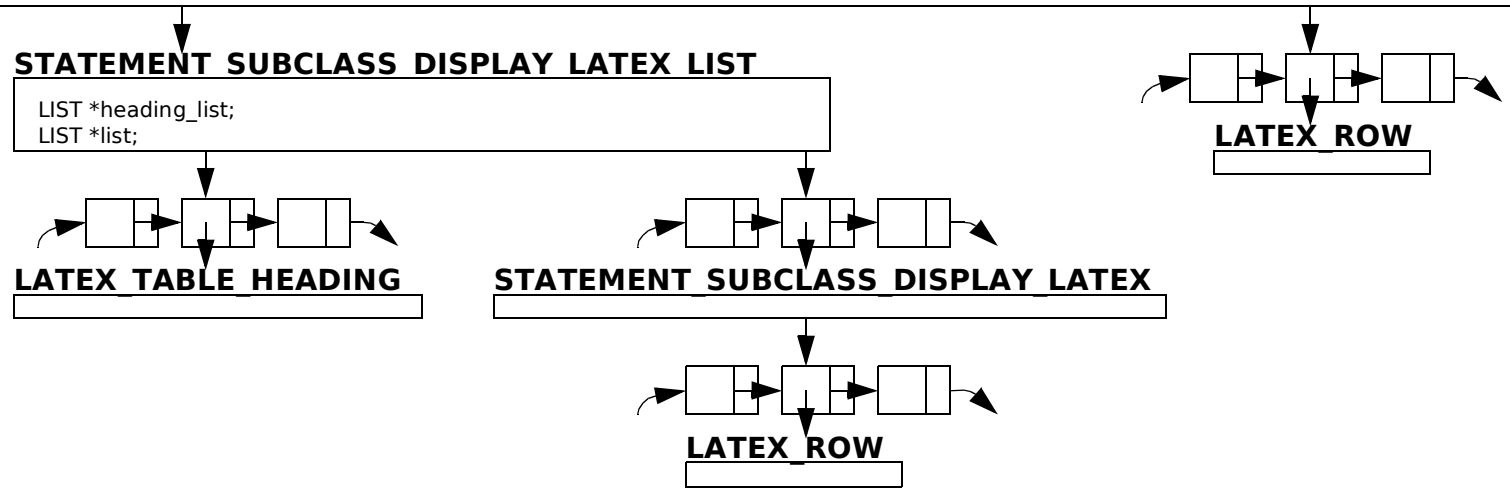


BALANCE_SHEET_LATEX (2)

```
/* Process (continued) */
else
if ( statement_subclassification_option == subclassification_omit )
{
   BALANCE_SHEET_SUBCLASS_OMIT_LATEX *
         balance_sheet_subclass_omit_latex =
               balance_sheet_subclass_omit_latex_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *balance_sheet_latex_table(
               statement->statement_caption->combined,
               balance_sheet_subclass_omit_latex->
               statement_subclass_omit_latex_list->
                     heading_list,
               balance_sheet_subclass_omit_latex->
                     row_list ) );
}
else
if ( statement_subclassification_option == subclassification_aggregate )
{
   BALANCE_SHEET_SUBCLASS_AGGR_LATEX *
         balance_sheet_subclass_aggr_latex =
               balance_sheet_subclass_aggr_latex_new(
                     statement->element_statement_list,
                     statement_prior_year_list,
                     element_equity_begin,
                     element_equity_current,
                     element_liability,
                     income_statement_fetch_net_income,
                     income_statement_net_income_label );

   list_set(
         latex->table_list,
         LATEX_TABLE *balance_sheet_latex_table(
               statement->statement_caption->combined,
               balance_sheet_subclass_aggr_latex->
                     statement_subclass_aggr_latex_list->
                     heading_list,
               balance_sheet_subclass_aggr_latex->
                     row_list ) );
}

/* Usage */
LATEX_TABLE *
   balance_sheet_latex_table(
         char *statement_caption_combined,
         LIST *heading_list,
         LIST *row_list );

/* Process */
LATEX_TABLE *latex_table = latex_table_new( statement_caption_combined );
latex_table->heading_list = heading_list;
latex_table->row_list = row_list;
```

BALANCE_SHEET_SUBCLASS_DISPLAY_LATEX (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_DISPLAY_LATEX *
   balance_sheet_subclass_display_latex_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_DISPLAY_LATEX *
   balance_sheet_subclass_display_latex_calloc(
         void );

STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST *
   statement_subclass_display_latex_list =
         statement_subclass_display_latex_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_display_latex_list_extract_row_list(
         statement_subclass_display_latex_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_display_latex_equity_row_list(
         list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */,
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```



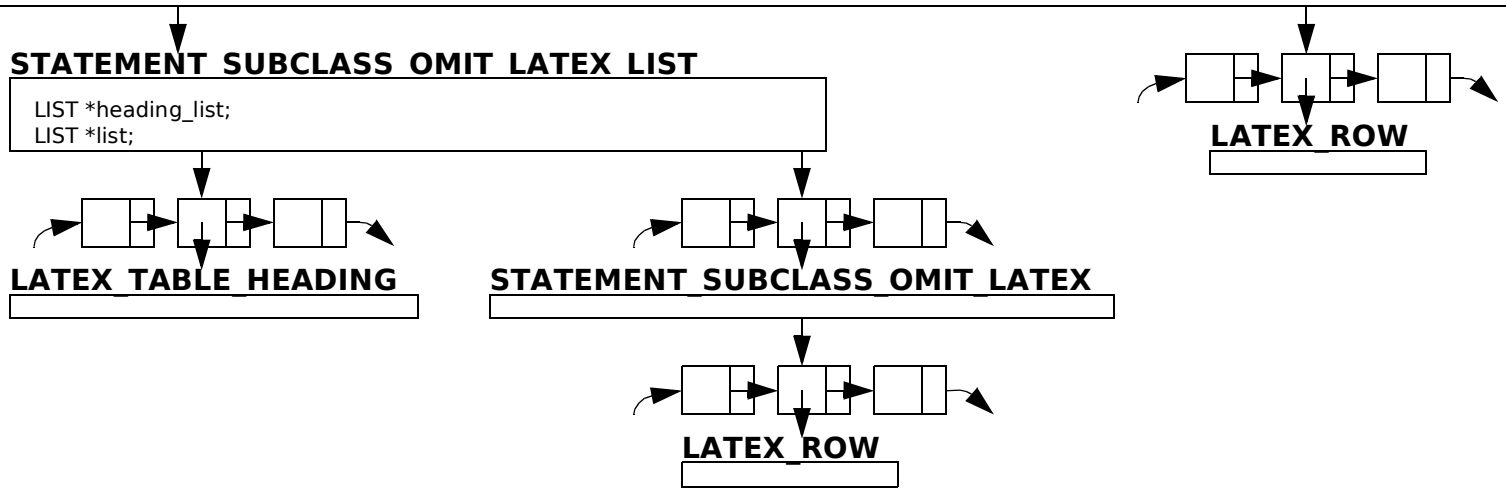
BALANCE_SHEET_SUBCLASS_DISPLAY_LATEX (2)

```
/* Usage */
LIST *balance_sheet_subclass_display_latex_equity_row_list(
   int statement_prior_year_list_length,
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *
   balance_sheet_equity =
         balance_sheet_equity_new(
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_display_latex_equity_begin_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_EQUITY_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_display_latex_transaction_amount_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_display_latex_net_income_row(
         int statement_prior_year_list_length,
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_display_latex_equity_end_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_display_latex_liability_plus_equity_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

BALANCE_SHEET_EQUITY

BALANCE_SHEET_SUBCLASS_OMIT_LATEX (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_OMIT_LATEX *
   balance_sheet_subclass_omit_latex_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_OMIT_LATEX *
   balance_sheet_subclass_omit_latex_calloc(
         void );

STATEMENT_SUBCLASS_OMIT_LATEX_LIST *
   statement_subclass_omit_latex_list =
         statement_subclass_omit_latex_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_omit_latex_list_extract_row_list(
         statement_subclass_omit_latex_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_omit_latex_equity_row_list(
         list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */,
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```

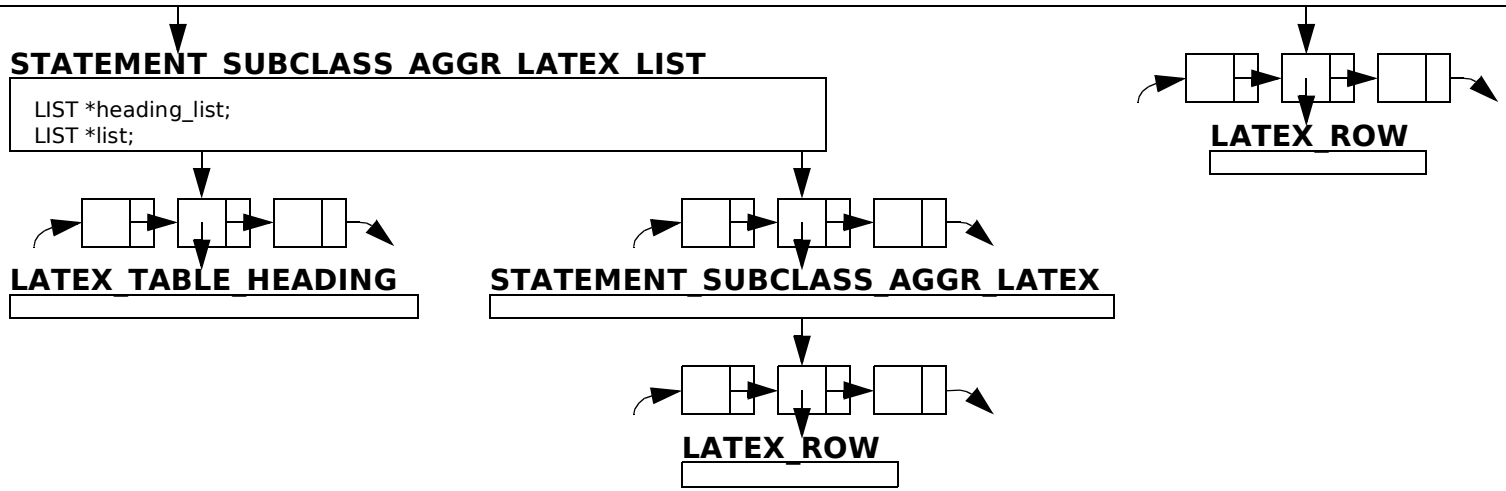


BALANCE_SHEET_SUBCLASS_OMIT_LATEX (2)

```
/* Usage */
LIST *balance_sheet_subclass_omit_latex_equity_row_list(
   int statement_prior_year_list_length,
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *
   balance_sheet_equity =
         balance_sheet_equity_new(
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_omit_latex_equity_begin_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_omit_latex_transaction_amount_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_omit_latex_net_income_row(
         int statement_prior_year_list_length,
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_omit_latex_equity_end_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_omit_latex_liability_plus_equity_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

BALANCE_SHEET_SUBCLASS_AGGR_LATEX (1)

```
/* Usage */
BALANCE_SHEET_SUBCLASS_AGGR_LATEX *
   balance_sheet_subclass_aggr_latex_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list(),
         ELEMENT *element_equity_begin,
         ELEMENT *element_equity_current,
         ELEMENT *element_liability,
         double income_statement_fetch_net_income(),
         char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_SUBCLASS_AGGR_LATEX *
   balance_sheet_subclass_aggr_latex_calloc(
         void );

STATEMENT_SUBCLASS_AGGR_LATEX_LIST *
   statement_subclass_aggr_latex_list =
         statement_subclass_aggr_latex_list_new(
               element_statement_list,
               statement_prior_year_list );

LIST *row_list =
   LIST *statement_subclass_aggr_latex_list_extract_row_list(
         statement_subclass_aggr_latex_list );

list_set_list(
   row_list,
   LIST *balance_sheet_subclass_aggr_latex_equity_row_list(
         list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */,
         element_equity_begin,
         element_equity_current,
         element_liability,
         income_statement_fetch_net_income,
         income_statement_net_income_label ) );
```



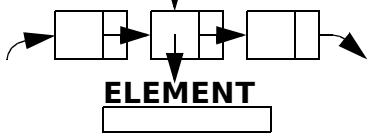
BALANCE_SHEET_SUBCLASS_AGGR_LATEX (2)

```
/* Usage */
LIST *balance_sheet_subclass_aggr_latex_equity_row_list(
   int statement_prior_year_list_length,
   ELEMENT *element_equity_begin,
   ELEMENT *element_equity_current,
   ELEMENT *element_liability,
   double income_statement_fetch_net_income(),
   char *income_statement_net_income_label() );

/* Process */
BALANCE_SHEET_EQUITY *
   balance_sheet_equity =
         balance_sheet_equity_new(
               element_equity_begin,
               element_equity_current,
               element_liability,
               income_statement_fetch_net_income );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_aggr_latex_equity_begin_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_BEGIN_BALANCE_LABEL,
         double balance_sheet_equity->begin_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_aggr_latex_transaction_amount_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_TRANSACTION_AMOUNT_LABEL,
         double balance_sheet_equity->transaction_amount ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_aggr_latex_net_income_row(
         int statement_prior_year_list_length,
         double income_statement_fetch_net_income,
         char *income_statement_net_income_label ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_aggr_latex_equity_end_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_END_BALANCE_LABEL,
         double balance_sheet_equity->end_balance ) );

list_set(
   row_list,
   LATEX_ROW *balance_sheet_subclass_aggr_latex_liability_plus_equity_row(
         int statement_prior_year_list_length,
         char *BALANCE_SHEET_LIABILITY_PLUS_EQUITY_LABEL,
         double balance_sheet_equity->liability_plus_equity ) );
```

STATEMENT (1)

```
/* Usage */
STATEMENT *statement_fetch(
      char *application_name,
      char *process_name,
      LIST *element_name_list,
      char *transaction_begin_date_string(),
      char *transaction_as_of_date(),
      char *transaction_date_time_closing(),
      boolean fetch_transaction );

/* Process */
STATEMENT *statement_calloc( void );

LIST *element_statement_list(
      element_name_list,
      transaction_date_time_closing,
      1 /* fetch_subclassification_list */,
      1 /* fetch_account_list */,
      1 /* fetch_journal_latest */,
      fetch_transaction );

void element_list_sum(
      element_statement_list() );

void element_percent_of_asset_set(
   LIST *element_statement_list );

void element_percent_of_revenue_set(
   LIST *element_statement_list );

if ( application_name )
{
      STATEMENT_CAPTION *statement_caption =
            statement_caption_new(
                  application_name,
                  process_name,
                  transaction_begin_date_string,
                  transaction_as_of_date );
}

/* Usage */
char *statement_date_american(
      char *date_time_string );

/* Process */
void date_convert_source_international(
      char date_american[],
      american,
      column(
            char date_string[],
            0,
            date_time_string ) );

char *strdup( date_american );
```

element_statement_list()



STATEMENT_CAPTION

```
/* Usage */
STATEMENT_CAPTION *statement_caption_new(
      char *application_name,
      char *process_name,
      char *begin_date_string,
      char *end_date_string );

/* Process */
STATEMENT_CAPTION *statement_caption_calloc(
      void );

/* Attributes */
char *begin_date_string;
char *end_date_string;

char *statement_caption_logo_filename(
      char *application_name,
      char *STATEMENT_LOGO_FILENAME_KEY );

char *statement_caption_title(
      application_name,
      (statement_caption_logo_filename())
            ? 1 : 0 /* exists_logo_filename */,
      process_name );

char *statement_caption_subtitle(
      statement_date_american(
            transaction_begin_date_string )
                  /* begin_date_string */,
      statement_date_american(
            transaction_as_of_date )
                  /* as_of_date */ );

char *statement_caption_combined(
      char *statement_caption_title(),
      char *statement_caption_subtitle() );

char *statement_caption_date_time_string(
      date_now_yyyy_mm_dd_hh_mm(
            date_utc_offset()
                  /* date_time_string */ ) );

char *statement_caption_frame_title(
      char *process_name,
      statement_caption_date_time_string() );
```

STATEMENT (2)

```
/* Attributes */
char *transaction_begin_date_string;
char *transaction_as_of_date;
char *transaction_date_time_closing;

/* Public */
enum statement_subclassification_option
      statement_resolve_subclassification_option(
            char *subclassification_option_string );

enum statement_output_medium
      statement_resolve_output_medium(
            char *output_medium_string );

boolean statement_pdf_landscape_boolean(
   int list_length( statement_prior_year_list )
            /* statement_prior_year_list_length */ );

char *statement_element_name(
   char *element_name );

char *statement_cell_data_label(
   char *name );

char *statement_pdf_prompt(
   char *process_name );

char *statement_subclass_display_name(
   char *element_name,
   char *subclassification->subclassification_name );

/* Driver */
void statement_latex_output(
   LATEX *latex,
   char *statement_link->ftp_output_filename,
   char *statement_pdf_prompt( process_name ),
   char *process_name,
   char *statement->
         statement_caption->
         date_time_string );

/* Process */
void latex_longtable_output(
   latex->output_stream,
   latex->landscape_flag,
   latex->table_list,
   latex->logo_filename,
   0 /* not omit_page_numbers */,
   date_time_string /* footline */ );

void fclose( latex->output_stream );

void latex_tex2pdf(
         latex->tex_filename,
         latex->working_directory );

void appaserver_library_output_ftp_prompt(
   ftp_output_filename,
   prompt,
   process_name /* target_frame */,
   (char *)0 /* mime_type */ );

/* Driver */
void statement_html_output(
   HTML_TABLE *html_table,
   char *title );
```

statement_subclassification_option

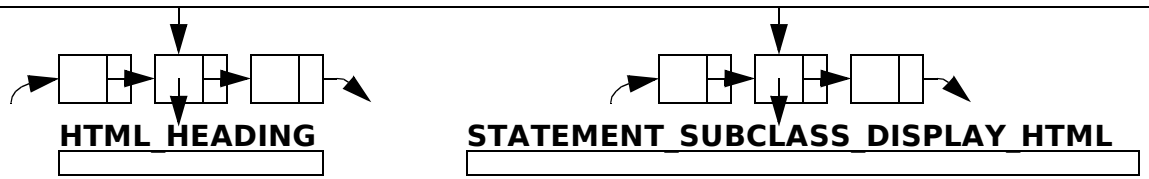
subclassification_aggregate,
subclassification_display,
subclassification_omit

statement_output_medium

statement_output_table,
statement_output_PDF,
statement_output_spreadsheet,
statement_output_stdout

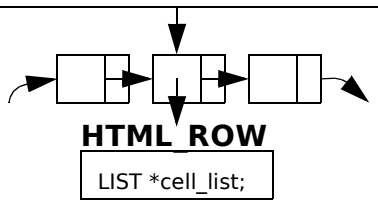
STATEMENT_SUBCLASS_DISPLAY_HTML_LIST

```
/* Usage */
STATEMENT_SUBCLASS_DISPLAY_HTML_LIST *
   statement_subclass_display_html_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_DISPLAY_HTML_LIST *
   statement_subclass_display_html_list_calloc(
         void );

LIST *statement_subclass_display_html_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_display_html_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_display_html_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Account”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Subclassification”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Element”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Standardized”,
         1 /* right_justified_boolean */ ) );;

void html_heading_list_right_justify_set(
   heading_list,
   statement_prior_year_heading_list(
         statement_prior_year_list ) );

/* Pubic */
LIST *statement_subclass_display_html_list_extract_row_list(
   STATEMENT_SUBCLASS_DISPLAY_HTML_LIST *
         statement_subclass_display_html_list );
```



STATEMENT_SUBCLASS_DISPLAY_HTML (1)

```
/* Usage */
STATEMENT_SUBCLASS_DISPLAY_HTML *
   statement_subclass_display_html_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_DISPLAY_HTML *
   statement_subclass_display_html_calloc(
         void );

list_set(
   row_list,
   HTML_ROW *statement_subclass_display_html_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for SUBCLASSIFICATION *subclassification in
   element->subclassification_statement_list
{
   if ( !subclassification->sum ) continue;

   char *statement_subclass_display_name(
         element_name,
         subclassification->subclassification_name );

   list_set(
         row_list,
         HTML_ROW *statement_subclass_display_html_subclassification_label_row(
               char *statement_subclass_display_name(),
               int statement_prior_year_list_length ) );

   list_set_list(
         row_list,
         statement_subclass_display_html_account_row_list(
               subclassification->account_statement_list,
               statement_prior_year_list ) );

   list_set(
         row_list,
         HTML_ROW *statement_subclass_display_html_subclassification_sum_row(
               char *subclassification->subclassification_name,
               double subclassification->sum,
               int subclassification->percent_of_asset,
               int subclassification->percent_of_revenue,
               LIST *statement_prior_year_list ) );
}

list_set(
   row_list,
   HTML_ROW *statement_subclass_display_html_element_sum_row(
         char *element->element_name,
         double element->sum,
         int percent_of_asset,
         int percent_of_revenue,
         LIST *statement_prior_year_list ) );

/* Usage */
LIST *statement_subclass_display_html_account_row_list(
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               (char *)0 /* transaction_as_of_date */,
               0 /* element_accumulate_debit */,
               account->account_journal_latest,
               account->action_string,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         HTML_ROW *statement_subclass_display_html_account_row(
               statement_account,
               statement_prior_year_list ) );
}
```



STATEMENT_SUBCLASS_DISPLAY_HTML (2)

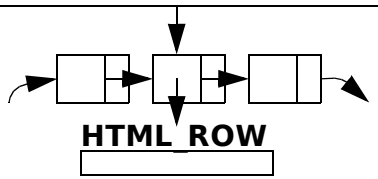
```
/* Usage */
HTML_ROW *statement_subclass_display_html_account_row(
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   char *statement_account->
         account->
         account_name );

void html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account->balance_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->
                     account->
                     account_name,
               statement_prior_year_list ) );
}
```

STATEMENT_SUBCLASS_OMIT_HTML_LIST

```
/* Usage */
STATEMENT_SUBCLASS_OMIT_HTML_LIST *
   statement_subclass_omit_html_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_OMIT_HTML_LIST *
   statement_subclass_omit_html_list_calloc(
         void );

LIST *statement_subclass_omit_html_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_omit_html_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_omit_html_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Account”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Element”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Standardized”,
         1 /* right_justified_boolean */ ) );

void html_heading_list_right_justify_set(
   heading_list,
   statement_prior_year_heading_list(
         statement_prior_year_list ) );

/* Pubic */
LIST *statement_subclass_omit_html_list_extract_row_list(
   STATEMENT_SUBCLASS_OMIT_HTML_LIST *
         statement_subclass_omit_html_list );
```

STATEMENT_SUBCLASS_OMIT_HTML

```
/* Usage */
STATEMENT_SUBCLASS_OMIT_HTML *
   statement_subclass_omit_html_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_OMIT_HTML *
   statement_subclass_omit_html_calloc(
         void );

list_set(
   row_list,
   HTML_ROW *
   statement_subclass_omit_html_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for ACCOUNT *account in
   element->account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               (char *)0 /* transaction_as_of_date */,
               0 /* element_accumulate_debit */,
               account->account_journal_latest,
               account->action_string,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         statement_subclass_omit_html_account_row(
               statement_account,
               statement_prior_year_list ) );
}

list_set(
   row_list,
   HTML_ROW *
   statement_subclass_omit_html_element_sum_row(
         char *element->element_name,
         double element->sum,
         int element->percent_of_asset,
         int element->percent_of_revenue,
         LIST *statement_prior_year_list ) );

/* Usage */
HTML_ROW *
   statement_subclass_omit_html_account_row(
         STATEMENT_ACCOUNT *statement_account,
         LIST *statement_prior_year_list );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   char *statement_account->
         account->
         account_name );

void html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account->balance_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   “”,
   0 /* not large_boolean */,
   0 * not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->
                     account->
                     account_name,
               statement_prior_year_list ) );
}
```

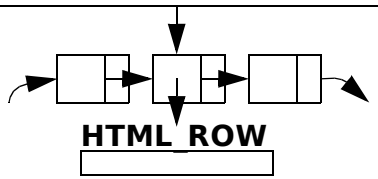


STATEMENT_SUBCLASS_AGGR_HTML_LIST

```
/* Usage */
STATEMENT_SUBCLASS_AGGR_HTML_LIST *
   statement_subclass_aggr_html_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_AGGR_HTML_LIST *
   statement_subclass_aggr_html_list_calloc(
         void );

LIST *statement_subclass_aggr_html_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_aggr_html_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_aggr_html_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “”,
         0 /* right_justify_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Balance”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Element”,
         1 /* right_justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Standardized”,
         1 /* right_justified_boolean */ ) );

void html_heading_list_right_justify_set(
   heading_list,
   statement_prior_year_heading_list(
         statement_prior_year_list ) );

/* Pubic */
LIST *statement_subclass_aggr_html_list_extract_row_list(
   STATEMENT_SUBCLASS_AGGR_HTML_LIST *
         statement_subclass_aggr_html_list );
```

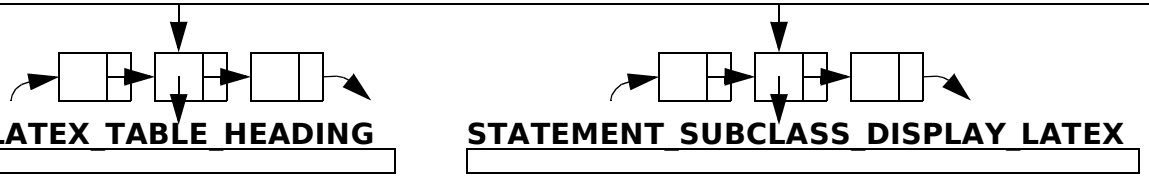
STATEMENT_SUBCLASS_AGGR_HTML

```
/* Usage */
STATEMENT_SUBCLASS_AGGR_HTML *
   statement_subclass_aggr_html_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_AGGR_HTML *
   statement_subclass_aggr_html_calloc(
         void );

list_set(
   row_list,
   HTML_ROW *
   statement_subclass_aggr_html_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for SUBCLASSIFICATION *subclassification in
   element->subclassification_statement_list
{
   if ( subclassification->sum )
   {
         list_set(
               row_list,
               statement_subclass_aggr_html_row(
                     subclassifiction,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   HTML_ROW *
   statement_subclass_aggr_html_element_sum_row(
         char *element->element_name,
         double element->sum,
         int element->percent_of_asset,
         int element->percent_of_revenue,
         LIST *statement_prior_year_list ) );

/* Usage */
HTML_ROW *
   statement_subclass_aggr_html_row(
         SUBCLASSIFICATION *subclassification,
         LIST *statement_prior_year_list );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   char *subclassification->subclassification_name );

void html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account_balance_string(
         subclassification->sum,
         (char *)0 /* account_action_string */,
         0 /* not round_dollar_boolean */ ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_account_percent_string(
         subclassification->percent_of_asset,
         subclassification->percent_of_revenue ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void html_cell_data_list_set(
         html_row->cell_list,
         statement_prior_year_subclassification_data_list(
               subclassification->subclassification_name,
               statement_prior_year_list ) );
}
```



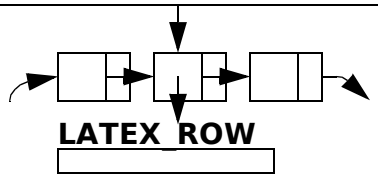
STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST

```
/* Usage */
STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST *
   statement_subclass_display_latex_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST *
   statement_subclass_display_latex_list_calloc(
         void );

LIST *statement_subclass_display_latex_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_display_latex_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_display_latex_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   LATEX_TABLE_HEADING *latex_table_heading_new(
         “”,
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ );

list_set(
   heading_list,
   LATEX_TABLE_HEADING *latex_table_heading_new(
         “Account”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   LATEX_TABLE_HEADING *latex_table_heading_new(
         “Subclassification”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   LATEX_TABLE_HEADING *latex_table_heading_new(
         “Element”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   LATEX_TABLE_HEADING *latex_table_heading_new(
         “Standardized”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set_list(
   heading_list,
   LIST *latex_table_right_heading_list(
         statement_prior_year_heading_list(
               statement_prior_year_list ) ) );

/* Pubic */
LIST *statement_subclass_display_latex_list_extract_row_list(
   STATEMENT_SUBCLASS_DISPLAY_LATEX_LIST *
         statement_subclass_display_latex_list );
```



STATEMENT_SUBCLASS_DISPLAY_LATEX (1)

```
/* Usage */
STATEMENT_SUBCLASS_DISPLAY_LATEX *
   statement_subclass_display_latex_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_DISPLAY_LATEX *
   statement_subclass_display_latex_calloc(
         void );

list_set(
   row_list,
   LATEX_ROW *statement_subclass_display_latex_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for SUBCLASSIFICATION *subclassification in
   element->subclassification_statement_list
{
   if ( !subclassification->sum ) continue;

   char *statement_subclass_display_name(
         element_name,
         subclassification->subclassification_name );

   list_set(
         row_list,
         LATEX_ROW *statement_subclass_display_latex_subclassification_label_row(
               char *statement_subclass_display_name(),
               int statement_prior_year_list_length ) );

   list_set_list(
         row_list,
         statement_subclass_display_latex_account_row_list(
               subclassification->account_statement_list,
               statement_prior_year_list ) );

   list_set(
         row_list,
         LATEX_ROW *statement_subclass_display_latex_subclassification_sum_row(
               char *subclassification->subclassification_name,
               double subclassification->sum,
               int subclassification->percent_of_asset,
               int subclassification->percent_of_revenue,
               LIST *statement_prior_year_list );
}

list_set(
   row_list,
   LATEX_ROW *statement_subclass_display_latex_element_sum_row(
         char *element->element_name,
         double element->sum,
         int percent_of_asset,
         int percent_of_revenue,
         LIST *statement_prior_year_list );

/* Usage */
LIST *statement_subclass_display_latex_account_row_list(
   LIST *account_statement_list,
   LIST *statement_prior_year_list );

/* Process */
for ACCOUNT *account in account_statement_list
{
   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               (char *)0 /* transaction_as_of_date */,
               0 /* element_accumulate_debit */,
               account->account_journal_latest,
               (char *)0 /* account_action_string */,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         LATEX_ROW *statement_subclass_display_latex_account_row(
               statement_account,
               statement_prior_year_list ) );
}
```



STATEMENT_SUBCLASS_DISPLAY_LATEX (2)

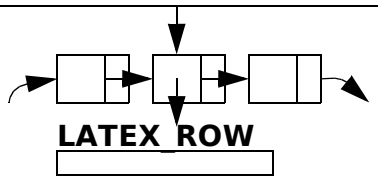
```
/* Usage */
LATEX_ROW *statement_subclass_display_latex_account_row(
   STATEMENT_ACCOUNT *statement_account,
   LIST *statement_prior_year_list );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   char *statement_account->
         account->
         account_name );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->balance_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->
                     account->
                     account_name,
               statement_prior_year_list ) );
}
```

STATEMENT_SUBCLASS_OMIT_LATEX_LIST

```
/* Usage */
STATEMENT_SUBCLASS_OMIT_LATEX_LIST *
   statement_subclass_omit_latex_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_OMIT_LATEX_LIST *
   statement_subclass_omit_latex_list_calloc(
         void );

LIST *statement_subclass_omit_latex_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_omit_latex_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_omit_latex_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “”,
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ );

list_set(
   heading_list,
   latex_table_heading_new(
         “Account”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Element”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Standardized”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set_list(
   heading_list,
   LIST *latex_table_right_heading_list(
         LIST *statement_prior_year_heading_list(
               statement_prior_year_list ) ) );

/* Pubic */
LIST *statement_subclass_omit_latex_list_extract_row_list(
   STATEMENT_SUBCLASS_OMIT_LATEX_LIST *
         statement_subclass_omit_latex_list );
```

STATEMENT_SUBCLASS_OMIT_LATEX

```
/* Usage */
STATEMENT_SUBCLASS_OMIT_LATEX *
   statement_subclass_omit_latex_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_OMIT_LATEX *
   statement_subclass_omit_latex_calloc(
         void );

list_set(
   row_list,
   LATEX_ROW *
   statement_subclass_omit_latex_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for ACCOUNT *account in
   element->account_statement_list
{
   if ( !account->sum ) continue;

   STATEMENT_ACCOUNT *statement_account =
         statement_account_new(
               (char *)0 /* transaction_as_of_date */,
               0 /* element_accumulate_debit */,
               account->account_journal_latest,
               (char *)0 /* account_action_string */,
               0 /* not round_dollar_boolean */,
               account );

   list_set(
         row_list,
         statement_subclass_omit_latex_account_row(
               statement_account,
               statement_prior_year_list ) );
}

list_set(
   row_list,
   LATEX_ROW *
   statement_subclass_omit_latex_element_sum_row(
         char *element->element_name,
         double element->sum,
         int element->percent_of_asset,
         int element->percent_of_revenue,
         LIST *statement_prior_year_list ) );

/* Usage */
LATEX_ROW *
   statement_subclass_omit_latex_account_row(
         STATEMENT_ACCOUNT *statement_account,
         LIST *statement_prior_year_list );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   char *statement_account->
         account->
         account_name );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->balance_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   “”,
   0 /* not large_boolean */,
   0 * not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account->percent_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         LIST *statement_prior_year_account_data_list(
               statement_account->
                     account->
                     account_name,
               statement_prior_year_list ) );
}
```

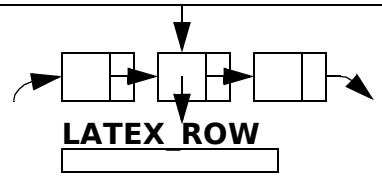


STATEMENT_SUBCLASS_AGGR_LATEX_LIST

```
/* Usage */
STATEMENT_SUBCLASS_AGGR_LATEX_LIST *
   statement_subclass_aggr_latex_list_new(
         LIST *element_statement_list,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_AGGR_LATEX_LIST *
   statement_subclass_aggr_latex_list_calloc(
         void );

LIST *statement_subclass_aggr_latex_list_heading_list(
   statement_prior_year_list );

for ELEMENT *element in element_statement_list
{
   if ( element->sum )
   {
         list_set(
               list,
               statement_subclass_aggr_latex_new(
                     element,
                     statement_prior_year_list ) );
   }
}

/* Usage */
LIST *statement_subclass_aggr_latex_list_heading_list(
   LIST *statement_prior_year_list );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “”,
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Balance”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Element”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Standardized”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set_list(
   heading_list,
   LIST *latex_table_right_heading_list(
         LIST *statement_prior_year_heading_list(
               statement_prior_year_list ) ) );

/* Pubic */
LIST *statement_subclass_aggr_latex_list_extract_row_list(
   STATEMENT_SUBCLASS_AGGR_LATEX_LIST *
         statement_subclass_aggr_latex_list );
```

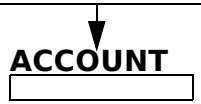
STATEMENT_SUBCLASS_AGGR_LATEX

```
/* Usage */
STATEMENT_SUBCLASS_AGGR_LATEX *
   statement_subclass_aggr_latex_new(
         ELEMENT *element,
         LIST *statement_prior_year_list );

/* Process */
STATEMENT_SUBCLASS_AGGR_LATEX *
   statement_subclass_aggr_latex_calloc(
         void );

list_set(
   row_list,
   LATEX_ROW *
   statement_subclass_aggr_latex_element_label_row(
         char *element->element_name,
         int list_length( statement_prior_year_list )
               /* statement_prior_year_list_length */ ) );

for SUBCLASSIFICATION *subclassification in
   element->subclassification_statement_list
{
   if ( subclassification->sum )
   {
         list_set(
               row_list,
               statement_subclass_aggr_latex_row(
                     subclassification,
                     statement_prior_year_list ) );
   }
}

list_set(
   row_list,
   LATEX_ROW *
   statement_subclass_aggr_latex_element_sum_row(
         char *element->element_name,
         double element->sum,
         int element->percent_of_asset,
         int element->percent_of_revenue,
         LIST *statement_prior_year_list ) );

/* Usage */
LATEX_ROW *
   statement_subclass_aggr_latex_row(
         SUBCLASSIFICATION *subclassification,
         LIST *statement_prior_year_list );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   char *subclassification->subclassification_name );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account_balance_string(
         subclassification->sum,
         (char *)0 /* account_action_string */,
         0 /* not round_dollar_boolean */ ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   “”,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_account_percent_string(
         subclassification->percent_of_asset,
         subclassification->percent_of_revenue ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

if ( list_length( statement_prior_year_list ) )
{
   void latex_column_data_list_set(
         latex_row->column_data_list,
         statement_prior_year_subclassification_data_list(
               subclassification->subclassification_name,
               statement_prior_year_list ) );
}
```



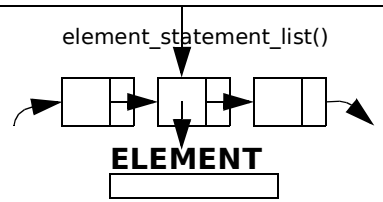
STATEMENT_ACCOUNT

```
/* Usage */
STATEMENT_ACCOUNT *
    statement_account_new(
        char *transaction_as_of_date(),
        boolean element_accumulate_debit,
        ACCOUNT_JOURNAL *account_journal_latest,
        char *account_action_string,
        boolean round_dollar_boolean,
        ACCOUNT *account );

/* Process */
STATEMENT_ACCOUNT *statement_account_calloc(
    void );

double balance = account_journal_latest->balance;

if ( balance < 0.0 )
{
    balance = -balance;
    element_accumulate_debit =
        1 - element_accumulate_debit;
}

char *statement_account_balance_string(
    balance,
    account_action_string,
    round_dollar_boolean );

if ( element_accumulate_debit )
{
    debit_string =
        statement_account_balance_string();
}
else
{
    credit_string =
        statement_account_balance_string();
}

char *statement_account_asset_percent_string(
    int account->percent_of_asset );

char *statement_account_revenue_percent_string(
    int account->percent_of_revenue );

char *statement_account_percent_string(
    int account->percent_of_asset,
    int account->percent_of_revenue );

if ( transaction_as_of_date )
{
    int date_days_between(
        account_journal_latest->
            transaction_date_time,
        transaction_as_of_date );

    boolean within_days_between_boolean =
        (date_days_between() <=
        STATEMENT_DAYS_FOR_EMPHASIS);
}

/* Usage */
char *statement_account_balance_string(
    double balance,
    char *account_action_string,
    boolean round_dollar_boolean );

/* Process */
char *statement_account_money_string(
    double balance,
    boolean round_dollar_boolean );

if ( account_action_string )
{
    sprintf(  balance_string[],
              “<a href=\”%s\”>%s</a>”,
              account_action_string,
              statement_account_money_string() );
}
else
{
    strcpy(   balance_string[],
              statement_account_money_string() );
}

char *strdup( balance_string );

/* Public attributes */
char *balance_string;
char *debit_string;
char *credit_string;
char *asset_percent_string;
char *revenue_percent_string;
char *percent_string;
ACCOUNT *account;
```



STATEMENT_PRIOR_YEAR (1)

```
/* Usage */
LIST *statement_prior_year_list(
    LIST *element_name_list,
    char *transaction_date_time_closing(),
    int prior_year_count,
    STATEMENT *statement );

/* Process */
for years_ago = 1 to prior_year_count
{
    list_set(
        prior_year_list,
        STATEMENT_PRIOR_YEAR *
            statement_prior_year_fetch(
                element_name_list,
                transaction_date_time_closing,
                years_ago,
                statement->
                    element_statement_list ) );
}

/* Usage */
STATEMENT_PRIOR_YEAR *
    statement_prior_year_fetch(
        LIST *element_name_list,
        char *transaction_date_time_closing(),
        int years_ago,
        LIST *current_element_statement_list );

/* Process */
STATEMENT_PRIOR_YEAR *
    statement_prior_year_calloc(
        void );

char *statement_prior_year_date_time_string(
    transaction_date_time_closing,
    years_ago );

LIST *element_statement_list(
    element_name_list,
    statement_prior_year_date_time_string(),
    1 /* fetch_subclassification_list */,
    1 /* fetch_account_list */,
    1 /* fetch_journal_latest *,
    0 /* not fetch_transaction */ );

void element_list_sum( element_statement_list() );

for ELEMENT *current_element in
    current_element_statement_list
{
    ELEMENT *prior_element =
        element_seek(
            current_element->element_name,
            element_statement_list() );

    if ( prior_element )
    {
        void element_delta_prior_percent_set(
            prior_element /* in/out */,
            current_element );
    }
}

/* Usage */
char *statement_prior_year_date_time_string(
    char *transaction_date_time_closing,
    int years_ago );

/* Process */
char *date_time_string =
    date_display19(
        date_subtract_year(
            date_19new(
                transaction_date_time_closing ),
            years_ago ) );

/* Usage */
LIST *statement_prior_year_account_data_list(
    char *account_name,
    LIST *statement_prior_year_list() );

/* Process */
for STATEMENT_PRIOR_YEAR *statement_prior_year in
    statement_prior_year_list
{
    ACCOUNT *prior_account =
        account_element_list_seek(
            account_name,
            statement_prior_year->
                element_statement_list );

    list_set(
        data_list,
        statement_prior_year_account_data(
            prior_account ) );
}

/* Usage */
char *statement_prior_year_account_data(
    ACCOUNT *prior_account );

/* Process */
if (  !prior_account
||    !prior_account->account_journal_latest->balance )
{
    boolean cell_empty = 1;
    int delta_prior_percent = 0;
    double prior_year_amount = 0.0;
}
else
{
    int delta_prior_percent =
        prior_account->delta_prior_percent;
    double prior_year_amount =
        prior_account->
            account_journal_latest->
            balance;
}

char *statement_prior_year_cell_display(
    cell_empty,
    delta_prior_percent,
    prior_year_amount );
```



STATEMENT_PRIOR_YEAR (2)

```
/* Usage */
LIST *statement_prior_year_subclassification_data_list(
	char *subclassification_name,
	LIST *statement_prior_year_list() );

/* Process */
for STATEMENT_PRIOR_YEAR *statement_prior_year in
	statement_prior_year_list
{
	SUBCLASSIFICATION *prior_subclassification =
		subclassification_element_list_seek(
			subclassification_name,
			statement_prior_year->
				element_statement_list );

	list_set(
		data_list,
		statement_prior_year_subclassification_data(
			prior_subclassification ) );
}

/* Usage */
char *statement_prior_year_subclassification_data(
	SUBCLASSIFICATION *prior_subclassification );

/* Process */
if (	!prior_subclassification
||	!prior_subclassification->sum )
{
	boolean cell_empty = 1;
}
else
{
	int delta_prior_percent =
		prior_subclassification->delta_prior_percent;
	double prior_year_amount =
		prior_subclassification->sum;
}

char *statement_prior_year_cell_display(
	cell_empty,
	delta_prior_percent,
	prior_year_amount );

/* Usage */
LIST *statement_prior_year_element_data_list(
	char *element_name,
	LIST *statement_prior_year_list() );

/* Process */
for STATEMENT_PRIOR_YEAR *statement_prior_year in
	statement_prior_year_list
{
	ELEMENT *prior_element =
		element_seek(
			element_name,
			statement_prior_year->
				element_statement_list );

	list_set(
		data_list,
		statement_prior_year_element_data(
			prior_element ) );
}

/* Usage */
char *statement_prior_year_element_data(
	ELEMENT *prior_element );

/* Process */
if ( !prior_element || !prior_element->sum )
{
	boolean cell_empty = 1;
}
else
{
	int delta_prior_percent =
		prior_element->delta_prior_percent;
	double prior_year_amount =
		prior_element->sum;
}

char *statement_prior_year_cell_display(
	cell_empty,
	delta_prior_percent,
	prior_year_amount );

/* Usage */
char *statement_prior_year_cell_display(
	boolean cell_empty,
	int delta_prior_percent,
	double prior_year_amount );

/* Process */
if ( cell_empty )
{
	*cell_display[] = '\0';
}
else
{
	sprintf(	cell_display[],
			"%d%c %s",
			delta_prior_percent,
			'%',
			timlib_place_commas_in_dollars(
				prior_year_amount );
}

strdup( cell_display );

/* Public */
LIST *statement_prior_year_heading_list(
	LIST *statement_prior_year_list );
```

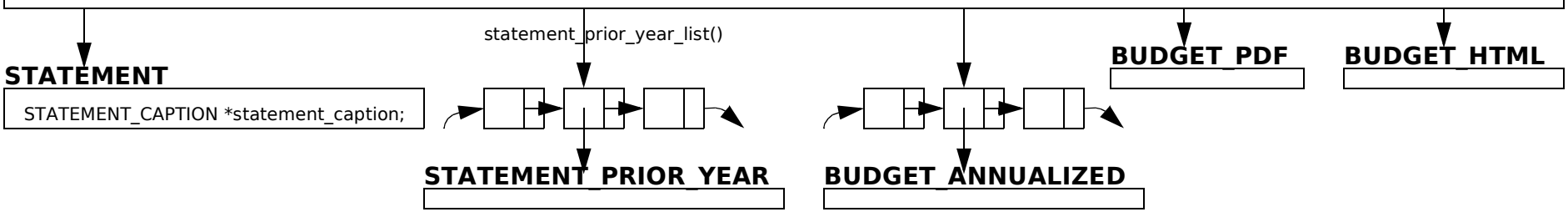
STATEMENT_LINK

```
/* Usage */
STATEMENT_LINK *statement_link_new(
   char *application_name,
   char *process_name,
   char *document_root_directory,
   char *transaction_begin_date_string(),
   char *transaction_as_of_date(),
   char *preclose_key,
   pid_t process_id );

/* Process */
STATEMENT_LINK *statement_link_calloc( void );

char *statement_link_filename_stem(
   char *process_name,
   char *preclose_key );

APPASERVER_LINK_FILE *appaserver_link_file =
   appaserver_link_file_new(
         application_http_prefix( application_name ),
               appaserver_library_get_server_address(),
               ( application_prepend_http_protocol_yn(
                     application_name ) == 'y' ),
               document_root_directory,
               statement_link_filename_stem(),
               application_name,
               process_id,
               (char *)0 /* session */,
               (char *)0 /* extension */ );

appaserver_link_file->extension = "tex";

appaserver_link_file->begin_date_string =
   transaction_begin_date_string;

appaserver_link_file->end_date_string =
   transaction_as_of_date();

char *tex_filename =
   strdup(
         appaserver_link_get_tail_half(
               (char *)0 /* application_name */,
               appaserver_link_file->filename_stem,
               appaserver_link_file->begin_date_string,
               appaserver_link_file->end_date_string,
               appaserver_link_file->process_id,
               appaserver_link_file->session,
               appaserver_link_file->extension ) );

appaserver_link_file->extension = "dvi";

char *dvi_filename =
   strdup(
         appaserver_link_get_tail_half(
               (char *)0 /* application_name */,
               appaserver_link_file->filename_stem,
               appaserver_link_file->begin_date_string,
               appaserver_link_file->end_date_string,
               appaserver_link_file->process_id,
               appaserver_link_file->session,
               appaserver_link_file->extension ) );

appaserver_link_file->extension = "pdf";

char *ftp_output_filename =
   appaserver_link_get_link_prompt(
         appaserver_link_file->
               link_prompt->
               prepend_http_boolean,
         appaserver_link_file->
               link_prompt->
               http_prefix,
         appaserver_link_file->
               link_prompt->
               server_address,
         appaserver_link_file->application_name,
         appaserver_link_file->filename_stem,
         appaserver_link_file->begin_date_string,
         appaserver_link_file->end_date_string,
         appaserver_link_file->process_id,
         appaserver_link_file->session,
         appaserver_link_file->extension );

char *working_directory =
   appaserver_link_source_directory(
         document_root_directory,
         application_name );
```

APPASERVER_LINK_FILE

BUDGET (1)

```
/* Usage */
BUDGET *budget_fetch(
   char *application_name,
   char *process_name,
   char *document_root_directory,
   char *as_of_date_string,
   char *output_medium_string );

/* Process */
BUDGET *budget_calloc( void );

enum statement_output_medium =
   statement_resolve_output_medium(
         output_medium_string );

char *transaction_as_of_date(
   TRANSACTION_TABLE,
   as_of_date_string );

DATE *budget_as_of_date(
   char *transaction_as_of_date() );

char *transaction_begin_date_string(
   TRANSACTION_TABLE,
   transaction_as_of_date() );

DATE *budget_begin_date(
   char *transaction_begin_date_string() );

int budget_days_so_far(
   DATE *budget_begin_date,
   DATE *budget_as_of_date );

int budget_days_in_year(
   DATE *budget_begin_date );

double budget_year_ratio(
   int budget_days_so_far(),
   int budget_days_in_year() );


LIST *budget_element_name_list(
   char *ELEMENT_REVENUE,
   char *ELEMENT_EXPENSE );

boolean transaction_closing_entry_exists(
   TRANSACTION_TABLE,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date() );

char *transaction_date_time_closing(
   TRANSACTION_PRECLOSE_TIME,
   TRANSACTION_CLOSE_TIME,
   transaction_as_of_date(),
   transaction_closing_entry_exists()
         /* preclose_time_boolean */ );

STATEMENT *statement =
   statement_fetch(
         application_name,
         process_name,
         budget_element_name_list(),
         transaction_begin_date_string(),
         transaction_as_of_date(),
         transaction_date_time_closing(),
         0 /* not fetch_transaction */ );

void element_list_account_statement_list_set(
   statement->element_statement_list );

char *budget_end_date_time_string(
   char *transaction_begin_date_string() );

LIST *statement_prior_year_list(
         budget_element_name_list(),
         budget_end_date_time_string(),
         1 /* prior_year_count */,
         statement );

LIST *budget_annualized_list(
   budget_year_ratio(),
   statement->element_statement_list,
   list_first( statement_prior_year_list() )
         /* statement_prior_year */ );

double budget_annualized_net_asset_change(
   budget_annualized_list() );
```



BUDGET (2)

```
/* Process (continued) */
if ( statement_output_medium ==
        statement_output_PDF )
{
   char *budget_year_percent_subtitle(
        statement->
             statement_caption->
             combined,
        budget_year_ratio() );

   BUDGET_PDF *budget_pdf =
        budget_pdf_new(
             application_name,
             process_name,
             document_root_directory,
             statement->transaction_begin_date_string,
             statement->transaction_as_of_date,
             statement->
                  statement_caption->
                  logo_filename,
             budget_year_percent_subtitle(),
             budget_annualized_list(),
             budget_annualized_net_asset_change(),
             getpid() /* process_id */ );
}
else
if ( statement_output_medium ==
        statement_output_table )
{
   char *budget_year_percent_subtitle(
        statement->
             statement_caption->
             subtitle,
             budget_year_ratio() );

   BUDGET_HTML *budget_html =
        budget_html_new(
             budget_year_percent_subtitle(),
             budget_annualized_list(),
             budget_annualized_net_asset_change() );
}

/* Usage */
char *budget_display(
   double budget );

/* Process */
if ( budget )
{
   return
   timlib_place_commas_in_money( budget );
}
else
{
   return (char *)0;
}

/* Usage */
char *budget_year_percent_subtitle(
   char *input_subtitle,
   double budget_year_ratio() );

/* Process */
sprintf(     subtitle[],
             "%s; Year Percent: %d%c",
             input_subtitle,
             float_ratio_to_percent(
                  budget_year_ratio ),
             '%' );

/* Driver */
printf( "%s\n",
        statement->
        statement_caption->
        frame_title );

if ( budget_pdf )
{
   void statement_latex_output(
        budget_pdf->
             budget_latex->
             latex,
        budget_pdf->
             statement_link->
             ftp_output_filename,
        statement_pdf_prompt( process_name ),
        process_name,
        statement->
             statement_caption->
             date_time_string );
}
else
if ( budget_html )
{
   void statement_html_output(
        budget_html->table,
             (char *)0 /* secondary_title */ );
}
```

BUDGET_ANNUALIZED

```
/* Usage */
LIST *budget_annualized_list(
   double budget_year_ratio(),
   LIST *element_statement_list,
   STATEMENT_PRIOR_YEAR *statement_prior_year );

/* Process */
for ELEMENT *element in element_statement_list
{
   for ACCOUNT *account in
         element->account_statement_list
   {
         list_set(
               list,
               BUDGET_ANNUALIZED *
                     budget_annualized_new(
                           budget_year_ratio,
                           element->element_name,
                           account,
                           statement_prior_year ) );
   }
}

/* Usage */
BUDGET_ANNUALIZED *budget_annualized_new(
   double budget_year_ratio,
   char *element_name,
   ACCOUNT *account,
   STATEMENT_PRIOR_YEAR *statement_prior_year );

/* Process */
BUDGET_ANNUALIZED *budget_annualized_calloc(
   void );

/* Attributes */
char *element_name;
ACCOUNT *account;

double budget_annualized_account_amount(
   double account->account_journal_latest->balance
         /* account_latest_balance */,
   int account->annual_budget );

double budget_annualized_amount(
   double budget_annualized_account_amount(),
   double budget_year_ratio );

if ( statement_prior_year )
{
   ACCOUNT *prior_account =
         account_element_list_seek(
               account->account_name,
               statement_prior_year->
                     element_statement_list );

   if ( prior_account )
   {
         double budget_annualized_budget(
               double prior_account->
                     account_journal_latest->
                     balance
                           /* prior_account_balance */ );

         double budget_annualized_difference(
               double budget_annualized_amount(),
               double budget_annualized_budget() );

         int budget_annualized_difference_percent(
               double budget_annualized_budget(),
               double budget_annualized_difference() );

         char *budget_annualized_difference_cell(
               double budget_annualized_difference(),
               int budget_annualized_difference_percent() );
   }
}

/* Usage */
double budget_annualized_net_asset_change(
   budget_annualized_list() );

/* Process */
double revenue_sum =
   budget_annualized_amount_sum(
         ELEMENT_REVENUE,
         budget_annualized_list );

double expense_sum =
   budget_annualized_amount_sum(
         ELEMENT_EXPENSE,
         budget_annualized_list );

return revenue_sum - expense_sum;

/* Usage */
double budget_annualized_amount_sum(
   char *element_name,
   LIST *budget_annualized_list );

/* Process */
```

BUDGET_HTML (1)

```
/* Usage */
BUDGET_HTML *budget_html_new(
   char *budget_year_percent_subtitle(),
   LIST *budget_annualized_list(),
   double  budget_annualized_net_asset_change() );

/* Process */
BUDGET_HTML *budget_html_calloc( void );

HTML_TABLE *budget_html_table(
   budget_year_percent_subtitle,
   budget_annualized_list,
   budget_annualized_net_asset_change );

/* Usage */
HTML_TABLE *budget_html_table(
   char *budget_year_percent_subtitle,
   LIST *budget_annualized_list,
   double budget_annualized_net_asset_change );

/* Process */
HTML_TABLE *html_table =
   html_table_new(
         (char *)0 /* title */,
         budget_year_percent_subtitle,
         (char *)0 /* sub_sub_title */ );

html_table->heading_list =
   budget_html_heading_list();

html_table->row_list =
   budget_html_row_list(
         budget_annualized_list );

list_set(
   html_table->row_list,
   HTML_ROW *budget_html_sum_row(
         double budget_annualized_net_asset_change ) );

/* Usage */
LIST *budget_html_heading_list( void );

/* Process */
list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Element”,
         0 /* not right_Justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Account”,
         0 /* not right_Justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Balance”,
         1 /* right_Justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Annualized”,
         1 /* right_Justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Budget”,
         1 /* right_Justified_boolean */ ) );

list_set(
   heading_list,
   HTML_HEADING *html_heading_new(
         “Difference”,
         1 /* right_Justified_boolean */ ) );

/* Usage */
LIST *budget_html_row_list(
   LIST *budget_annualized_list );

/* Process */
for BUDGET_ANNUALIZED *budget_annualized in
   budget_annualized_list
{
   list_set(
         row_list,
         HTML_ROW *budget_html_row(
               budget_annualized->element_name,
               budget_annualized->
                     account->
                     account_name,
               budget_annualized->account_amount,
               budget_annualized->amount,
               budget_annualized->budget,
               budget_annualized->difference_cell ) );
}
```

HTML_TABLE

```
LIST *heading_list;
LIST *row_list;
```

BUDGET_HTML (2)

```
/* Usage */
HTML_ROW *budget_html_row(
   char *element_name,
   char *account_name,
   double account_amount,
   double annualized_amount,
   double budget,
   char *difference_cell );

/* Process */
HTML_ROW *html_row = html_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *statement_cell_data_label(
   account_name /* name */ );

void html_cell_data_set(
   html_row->cell_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   timlib_place_commas_in_money(
         account_amount ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   timlib_place_commas_in_money(
         annualized_amount ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   char *budget_display( budget ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void html_cell_data_set(
   html_row->cell_list,
   difference_cell,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```

BUDGET_PDF

```
/* Usage */
BUDGET_PDF *budget_pdf_new(
   char *application_name,
   char *process_name,
   char *document_root_directory,
   char *statement->transaction_begin_date_string,
   char *statement->transaction_as_of_date,
   char *statement_logo_filename,
   char *budget_year_percent_subtitle(),
   LIST *budget_annualized_list(),
   double  budget_annualized_net_asset_change(),
   pid_t process_id );

/* Process */
BUDGET_PDF *budget_pdf_calloc(
   void );

STATEMENT_LINK *statement_link =
   statement_link_new(
         application_name,
         process_name,
         document_root_directory,
         transaction_begin_date_string,
         transaction_as_of_date,
         (char *)0 /* statement_pdf_preclose_key */,
         process_id );

BUDGET_LATEX *budget_latex =
   budget_latex_new(
         statement_link->tex_filename,
         statement_link->dvi_filename,
         statement_link->working_directory,
         statement_logo_filename,
         budget_year_percent_subtitle,
         budget_annualized_list,
         budget_annualized_net_asset_change() );
```

statement_link

budget_latex

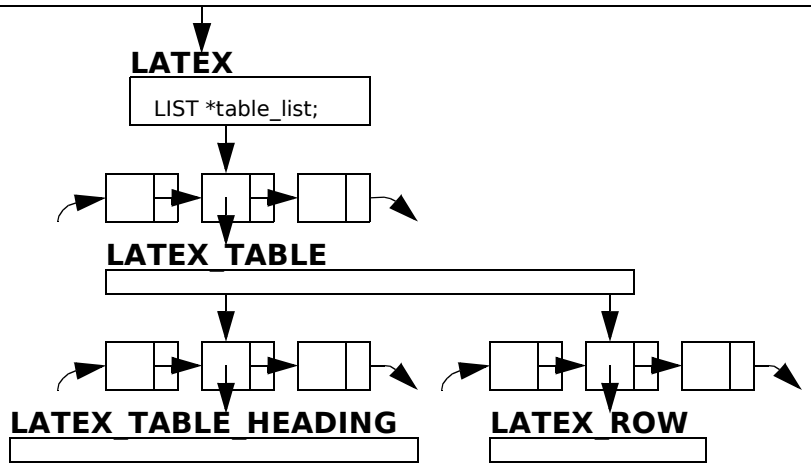
STATEMENT_LINK

```
char *tex_filename;
char *dvi_filename;
char *working_directory;
```

BUDGET_LATEX

BUDGET_LATEX (1)

```
/* Usage */
BUDGET_LATEX *budget_latex_new(
   char *statement_link->tex_filename,
   char *statement_link->dvi_filename,
   char *statement_link->working_directory,
   char *statement_logo_filename,
   char *budget_year_percent_subtitle(),
   LIST *budget_annualized_list,
   double  budget_annualized_net_asset_change() );

/* Process */
BUDGET_LATEX *budget_latex_calloc( void );

LATEX *latex =
   latex_new(
         tex_filename,
         dvi_filename,
         working_directory,
         0 /* not landscape_boolean */,
         statement_logo_filename );

list_set(
   latex->table_list,
   LATEX_TABLE *budget_latex_table(
         budget_year_percent_subtitle,
         budget_annualized_list,
         budget_annualized_net_asset_change ) );

/* Usage */
LATEX_TABLE *budget_latex_table(
   char *budget_year_percent_subtitle,
   LIST *budget_annualized_list,
   double  budget_annualized_net_asset_change() );

/* Process */
LATEX_TABLE *latex_table =
   latex_table_new(
         budget_year_percent_subtitle );

latex_table->heading_list =
   LIST *budget_latex_heading_list();

latex_table->row_list =
   LIST *budget_latex_row_list(
         budget_annualized_list );

list_set(
   latex_table->row_list,
   LATEX_ROW *budget_latex_sum_row(
         double budget_annualized_net_asset_change ) );

/* Usage */
LIST *budget_latex_heading_list( void );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “Element”,
         0 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Account”,
         0 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Balance”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Annualized”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Budget”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “Difference”,
         1 /* right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );
```



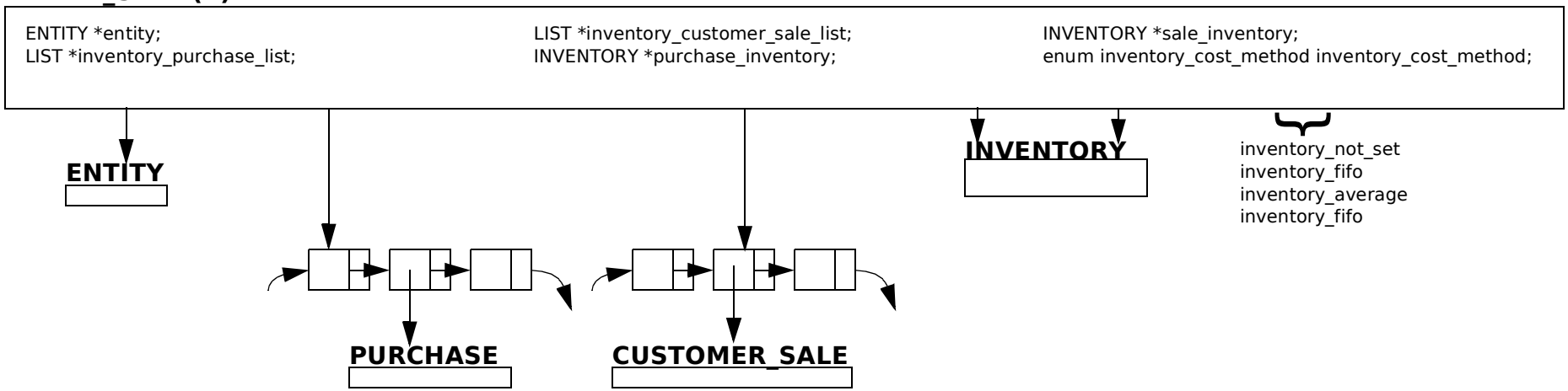
BUDGET_LATEX (2)

```
/* Usage */
LIST *budget_latex_row_list(
   LIST *budget_annualized_list() );

/* Process */
for BUDGET_ANNUALIZED *budget_annualized in budget_annualized_list
{
   list_set(
         row_list,
         LATEX_ROW *budget_latex_row(
               budget_annualized->element_name,
               budget_annualized->account->account_name,
               budget_annualized->account_amount,
               budget_annualized->amount,
               budget_annualized->budget,
               budget_annualized->difference_cell ) );
}

/* Usage */
LATEX_ROW *budget_latex_row(
   char *element_name,
   char *account_name,
   double account_amount,
   double annualized_amount,
   double budget,
   char *difference_cell );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

char *statement_cell_data_label(
   element_name /* name */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

char *statement_cell_data_label(
   account_name /* name */ );

void latex_column_data_set(
   latex_row->column_data_list,
   statement_cell_data_label(),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   timlib_place_commas_in_money( account_amount ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   timlib_place_commas_in_money( annualized_amount ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   char *budget_display( budget ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

void latex_column_data_set(
   latex_row->column_data_list,
   difference_cell,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```

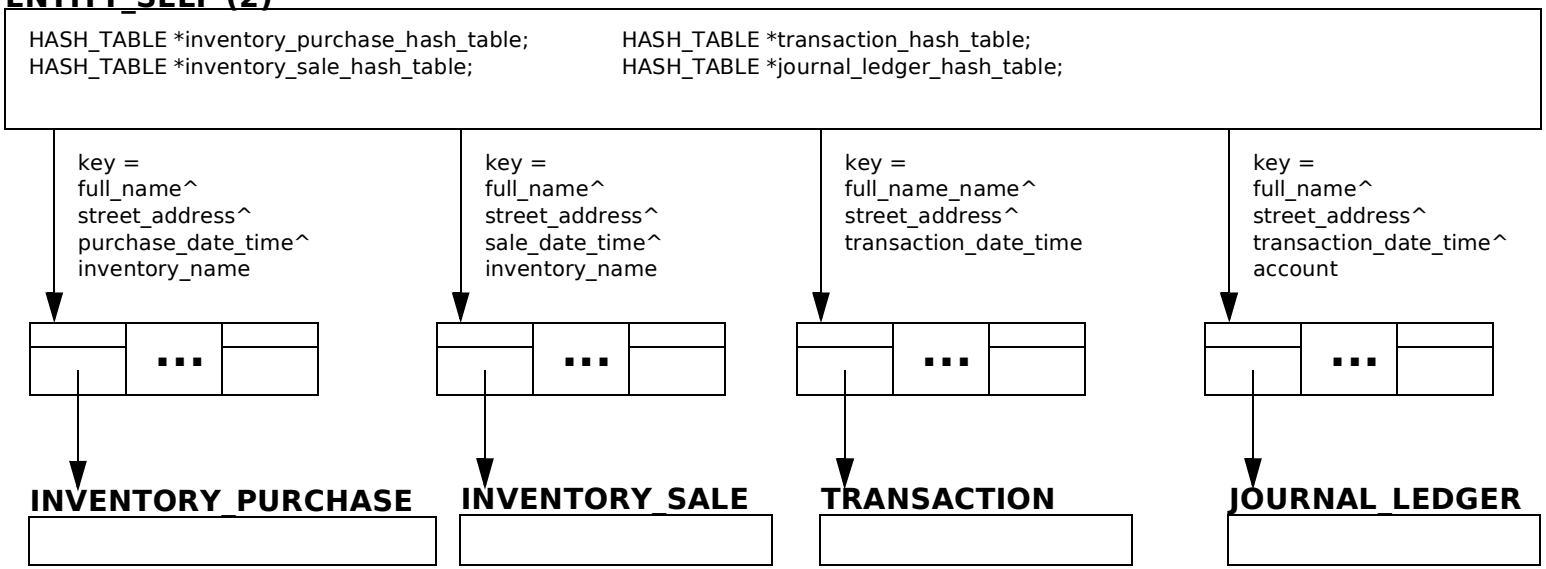
ENTITY

```
/* Usage */
LIST *entity_system_list(
	char *system_string );

/* Process */
FILE *entity_input_pipe(
	char *system_string );

for char input[] in char *string_input( entity_input_pipe()
{
	list_set(
		entity_list,
		entity_parse( input ) );
}

pclose( entity_input_pipe() );

/* Usage */
ENTITY *entity_fetch(
	char *full_name,
	char *street_address );

/* Process */
char *entity_primary_where(
	char *full_name,
	char *street_address );

char *entity_system_string(
	char *ENTITY_SELECT,
	char *ENTITY_TABLE,
	char *entity_primary_where );

FILE *entity_input_pipe( entity_system_string() );

ENTITY *entity_parse(
	char *string_input(
		entity_input_pipe() ) );

ENTITY *entity_new(
	char *strdup( full_name ),
	char *strdup( street_address ) );

ENTITY *entity_calloc( void );

pclose( entity_input_pipe() );

/* Attributes */
char *full_name;
char *street_address;
char *unit;
char *city;
char *state_code;
char *zip_code;
char *phone_number;
char *email_address;

/* Public */
LIST *entity_full_street_list(
	LIST *full_name_list,
	LIST *street_address_list );

char *entity_escape_full_name(
	char *full_name );

char *entity_street_address(
	char *full_name );

ENTITY *entity_full_name_seek(
	char *full_name );
```

ENTITY_SELF (1)

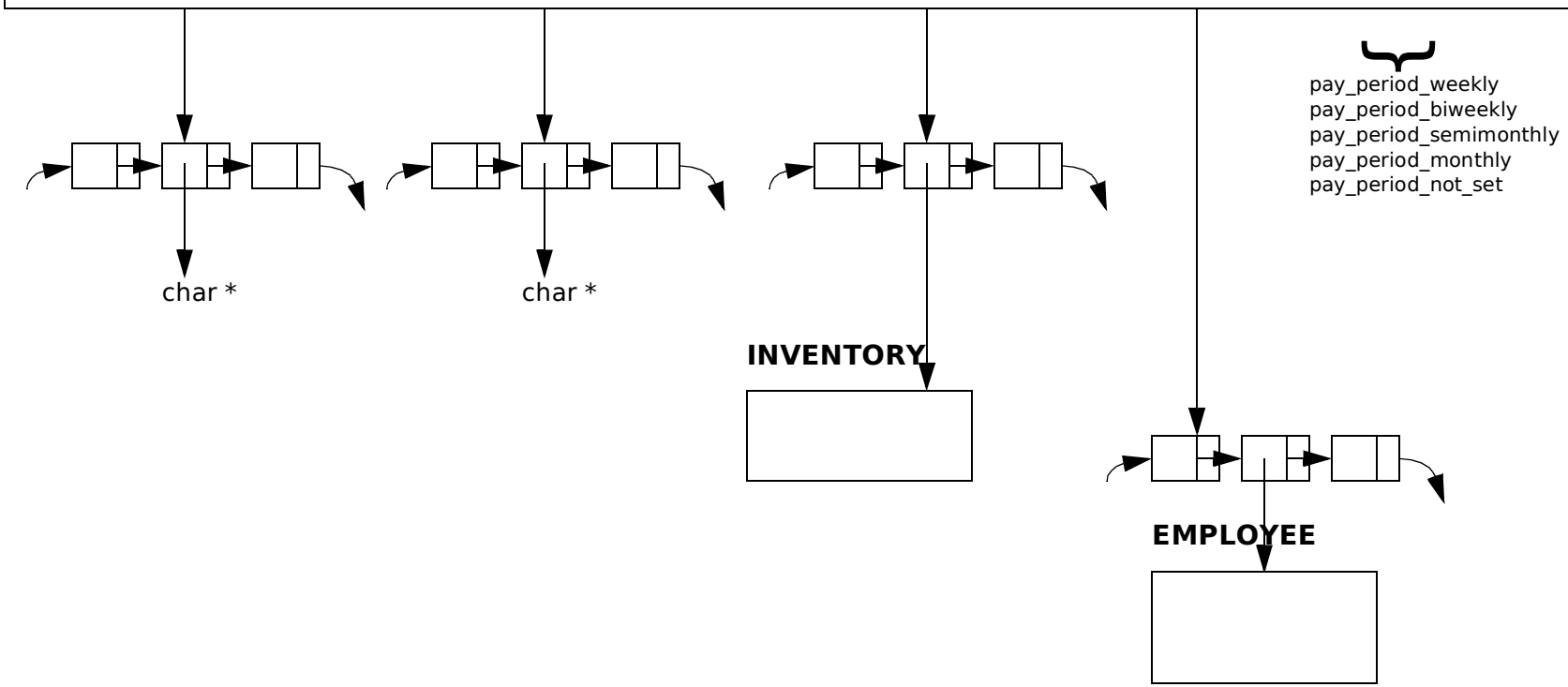


ENTITY_SELF (2)



ENTITY_SELF (3)

```
LIST *inventory_purchase_name_list;
LIST *inventory_sale_name_list;
LIST *inventory_list;
LIST *employee_list;
enum payroll_pay_period payroll_pay_period;
```



ENTITY_SELF (4)

```
char *payroll_beginning_day
double social_security_combined_tax_rate
int social_security_payroll_ceiling
double medicare_combined_tax_rate
double medicare_additional_withholding_rate
int medicare_additional_gross_pay_floor
double federal_withholding_allowance_period_value
double federal_nonresident_withholding_income_premium
double state_withholding_allowance_period_value
double state_itemized_allowance_period_value
int federal_unemployment_wage_base
double federal_unemployment_tax_standard_rate
double federal_unemployment_threshold_rate
double federal_unemployment_tax_minimum_rate
int state_unemployment_wage_base
double state_unemployment_tax_rate
double state_sales_tax_rate;

char *entity_self_paypal_cash_account_name();
```

SALE (1)

```
ENTITY *customer_entity =
      entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *completed_date_time;
double shipping_revenue;
enum title_passage_rule title_passage_rule;

/* Returns sale_steady_state() */
SALE *sale_fetch(
      full_name,
      street_address,
      sale_date_time );

char *sale_primary_where(
      full_name,
      street_address,
      sale_date_time );

double sale_gross_revenue(
      inventory_sale_total(),
      fixed_service_sale_total(),
      hourly_service_sale_total() );

double sale_sales_tax(
      inventory_sale_list(),
      entity_self_sales_tax_rate() );

double sale_invoice_amount(
      sale_gross_revenue(),
      sale_sales_tax(),
      shipping_charge );

double Sale_amount_due(
      sale_invoice_amount(),
      customer_payment_total() )
{
      return A - B;
}

TRANSACTION *sale_transaction(
      full_name,
      street_address,
      completed_date_time,
      invoice_amount,
      gross_revenue,
      sale_tax,
      shipping_charge,
      account_receivable(),
      account_revenue(),
      account_shipping_revenue(),
      account_sales_tax_payable() );

void sale_update(
      inventory_sale_total(),
      fixed_service_sale_total(),
      hourly_service_sale_total(),
      sale_gross_revenue()
      sale_sales_tax(),
      sale_invoice_amount(),
      payment_total(),
      sale_amount_due(),
      sale_transaction()->
            transaction_date_time,
      full_name,
      street_address,
      sale_date_time );

SALE *sale_steady_state(
      full_name,
      street_address,
      sale_date_time,
      inventory_sale_list(),
      fixed_service_sale_list(),
      hourly_service_sale_list(),
      customer_payment_list(),
      shipping_charge,
      entity_self_sales_tax_rate(),
      completed_date_time,
      account_receivable(),
      account_revenue(),
      account_shipping_revenue(),
      account_sales_tax_payable() );
```

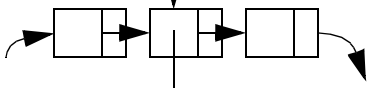
TRANSACTION

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

SALE (2)

```
LIST *customer_payment_list(
      full_name,
      street_address,
      sale_date_time );
```

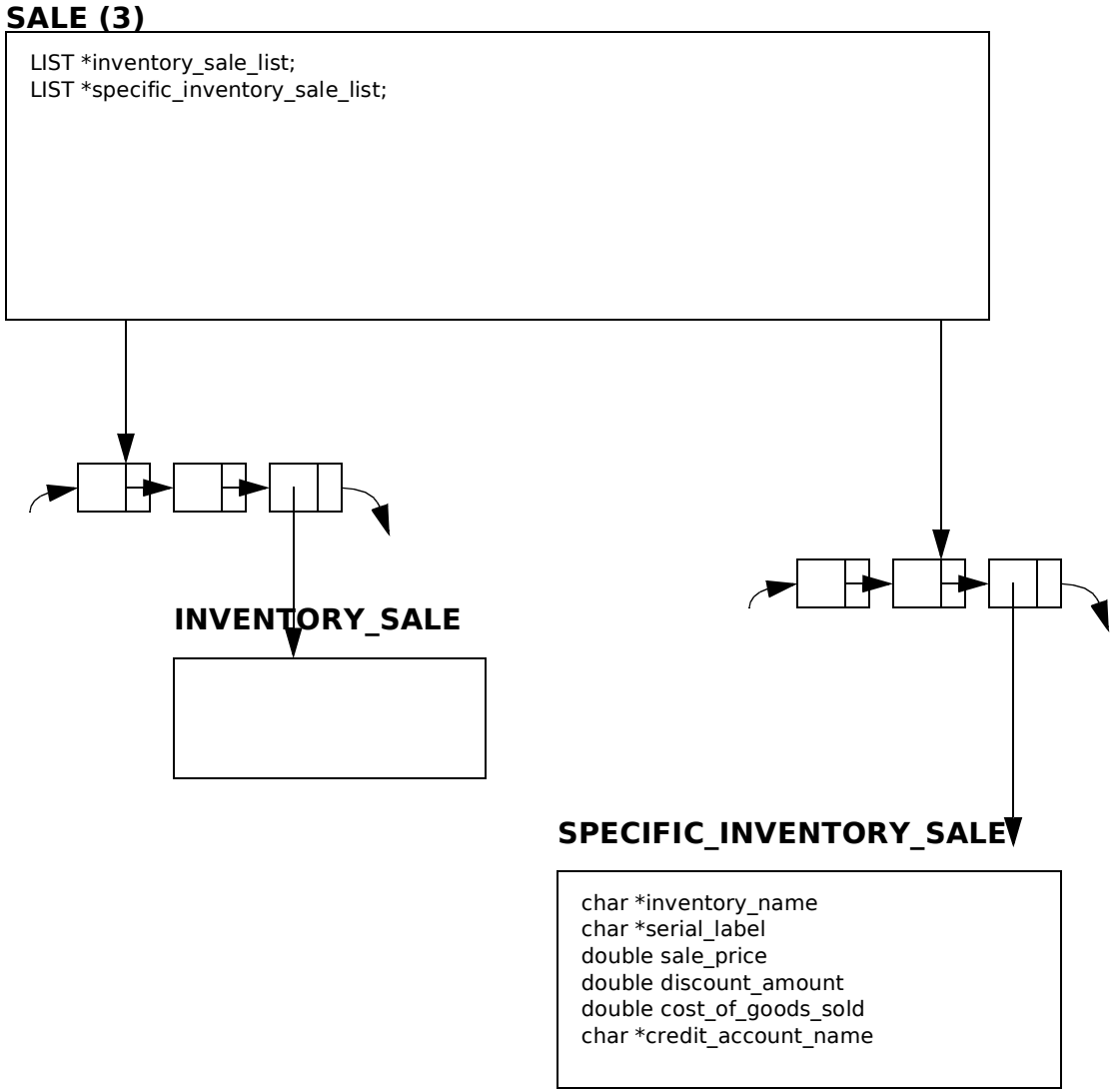


CUSTOMER_PAYMENT

```
ENTITY *customer_entity =
      entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

double customer_payment_total(
      customer_payment_list() )
{
      return sum( A->payment_amount );
}

CUSTOMER_PAYMENT *customer_payment_new(
      full_name,
      street_address,
      sale_date_time,
      payment_date_time );

char *customer_payment_select(
      void );

char *customer_payment_sys_string(
      where );

LIST *customer_payment_system_list(
      sys_string );

CUSTOMER_PAYMENT *customer_payment_parse(
      input );

TRANSACTION *customer_payment_transaction(
      full_name,
      street_address,
      payment_date_time,
      payment_amount,
      account_cash(),
      account_receivable() );
```

ENTITY

TRANSACTION


SALE (3)
LIST *inventory_sale_list;
LIST *specific_inventory_sale_list;
INVENTORY_SALE
SPECIFIC_INVENTORY_SALE
char *inventory_name
char *serial_label
double sale_price
double discount_amount
double cost_of_goods_sold
char *credit_account_name

HOURLY_SERVICE_SALE

```
ENTITY *customer_entity =
    entity_new(
        full_name,
        street_address );
char *sale_date_time;
char *service_name;
char *service_description;
double hourly_rate;
double discount_amount;
int estimated_hours;

char *hourly_service_sale_revenue_account_name(
    service_name,
    service_description );

LIST *hourly_service_work_list =
    work_hourly_list(
        full_name,
        street_address,
        sale_date_time,
        service_name,
        service_description );

double hourly_service_sale_work_hours =
    work_list_hours(
        hourly_service_work_list() );

double hourly_service_sale_net_revenue(
    hourly_rate,
    hourly_service_sale_work_hours,
    discount_amount )
{
    return (A x B) - C;
}
```

SALE (4)

```
LIST *    fixed_service_sale_list
LIST *    hourly_service_sale_list
```

FIXED_SERVICE_SALE

```
ENTITY *customer_entity =
    entity_new(
        full_name,
        street_address );
char *sale_date_time;
char *service_name;
double fixed_price;
double discount_amount;
int estimated_hours;
char *fixed_service_sale_revenue_accouint_name(
    service_name );

LIST *fixed_service_sale_work_list =
    work_fixed_list(
        full_name,
        street_address,
        sale_date_time,
        service_name );

double fixed_service_sale_work_hours =
    work_list_hours(
        fixed_service_sale_work_list() );

double fixed_service_sale_net_revenue(
    fixed_price,
    discount_amount )
{
    return A - B;
}
```

WORK

```
ENTITY *customer_entity = entity_new();
char *sale_date_time;
char *service_name;
char *service_description;
char *begin_date_time;
char *end_date_time;
char *login_name;
char *activity_name;

double work_hours(
    begin_date_time,
    end_date_time );

double work_list_hours(
    work_list );
```

PURCHASE (1)

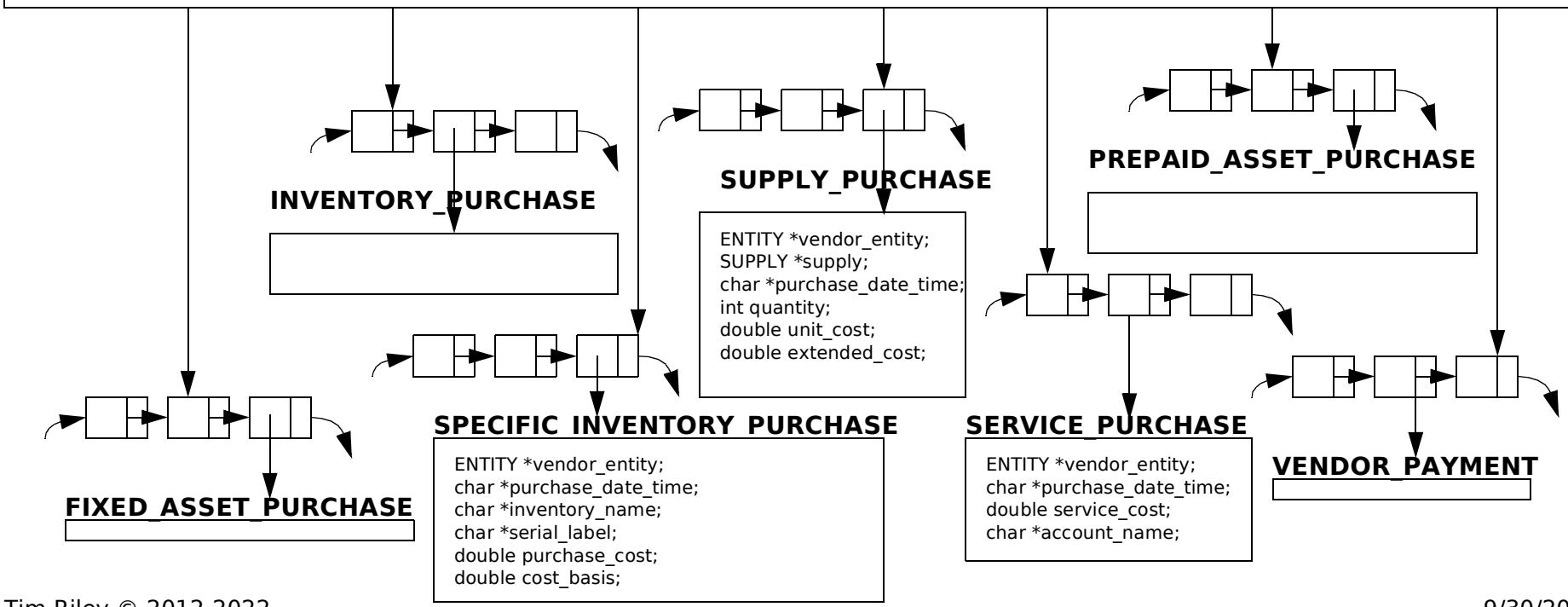
```
/* Usage */
PURCHASE *purchase_fetch(
    char *full_name,
    char *street_address,
    char *purchase_date_time );

/* Input */
ENTITY *vendor_entity =
    entity_new(
        full_name,
        street_address );

char *purchase_date_time;
double sales_tax;
double freight_in;
char *title_passage_rule_string;
char *shipped_date;
char *arrived_date_time;
char *program_name;
char *property_street_address;

/* Process */
enum title_passage_rule title_passage_rule =
    predictive_title_passage_rule_resolve(
        title_passage_rule_string );

char *purchase_primary_where(
    char *full_name,
    char *street_address,
    char *purchase_date_time );

char *purchase_system_string(
    char *where,
    char *order );

LIST *purchase_system_list(
    purchase_system_string() );

PURCHASE *purchase_parse(
    char *input );

LIST *fixed_asset_purchase_list =
    fixed_asset_purchase_list_fetch(
        vendor_entity->full_name,
        vendor_entity->street_address,
        purchase_date_time );

LIST *vendor_payment_list =
    vendor_payment_list_fetch(
        full_name,
        street_address,
        purchase_date_time );

PURCHASE *purchase_steady_state(
    sales_tax,
    freight_in,
    arrived_date,
    fixed_asset_purchase_list,
    inventory_purchase_list,
    specific_inventory_purchase_list,
    supply_purchase_list,
    prepaid_asset_purchase_list,
    inventory_purchase_return_list,
    vendor_payment_list,
    purchase );

double fixed_asset_purchase_total =
    fixed_asset_purchase_total(
        fixed_asset_purchase_list );

double inventory_purchase_total =
    inventory_purchase_total(
        inventory_purchase_list );

double specific_inventory_purchase_total =
    specific_inventory_purchase_total(
        specific_inventory_purchase_list );

double supply_purchase_total =
    supply_purchase_total(
        supply_purchase_list );

double prepaid_asset_purchase_total =
    prepaid_asset_purchase_total(
        prepaid_asset_purchase_list );

double inventory_purchase_return_total =
    inventory_purchase_return_total(
        inventory_purchase_return_list );

double vendor_payment_total =
    vendor_payment_total(
        vendor_payment_list );

double purchase_invoice_amount(
    fixed_asset_purchase_total,
    inventory_purchase_total,
    specific_inventory_purchase_total,
    supply_purchase_total,
    prepaid_asset_purchase_total,
    sales_tax,
    freight_in );

double purchase_amount_due(
        purchase_invoice_amount,
        vendor_payment_total )
{
    return A - B;
}

double purchase_asset_cost_basis(
        asset_cost,
        sales_tax,
        freight_in,
        fixed_asset_purchase_total(),
        inventory_purchase_total(),
        specific_inventory_purchase_total(),
        supply_expense_purchase_total(),
        prepaid_asset_purchase_total() );
```

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

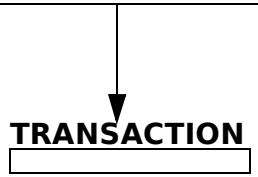
PURCHASE (2)

```
TRANSACTION *purchase_transaction =
    purchase_fixed_asset_transaction(
        full_name,
        street_address,
        arrived_date_time,
        purchase_invoice_amount,
        sales_tax,
        freight_in,
        fixed_asset_purchase_list(),
        account_payable() );

/* Returns true transaction_date_time */
char *purchase_transaction_refresh(
    purchase_transaction->
        transaction_amount,
    purchase_transaction->
        journal_list,
    purchase_transaction->memo,
    full_name,
    street_address,
    purchase_transaction->
        transaction_date_time );

void purchase_update(
    fixed_asset_purchase_total,
    purchase_invoice_amount,
    vendor_payment_total,
    purchase_amount_due,
    purchase_transaction->
        transaction_date_time,
    full_name,
    street_address,
    purchase_date_time );

LIST *inventory_purchase_list
LIST *specific_inventory_purchase_list
LIST *service_purchase_list
LIST *supply_purchase_list
LIST *prepaid_asset_purchase_list;
char *property_street_address;
```



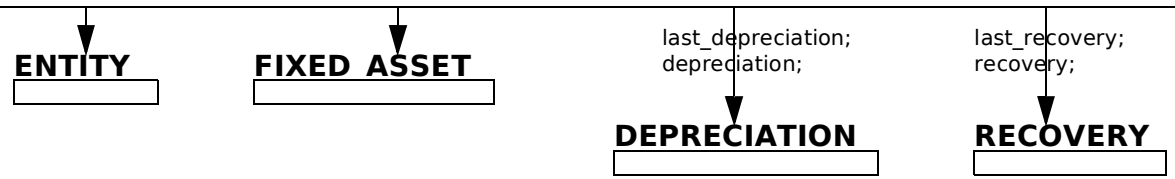
VENDOR_PAYMENT

```
/* Usage */
VENDOR_PAYMENT *vendor_payment_fetch(
	char *full_name,
	char *street_address,
	char *purchase_date_time,
	char *payment_date_time );

LIST *vendor_payment_list_fetch(
	full_name,
	street_address,
	purchase_date_time );

/* Input */
ENTITY *vendor_entity;
char *purchase_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

/* Process */
VENDOR_PAYMENT *vendor_payment_new(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time );

char *vendor_payment_primary_where(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time );

char *vendor_payment_system_string(
	purchase_primary_where() );

LIST *vendor_payment_system_list(
	vendor_payment_system_string() );

VENDOR_PAYMENT *vendor_payment_parse(
	input );

double vendor_payment_total(
	LIST *vendor_payment_system_list() )
{
	return sum(
			A->
			vendor_payment->
			payment_amount );
}

TRANSACTION *vendor_payment_transaction(
	full_name,
	street_address,
	payment_date_time,
	payment_amount,
	check_number,
	account_payable(),
	account_cash(),
	account_uncleared_checks() );

LIST *vendor_payment_journal_list(
	payment_amount,
	check_number,
	account_payable(),
	account_cash(),
	account_uncleared_checks() );

void vendor_payment_insert(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time,
	payment_amount,
	check_number );

VENDOR_PAYMENT *vendor_payment_seek(
	vendor_payment_list,
	payment_date_time );

char *vendor_payment_update_system_string(
	void );
```



FIXED_ASSET_PURCHASE (1)

```
/* Usage */
LIST *fixed_asset_purchase_list_fetch(
      char *fixed_asset_purchase_depreciation_where(),
      boolean fetch_last_depreciation,
      boolean fetch_last_recovery );

/* Process */
char *fixed_asset_purchase_system_string(
      FIXED_ASSET_PURCHASE_TABLE,
      fixed_asset_purchase_depreciation_where,
      “service_placement_date” /* order */ );

LIST *fixed_asset_purchase_system_list(
      fixed_asset_purchase_system_string(),
      fetch_last_depreciation,
      fetch_last_recovery );

/* Usage */
LIST *fixed_asset_purchase_system_list(
      char *fixed_asset_purchase_system_string(),
      boolean fetch_last_depreciation,
      boolean fetch_last_recovery );

/* Process */
FILE *fixed_asset_purchase_input_pipe(
      char *fixed_asset_purchase_system_string() );

while (string_input(
            input[],
            fixed_asset_purchase_input_pipe() )
{
      list_set(
            list,
            fixed_asset_purchase_parse(
                  input,
                  fetch_last_depreciation,
                  fetch_last_recovery ) );
}

pclose( fixed_asset_purchase_input_pipe() );

/* Usage */
FIXED_ASSET_PURCHASE *
      fixed_asset_purchase_fetch(
            char *asset_name,
            char *serial_label,
            boolean fetch_last_depreciation,
            boolean fetch_last_recovery );

/* Process */
char *fixed_asset_purchase_primary_where(
      asset_name,
      serial_label );

char *fixed_asset_purchase_system_string(
      FIXED_ASSET_PURCHASE_TABLE,
      fixed_asset_purchase_primary_where(),
      (char *)0 /* order */ );

char *string_pipe_fetch(
      fixed_asset_purchase_system_string() );

FIXED_ASSET_PURCHASE *
      fixed_asset_purchase_parse(
            string_pipe_fetch() /* input */,
            fetch_last_depreciation,
            fetch_last_recovery );

/* Usage */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_parse(
      char *input,
      boolean fetch_last_depreciation,
      boolean fetch_last_recovery );

/* Process */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_new(
      asset_name,
      serial_number );

enum depreciation_method depreciation_method =
      depreciation_method_resolve();

if ( fetch_last_depreciation )
{
      last_deprecation =
            depreciation_fetch(
                  asset_name,
                  serial_label,
                  depreciation_prior_depreciation_date()
                        /* depreciation_date */ );
}

if ( fetch_last_recovery )
{
      last_recovery =
            recovery_fetch(
                  asset_name,
                  serial_label,
                  recovery_prior_tax_year()
                        /* tax_year */ );
}

/* Attributes */
char *asset_name;
char *serial_label;
ENTITY *vendor_entity;
char *purchase_date_time;
char *service_placement_date;
double fixed_asset_cost;
int units_produced_so_far;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
enum depreciation_method depreciation_method;
int estimated_useful_life_years;
int estimated_useful_life_units;
int estimated_residual_value;
int declining_balance_n;
double cost_basis;
double finance_accumulated_depreciation;
double tax_adjusted_basis;
FIXED_ASSET *fixed_asset;
DEPRECIATION *last_depreciation;
DEPRECIATION *depreciation;
RECOVERY *last_recovery;
RECOVERY *recovery;
TRANSACTION *depreciation_transaction;
```



FIXED_ASSET_PURCHASE (2)

```
/* Usage */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_new(
      char *asset_name,
      char *serial_number );

/* Process */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_calloc(
   void );

FIXED_ASSET *fixed_asset =
      fixed_asset_fetch(
            asset_name );

/* Usage */
char *fixed_asset_purchase_primary_where(
   char *asset_name,
   char *serial_label );

/* Process */
char *fixed_asset_name_escape( asset_name );

/* Usage */
/* Returns fixed_asset_purchase_list_fetch() */
LIST *fixed_asset_purchase_depreciation_list(
   LIST *fixed_asset_purchase_list_fetch(),
   char *fixed_asset_purchase_depreciation_date() );

/* Process */
ENTITY_SELF *entity_self = entity_self_fetch();

char *account_depreciation_expense(
   ACCOUNT_DEPRECIATION_KEY );

char *account_accumulated_depreciation(
   ACCOUNT_ACCUMULATED_DEPRECIATION_KEY );
```

```
for FIXED_ASSET_PURCHASE *fixed_asset_purchase in
   fixed_asset_purchase_list_fetch()
{
   char *prior_depreciation_date =
         fixed_asset_purchase_prior_depreciation_date(
               DEPRECIATION *
                     fixed_asset_purchase->
                     last_depreciation );

   fixed_asset_purchase->depreciation =
         DEPRECIATION *depreciation_evaluate(
               asset_name,
               serial_label,
               depreciation_method_evaluate(),
               service_placement_date,
               prior_depreciation_date,
               cost_basis,
               units_produced_so_far,
               estimated_residual_value,
               estimated_useful_life_years,
               estimated_useful_life_units,
               declining_balance_n,
               finance_accumulated_depreciation
                     /* prior_accumulated_depreciation */ );

   if (  fixed_asset_purchase->
               depreciation
   &&  fixed_asset_purchase->
               depreciation->
               depreciation_amount > 0.0 )
   {
         TRANSACTION *depreciation_transaction(
               entity_self->entity->full_name,
               entity_self->entity->street_address,
               fixed_asset_purchase_depreciation_date(),
               depreciation->depreciation_amount,
               account_depreciation_expense(),
               account_accumulated_depreciation() );
   }
}

/* Usage */
void fixed_asset_purchase_list_update(
   LIST *fixed_asset_purchase_list );

/* Process */
for FIXED_ASSET_PURCHASE *fixed_asset_purchase in
   fixed_asset_purchase_list
{
   void fixed_asset_purchase_update(
         fixed_asset_purchase );
}

/* Usage */
void fixed_asset_purchase_update(
   FIXED_ASSET_PURCHASE *fixed_asset_purchase );

/* Process */
FILE *fixed_asset_purchase_update_pipe(
   char *FIXED_ASSET_PURCHASE_TABLE,
   char *FIXED_ASSET_PURCHASE_PRIMARY_KEY );

void fixed_asset_purchase_update_execute(
   FILE *fixed_asset_purchase_update_pipe(),
   double fixed_asset_purchase->cost_basis,
   double fixed_asset_purchase->
         finance_accumulated_depreciation,
   double fixed_asset_purchase->tax_adjusted_basis,
   char *fixed_asset_purchase->asset_name,
   char *fixed_asset_purchase->serial_label );

void pclose( fixed_asset_purchase_update_pipe() );
```

TRANSACTION

FIXED_ASSET_PURCHASE (3)

```
/* Usage */
LIST *fixed_asset_purchase_transaction_list_extract(
   LIST *fixed_asset_purchase_depreciation_list() );

/* Process */
for FIXED_ASSET_PURCHASE *fixed_asset_purchase in
   fixed_asset_purchase_depreciation_list
{
   if ( fixed_asset_purchase->depreciation_transaction )
   {
         list_set(
               transaction_list,
               fixed_asset_purchase->
                     depreciation_transaction );
   }
}

/* Usage */
LIST *fixed_asset_purchase_depreciation_list_extract(
   LIST *fixed_asset_purchase__depreciation_list );

/* Process */
for FIXED_ASSET_PURCHASE *fixed_asset_purchase in
   fixed_asset_purchase_depreciation_list
{
   if (   fixed_asset_purchase->depreciation
   &&  fixed_asset_purchase->
               depreciation_transaction )
   {
         fixed_asset_purchase->
               depreciation->
               transaction_date_time =
                     fixed_asset_purchase->
                           depreciation_transaction->
                           transaction_date_time;

         list_set(
               depreciation_list,
               fixed_asset_purchase->depreciation );
   }
}

/* Usage */
void fixed_asset_purchase_transaction_list_insert(
   /* May reset depreciation_transaction->
         transaction_date_time */
   LIST *
         fixed_asset_purchase_transaction_list_extract() );

/* Process */
void transaction_list_insert(
   fixed_asset_purchase_transaction_list_extract,
   1 /* insert_journal_list_boolean  */ );

/* Usage */
void fixed_asset_purchase_depreciation_list_insert(
   LIST *
   fixed_asset_purchase_depreciation_list_extract() );

/* Process */
void depreciation_list_insert(
   fixed_asset_purchase_depreciation_list_extract );

/* Driver */
void fixed_asset_purchase_depreciation_display(
   LIST *fixed_asset_purchase_depreciation_list() );

/* Driver */
void fixed_asset_purchase_depreciation_insert(
   LIST *fixed_asset_purchase_depreciation_list() );

/* Process */
LIST *fixed_asset_purchase_transaction_list_extract(
   LIST *fixed_asset_purchase_depreciation_list );

/* May reset transaction->transaction_date_time */
void fixed_asset_purchase_transaction_list_insert(
   fixed_asset_purchase_transaction_list_extract() );

LIST *fixed_asset_purchase_depreciation_list_extract(
   LIST *fixed_asset_purchase_depreciation_list );

void fixed_asset_purchase_depreciation_list_insert(
   fixed_asset_purchase_depreciation_list_extract() );

void transaction_list_html_display(
   fixed_asset_purchase_transaction_list_extract() );

/* Public */
char *fixed_asset_purchase_depreciation_where(
   char *fixed_asset_purchase_depreciation_date() );

char *fixed_asset_purchase_depreciation_date(
   void );

double fixed_asset_purchase_cost_basis(
   double fixed_asset_cost );

double fixed_asset_purchase_tax_adjusted_basis(
   double fixed_asset_cost );

char *fixed_asset_purchase_system_string(
   char *FIXED_ASSET_PURCHASE_TABLE,
   char *where,
   char *order );

double tax_accumulated_depreciation =
   recovery_accumulated();

void fixed_asset_purchase_finance_fetch_update(
   char *asset_name,
   char *serial_label );

RECOVERY *recovery =
   recovery_evaluate();

void fixed_asset_purchase_cost_fetch_update(
   char *asset_name,
   char *serial_label );

LIST *fixed_asset_purchase_list_cost_recover(
   LIST *fixed_asset_purchase_list,
   int tax_year );

LIST *fixed_asset_purchase_cost_recovery_list(
   LIST *fixed_asset_purchase_list );

void
fixed_asset_purchase_negate_depreciation_amount(
   LIST *fixed_asset_purchase_list );

void
fixed_asset_purchase_negate_recovery_amount(
   LIST *fixed_asset_purchase_list );
```

FIXED_ASSET

```
/* Usage */
FIXED_ASSET *fixed_asset_fetch(
      char *asset_name );

/* Process */
char *fixed_asset_primary_where(
      char *asset_name );

char *fixed_asset_name_escape(
      char *fixed_asset );

char *fixed_asset_system_string(
      char *fixed_asset_primary_where() );

FIXED_ASSET *fixed_asset_parse(
      char *input );

FIXED_ASSET *fixed_asset_new(
      char *asset_name );

/* Attributes */
char *asset_name;
char *account_name;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
double activity_energy_kilowatt_draw;
double activity_depreciation_per_hour;
```

RECOVERY

```
/* Usage */
RECOVERY *recovery_evaluate(
    char *asset_name,
    char *serial_label,
    int tax_year,
    double fixed_asset_purchase->
        cost_basis,
    char *fixed_asset_purchase->
        service_placement_date,
    char *fixed_asset_purchase->
        disposal_date,
    char *fixed_asset_purchase->
        cost_recovery_period_string,
    char *fixed_asset_purchase->
        cost_recovery_method,
    char *fixed_asset_purchase->
        cost_recovery_conversion );

/* Process */

/* Usage */
RECOVERY *recovery_fetch(
    char *asset_name,
    char *serial_label,
    int tax_year );

/* Process */

/* Usage */
LIST *recovery_list_fetch(
    char *asset_name,
    char *serial_label );

/* Process */

/* Usage */
void recovery_fixed_asset_list_set(
    LIST *fixed_asset_list,
    int tax_year );

/* Process */

/* Input */
char *asset_name;
char *serial_label;
int tax_year;
double cost_basis;
char *service_placement_date;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;

/* Process */
char *recovery_system_string(
    char *where );

LIST *recovery_system_list(
    recovery_system_string() );

char *recovery_primary_where(
    char *asset_name,
    char *serial_label,
    int tax_year );

double recovery_period_years(
    char *cost_recovery_period_string );

int service_placement_month =
    date_month(
        date_yyyy_mm_dd_new(
            service_placement_date ) );

int service_placement_year =
    date_year(
        date_yyyy_mm_dd_new(
            service_placement_date ) );

int disposal_month =
    date_month(
        date_yyyy_mm_dd_new(
            disposal_date ) );

int disposal_year =
    date_year(
        date_yyyy_mm_dd_new(
            disposal_date ) );

int recovery_service_placement_years(
    int tax_year,
    int service_placement_year )
{
    return A - B + 1;
}

char *recovery_subquery_where(
    int tax_year );

RECOVERY_STRAIGHT_LINE *straight_line =
    recovery_statutory_straight_line_evaluate(
        tax_year,
        cost_basis,
        service_placement_month,
        service_placement_year,
        disposal_month,
        disposal_year,
        recovery_period_years(),
        cost_recovery_conversion );

RECOVERY_ACCELERATED *accelerated =
    recovery_statutory_accelerated_evaluate(
        tax_year,
        cost_basis,
        service_placement_month,
        service_placement_year,
        disposal_month,
        disposal_year,
        recovery_period_years(),
        cost_recovery_conversion );

void recovery_fixed_asset_list_display(
    FILE *output_pipe,
    LIST *fixed_asset_list );

void recovery_list_insert(
    LIST *recovery_list );

double recovery_rate(
    straight_line,
    accelerated );

double recovery_amount(
    straight_line,
    accelerated );
```

RECOVERY_STRAIGHT_LINE **RECOVERY_ACCELERATED**

RECOVERY_STRAIGHT_LINE

```
/* Usage */
RECOVERY_STRAIGHT_LINE *
    recovery_statutory_straight_line_evaluate(
        int tax_year,
        double cost_basis,
        int service_placement_month,
        int service_placement_year,
        int disposal_month,
        int disposal_year,
        double recovery_period_years,
        char *cost_recovery_conversion );

/* Process */
RECOVERY_STRAIGHT_LINE *recovery_straight_line_calloc();

double service_placement_years =
    recovery_service_placement_years(
        tax_year,
        service_placement_year );

double straight_line_rate =
    recovery_statutory_straight_line_half_year_rate(
        int recovery_service_placement_years(),
        int tax_year,
        int disposal_year,
        double recovery_period_years );

double recovery_statutory_straight_line_half_year_mid_year_rate(
    double recovery_period_years );

double recovery_statutory_straight_line_half_year_book_ends_rate(
    double recovery_statutory_straight_line_mid_year_rate() );

double straight_line_amount =
    recovery_straight_line_amount(
        double straight_line_rate,
        double cost_basis )
{
    return A X B;
}
```

RECOVERY_ACCELERATED

```
/* Usage */
RECOVERY_ACCELERATED *
    recovery_statutory_accelerated_evaluate(
        int tax_year,
        double cost_basis,
        int service_placement_month,
        int service_placement_year,
        int disposal_month,
        int disposal_year,
        double recovery_period_years,
        char *cost_recovery_conversion );

/* Process */
RECOVERY_ACCELERATED *recovery_accelerated_calloc();

double accelerated_rate =
    recovery_statutory_accelerated_half_year_rate(
        int recovery_service_placement_years(),
        int tax_year,
        int disposal_year,
        double recovery_period_years );

double recovery_statutory_accelerated_half_year_evaluate(
    int service_placement_years,
    double recovery_period_years );

double accelerated_amount =
    recovery_accelerated_amount(
        double accelerated_rate,
        double cost_basis );
```

DEPRECIATION (1)

```
/* Usage */
DEPRECIATION *depreciation_evaluate(
	char *asset_name,
	char *serial_label,
	enum depreciation_method
		depreciation_method_evaluate(),
	char *service_placement_date,
	char *depreciation_prior_depreciation_date(),
	char *fixed_asset_purchase_depreciation_date()
		/* depreciation_date */,
	double cost_basis,
	int units_produced_so_far,
	int estimated_residual_value,
	int estimated_useful_life_years,
	int estimated_useful_life_units,
	int declining_balance_n,
	double fixed_asset_purchase->
		finance_accumulated_depreciation
		  /* prior_accumulated_depreciation */ );

/* Process */
DEPRECIATION *depreciation_new(
	asset_name,
	serial_label,
	depreciation_date );

int depreciation_units_produced(
	asset_name,
	serial_label,
	depreciation_method,
	units_produced_so_far );

double depreciation_amount(
	depreciation_method,
	service_placement_date,
	depreciation_prior_depreciation_date,
	fixed_asset_purchase_depreciation_date,
	cost_basis,
	depreciation_units_produced(),
	estimated_residual_value,
	estimated_useful_life_years,
	estimated_useful_life_units,
	declining_balance_n,
	prior_accumulated_depreciation );

if ( !depreciation_amount() )
{
	return (DEPRECIATION *)0;
}

double finance_accumulated_depreciation =
   double depreciation_accumulated(
		double prior_accumulated_depreciation,
		double depreciation_amount() );

static ENTITY_SELF *entity_self =
	entity_self_fetch();

/* Usage */
DEPRECIATION *depreciation_parse(
	char *input );

/* Process */
DEPRECIATION *depreciation_new(
	asset_name,
	serial_label,
	depreciation_date );

if ( *full_name[] )
{
	ENTITY_SELF *entity_self =
		ENTITY_SELF *entity_self_new(
			strdup( full_name ),
			strdup( street_address ) );
}

/* Attributes */
char *asset_name;
char *serial_label;
char *depreciation_date;
enum depreciation_method depreciation_method;
int units_produced;
double amount;
double finance_accumulated_depreciation;
ENTITY_SELF *entity_self;
char *transaction_date_time;

/* Usage */
DEPRECIATION *depreciation_new(
	char *asset_name,
	char *serial_label,
	char *depreciation_date );

/* Process */
DEPRECIATION *depreciation_calloc( void );
```

DEPRECIATION (2)

```
/* Usage */
double depreciation_amount(
    enum depreciation_method depreciation_method,
    char *service_placement_date,
    depreciation_prior_depreciation_date(),
    fixed_asset_purchase_depreciation_date(),
    double cost_basis,
    int depreciation_units_produced(),
    int estimated_residual_value,
    int estimated_useful_life_years,
    int estimated_useful_life_units,
    int declining_balance_n,
    double prior_accumulated_depreciation );

/* Process */
if ( depreciation_method == not_depreciated )
{
    double amount = 0.0;
}
else
if ( depreciation_method == straight_line )
{
    double amount =
        double depreciation_straight_line(
            char *service_placement_date,
            char *prior_depreciation_date,
            char *depreciation_date,
            double cost_basis,
            double estimated_residual_value,
            int estimated_useful_life_years,
            double prior_accumulated_depreciation );
}
else
if ( depreciation_method == sum_of_years_digits )
{
    double amount =
        double depreciation_sum_of_years_digits(
            char *service_placement_date,
            char *prior_depreciation_date,
            char *depreciation_date,
            double cost_basis,
            double estimated_residual_value,
            int estimated_useful_life_years,
            double prior_accumulated_depreciation );
}
else
if ( depreciation_method == double_declining_balance )
{
    double amount =
        depreciation_double_declining_balance(
            char *service_placement_date,
            char *prior_depreciation_date,
            char *depreciation_date,
            double cost_basis,
            double estimated_residual_value,
            int estimated_useful_life_years,
            double prior_accumulated_depreciation );
}
else
if ( depreciation_method == n_declining_balance )
{
    double amount =
        double depreciation_n_declining_balance(
            char *service_placement_date,
            char *prior_depreciation_date,
            char *depreciation_date,
            double cost_basis,
            double estimated_residual_value,
            int estimated_useful_life_years,
            int declining_balance_n,
            double prior_accumulated_depreciation );
}
else
if ( depreciation_method == units_of_production )
{
    double amount =
        double depreciation_units_of_production(
            double cost_basis,
            double estimated_residual_value,
            int estimated_useful_life_units,
            int units_produced,
            double prior_accumulated_depreciation );
}

if ( amount < 0.0 ) amount = 0.0;
```

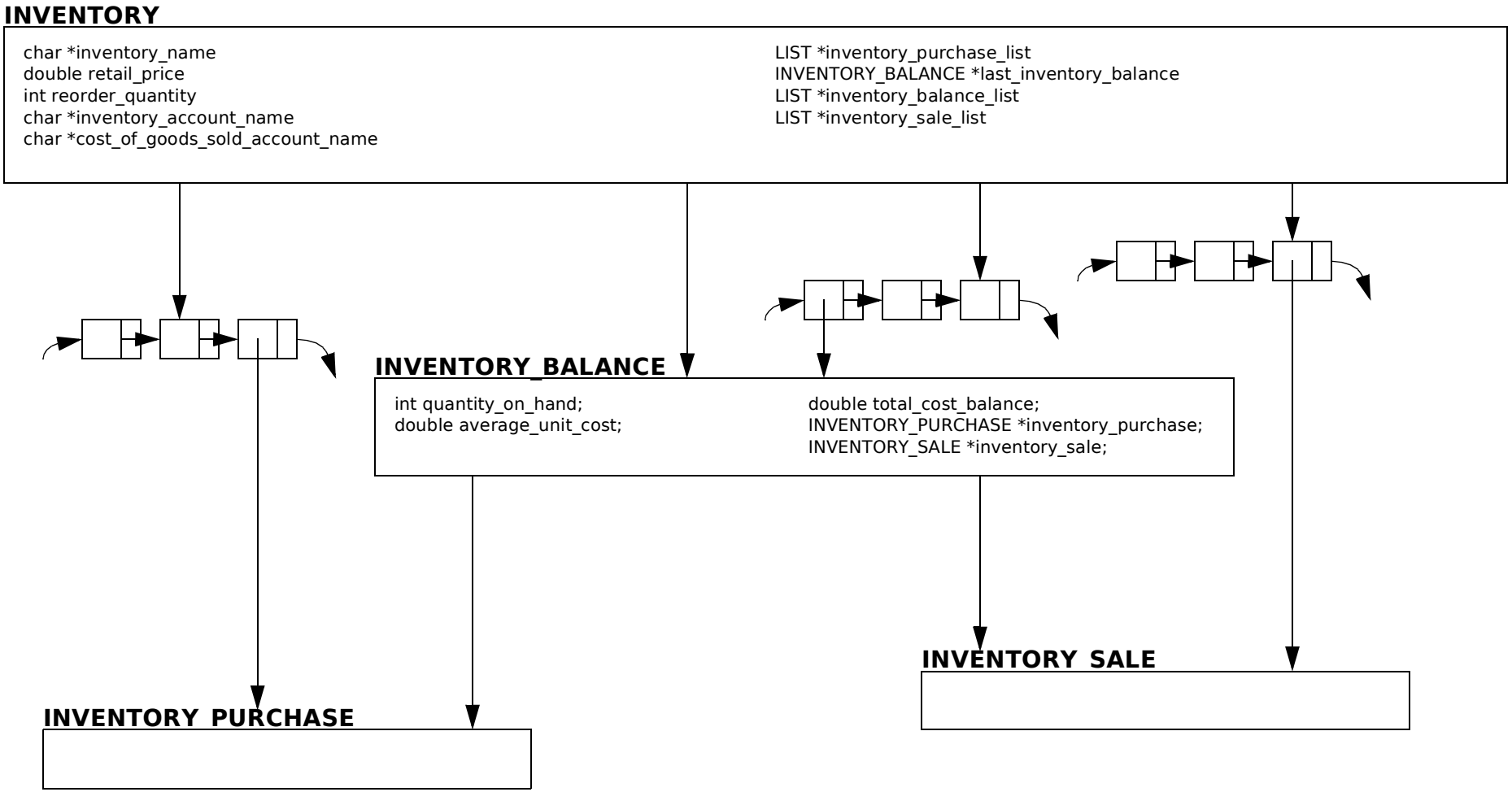
DEPRECIATION (3)

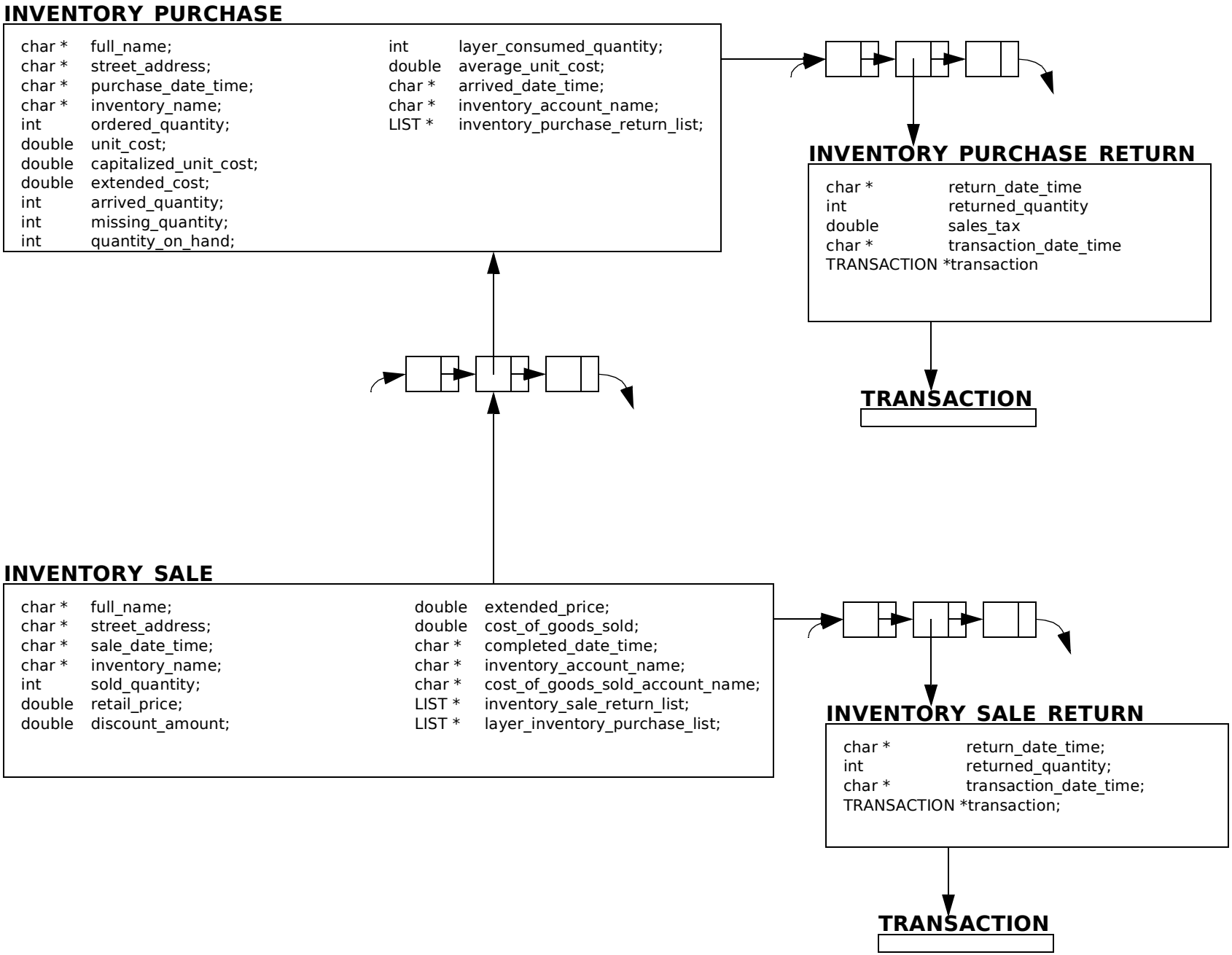
```
/* Usage */
int depreciation_units_produced(
	char *asset_name,
	char *serial_label,
	enum depreciation_method depreciation_method,
	int units_produced_so_far );

/* Process */
if ( depreciation_method != units_of_production )
{
	return 0;
}

int depreciation_units_produced_total(
	char *DEPRECIATION_TABLE,
	char *asset_name,
	char *serial_label );

int depreciation_units_produced_current(
	int units_produced_so_far,
	int depreciation_units_produced_total() );

return depreciation_units_produced_current();

/* Usage */
DEPRECIATION *depreciation_fetch(
	char *asset_name,
	char *serial_label,
	char *depreciation_date );

/* Process */
char *depreciation_primary_where(
	char *asset_name,
	char *serial_label,
	char *depreciation_date );

char *depreciation_system_string(
	char *DEPRECIATION_TABLE,
	char *where );

char *string_pipe_fetch(
	depreciation_system_string() );

DEPRECIATION *depreciation_parse(
	string_pipe_fetch()
		/* input */ );

/* Usage */
void depreciation_list_insert(
	LIST *fixed_asset_purchase_depreciation_list()
		/* depreciation_list */ );

/* Process */
FILE *depreciation_insert_pipe_open(
	char *DEPRECIATION_TABLE,
	char *DEPRECIATION_INSERT_COLUMNS );

for DEPRECIATION *depreciation in depreciation_list
{
	void depreciation_insert_pipe(
		char *depreciation->asset_name,
		char *depreciation->serial_label,
		char *depreciation->depreciation_date,
		int depreciation->units_produced,
		double depreciation->amount,
		char *depreciation->
			entity_self->
			entity->
			full_name,
		char *depreciation->
			entity_self->
			entity->
			street_address,
		char *transaction_date_time,
		FILE *depreciation_insert_pipe_open() );
}

void pclose( depreciation_insert_pipe_open() );

/* Usage */
void depreciation_update(
	int units_produced,
	double depreciation_amount,
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	char *asset_name,
	char *serial_label,
	char *depreciation_date );

/* Process */
FILE *depreciation_update_pipe_open(
	char *DEPRECIATION_TABLE,
	char *DEPRECIATION_PRIMARY_KEY );

void depreciation_update_pipe(
	int units_produced,
	double depreciation_amount,
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	char *fixed_asset_name_escape(
		asset_name ),
	char *serial_label,
	char *depreciation_date,
	FILE *depreciation_update_pipe_open() );

void pclose( depreciation_update_pipe_open() );
```

DEPRECIATION (4)

```
/* Usage */
LIST *depreciation_list_fetch(
    char *DEPRECIATION_TABLE,
    char *asset_name,
    char *serial_label );

/* Process */
char *fixed_asset_purchase_primary_where(
    asset_name,
    serial_label );

char *depreciation_system_string(
    DEPRECIATION_TABLE,
    fixed_asset_purchase_primary_where() );

LIST *depreciation_system_list(
    depreciation_system_string() );

/* Usage */
LIST *depreciation_system_list(
    char *depreciation_system_string() )

/* Process */
FILE *depreciation_input_pipe_open(
    char *depreciation_system_string );

for input[] in
    string_input(
        depreciation_input_pipe_open() )
{
    list_set(
        list,
        depreciation_parse( input ) );
}

void pclose( depreciation_input_pipe_open() );

/* Usage */
TRANSACTION *depreciation_transaction(
    char *entity_self->entity->full_name,
    char *entity_self->entity->street_address,
    char *fixed_asset_purchase_depreciation_date(),
    double depreciation_amount(),
    char *account_depreciation_expense(),
    char *account_accumulated_depreciation() );

/* Process */
/* Increments second each invocation. */
char *transaction_increment_date_time(
    fixed_asset_purchase_depreciation_date );

/* Public */
enum depreciation_method
    depreciation_method_evaluate(
        depreciation_method_string );

char *depreciation_method_string(
    enum depreciation_method
        depreciation_method );

double depreciation_fraction_of_year(
    char *service_placement_date,
    char *prior_depreciation_date,
    char *depreciation_date_string );

char *depreciation_prior_depreciation_date(
    char *DEPRECIATION_TABLE,
    char *asset_name,
    char *serial_label,
    char *depreciation_date );

char *depreciation_subquery_exists_where(
    char *DEPRECIATION_TABLE,
    char *FIXED_ASSET_PURCHASE_TABLE,
    char *depreciation_date );

void depreciation_delete_open(
    char *DEPRECIATION_PRIMARY_KEY,
    char *DEPRECIATION_TABLE );
```

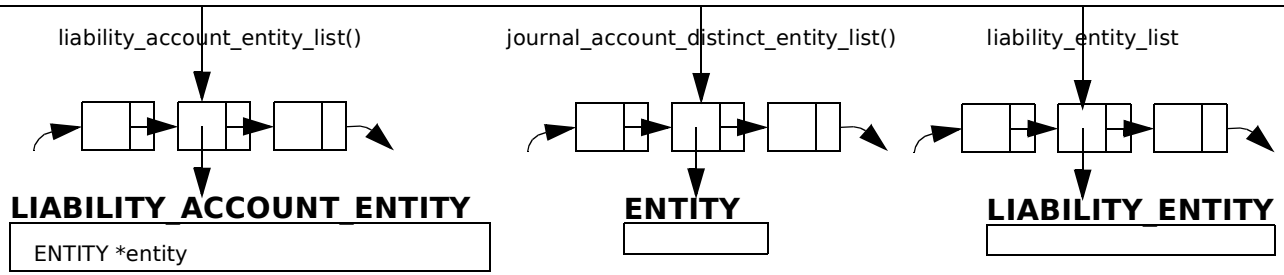
```
enum depreciation_method
{
straight_line,                  sum_of_years_digits,
double_declining_balance,       units_of_production,
n_declining_balance,            not_depreciated
```


INVENTORY
char *inventory_name
double retail_price
int reorder_quantity
char *inventory_account_name
char *cost_of_goods_sold_account_name
LIST *inventory_purchase_list
INVENTORY_BALANCE *last_inventory_balance
LIST *inventory_balance_list
LIST *inventory_sale_list
INVENTORY_BALANCE
int quantity_on_hand;
double average_unit_cost;
double total_cost_balance;
INVENTORY_PURCHASE *inventory_purchase;
INVENTORY_SALE *inventory_sale;
INVENTORY_SALE
INVENTORY_PURCHASE


INVENTORY_PURCHASE
char * full_name;
char * street_address;
char * purchase_date_time;
char * inventory_name;
int ordered_quantity;
double unit_cost;
double capitalized_unit_cost;
double extended_cost;
int arrived_quantity;
int missing_quantity;
int quantity_on_hand;
int layer_consumed_quantity;
double average_unit_cost;
char * arrived_date_time;
char * inventory_account_name;
LIST * inventory_purchase_return_list;
INVENTORY_PURCHASE_RETURN
char * return_date_time
int returned_quantity
double sales_tax
char * transaction_date_time
TRANSACTION *transaction
TRANSACTION
INVENTORY_SALE
char * full_name;
char * street_address;
char * sale_date_time;
char * inventory_name;
int sold_quantity;
double retail_price;
double discount_amount;
double extended_price;
double cost_of_goods_sold;
char * completed_date_time;
char * inventory_account_name;
char * cost_of_goods_sold_account_name;
LIST * inventory_sale_return_list;
LIST * layer_inventory_purchase_list;
INVENTORY_SALE_RETURN
char * return_date_time;
int returned_quantity;
char * transaction_date_time;
TRANSACTION *transaction;
TRANSACTION

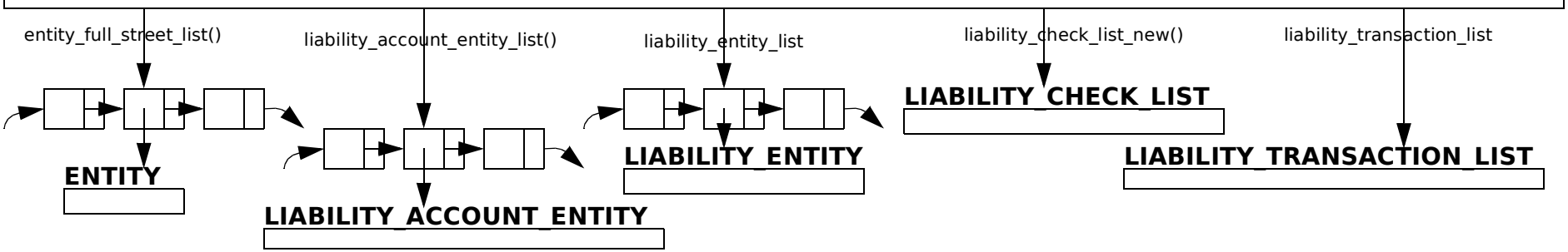
LIABILITY_CALCULATE

```
/* Usage */
LIABILITY_CALCULATE *liability_calculate_new(
   char *application_name );

/* Process */
LIABILITY_CALCULATE *liability_calculate_calloc( void );

LIST *liability_account_entity_list();

LIST *account_current_liability_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_CURRENT_LIABILITY,
   ACCOUNT_UNCLEARED_CHECKS,
   ACCOUNT_CREDIT_CARD_KEY );

LIST *journal_account_distinct_entity_list(
   JOURNAL_TABLE,
   account_current_liability_name_list()
         /* account_name_list */ );

if ( !list_rewind( journal_account_distinct_entity_list() )
{
   return (LIABILITY_CALCULATE *)0;
}

LIST *account_receivable_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_RECEIVABLE );

LIST *liability_entity_list = list_new();

ENTITY_SELF *entity_self =
   ENTITY_SELF *entity_self_fetch();

for ENTITY *entity in journal_account_distinct_entity_list
{
   if (   strcmp(
                entity->full_name,
                entity_self->entity->full_name ) == 0
   &&  strcmp(
                entity->street_address,
                entity_self->entity->street_address ) == 0 )
   {
         continue;
   }

   list_set(
         liability_entity_list,
         liability_entity_new(
                0.0 /* dialog_box_payment_amount */,
                liability_account_entity_list(),
                account_current_liability_name_list(),
                account_receivable_name_list(),
                entity ) );
}

if ( !list_length( liability_entity_list ) )
{
   return (LIABILITY_CALCULATE *)0;
}

/* Driver */
void liability_calculate_stdout(
   LIST *liability_entity_list );

/* Process */
for LIABILITY_ENTITY *liability_entity in liability_entity_list
{
   if ( liability_entity->amount_due > 0.0 )
   {
         printf(    "%s\n",
                    liability_entity_display(
                          liability_entity );
   }
}
```



LIABILITY_PAYMENT (1)

```
/* Usage */
LIABILITY_PAYMENT *liability_payment_new(
   char *application_name,
   double dialog_box_payment_amount,
   int starting_check_number,
   char *dialog_box_memo,
   char *document_root_directory,
   char *process_name,
   char *session_key,
   LIST *entity_full_street_list(
         full_name_list,
         street_address_list ) );

/* Process */
LIABILITY_PAYMENT *liability_payment_calloc( void );

if ( !list_length( entity_full_street_list() )
{
   char *liability_payment_error_message();
   return this;
}

char *transaction_memo( dialog_box_memo );

/* For dialog_box_payment_amount or memo
only one entity at a time. */

if ( list_length( entity_full_street_list() > 1
&& (    dialogy_box_payment_amount
||      *transaction_memo() ) )
{
   char *liability_payment_error_message();
   return this;
}

LIST *liability_account_entity_list();

LIST *account_current_liability_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_CURRENT_LIABILITY,
   ACCOUNT_UNCLEARED_CHECKS );

LIST *account_receivable_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_RECEIVABLE );

LIST *liability_entity_list = list_new();

if ( list_length( entity_full_street_list() == 1 )
{
   list_set(
         liability_entity_list,
         liability_entity_new(
               dialog_box_payment_amount,
               liability_account_entity_list(),
               account_current_liability_name_list(),
               account_receivable_name_list(),
               list_first( entity_full_street_list() ) ) );
}
else
{
   for ENTITY *entity in entity_full_street_list
   {
         list_set(
               liability_entity_list,
               liability_entity_new(
                     dialog_box_payment_amount,
                     liability_account_entity_list(),
                     account_current_liability_name_list(),
                     account_receivable_name_list(),
                     entity ) );
   }
}

if ( !list_length( liability_entity_list ) )
{
   char *liability_payment_error_message();
   return this;
}

if ( starting_check_number )
{
   LIABILITY_CHECK_LIST *liability_check_list =
         liability_check_list_new(
               application_name,
               starting_check_number,
               transaction_memo(),
               document_root_directory,
               process_name,
               session_key,
               liability_entity_list );
}

char *liability_payment_credit_account_name(
   int starting_check_number,
   char *account_cash(),
   char *account_uncleared_checks() );

LIABILITY_TRANSACTION_LIST *liability_transaction_list =
   liability_transaction_list_new(
         dialog_box_payment_amount,
         starting_check_number,
         transaction_memo(),
         liability_entity_list,
         liability_payment_credit_account_name() );

if ( !liability_transaction_list )
{
   char *liability_payment_error_message();
   return this;
}
```



LIABILITY_PAYMENT (2)

```
/* Private */
char *liability_payment_error_message(
   char *message );

/* Driver */
if ( liability_check_list )
{
      appaserver_library_output_ftp_prompt(
            liability_check_list->
                  liability_check_appaserver_link->
                  ftp_filename,
            LIABILITY_PROMPT,
            "check_page" /* target_frame */,
            (char *)0 /* application_type */ );
}

if ( execute )
{
      /* May reset transaction_date_time */
      void liability_transaction_list_insert(
            liability_transaction_list );
}

void liability_transaction_list_html_display(
      liability_transaction_list );
```

LIABILITY_ACCOUNT_ENTITY

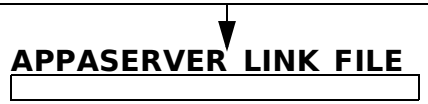
```
/* Usage */
LIST *liability_account_entity_list( void );

/* Process */
char *liability_account_entity_system_string(
        char *LIABILITY_ACCOUNT_ENTITY_SELECT,
        char *LIABILITY_ACCOUNT_ENTITY_TABLE );

LIST *liability_account_entity_system_list(
        char *liability_account_entity_system_string() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_parse(
                char *string_input() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_new(
                char *strdup( account_name ) );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_calloc(
                void );

ENTITY *entity =
        entity_new(
                strdup( full_name ),
                strdup( street_address ) );

/* Attributes */
char *account_name;
ENTITY *entity;

/* Public */
LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_seek_account(
                char *account_name,
                LIST *liability_account_entity_list() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_seek_entity(
                char *full_name,
                char *street_address,
                LIST *liability_account_entity_list() );
```

LIABILITY_CHECK_LIST

```
/* Usage */
LIABILITY_CHECK_LIST *liability_check_list_new(
        char *application_name,
        int starting_check_number,
        char *transaction_memo(),
        char *document_root_directory,
        char *process_name,
        char *session_key,
        LIST *liability_entity_list() );

/* Process */
LIABILITY_CHECK_LIST *liability_check_list_calloc( void );

char *liability_check_list_documentclass( void );

char *liability_check_list_usepackage( void );

char *liability_check_list_pagenumbering_gobble( void );

char *liability_check_list_begin_document( void );

char *liability_check_list_heading(
        char *liability_check_list_documentclass(),
        char *liability_check_list_usepackage(),
        char *liability_check_list_pagenumbering_gobble(),
        char *liability_check_list_begin_document() );

LIST *list = list_new();

for LIABILITY_ENTITY *liability_entity in
        liability_enitity_list
{
        list_set(
                list,
                liability_check_new(
                        starting_check_number++
                                /* check_number */,
                        transaction_memo,
                        liability_entity ) );
}

char *liability_check_list_end_document( void );

LIABILITY_CHECK_APPASERVER_LINK *
        liability_check_appaserver_link =
                liability_check_appaserver_link_new(
                        application_name,
                        document_root_directory,
                        process_name,
                        session_key );

FILE *output_file =
        fopen(
                liability_check_appaserver_link->
                        output_filename,
                "w" );

fprintf(    output_file,
            "%s\n",
            liability_check_list_heading() );

for LIABILITY_CHECK *liability_check in list
{
        fprintf(    output_file,
                    "%s\n",
                    liability_check->output_string );
}

fprintf(    output_file,
            "%s\n",
            liability_check_list_end_document() );

fclose( output_file );

latex_tex2pdf(
        liability_check_appaserver_link->
                output_filename,
        appaserver_link_source_directory(
                document_root_directory,
                application_name )
                        /* working_directory */ );
```

LIABILITY_CHECK_APPASERVER_LINK

```
/* Usage */
LIABILITY_CHECK_APPASERVER_LINK *
    liability_check_appaserver_link_new(
    char *application_name,
    char *document_root_directory,
    char *process_name,
    char *session_key );

/* Process */
LIABILITY_CHECK_APPASERVER_LINK *
    liability_check_appaserver_link_calloc(
        void );

APPASERVER_LINK_FILE *appaserver_link_file =
    appaserver_link_file_new(
        application_http_prefix( application_name ),
        appaserver_library_get_server_address(),
        ( application_prepend_http_protocol_yn(
            application_name ) == 'y' ),
        document_root_directory,
        process_name /* filename_stem */,
        application_name,
        getpid(),
        (char *)0 /* session_key */,
         "tex" );

char *output_filename =
    appaserver_link_get_output_filename(
        appaserver_link_file->
            output_file->
            document_root_directory,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

appaserver_link_file->extension = "pdf";

char *pdf_output_filename =
    appaserver_link_get_link_prompt(
        appaserver_link_file->
            link_prompt->
            prepend_http_boolean,
        appaserver_link_file->
            link_prompt->
            http_prefix,
        appaserver_link_file->
            link_prompt->
            server_address,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

char *ftp_filename =
    appaserver_link_get_link_prompt(
        appaserver_link_file->
            link_prompt->
            prepend_http_boolean,
        appaserver_link_file->
            link_prompt->
            http_prefix,
        appaserver_link_file->
            link_prompt->
            server_address,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

appaserver_link_file->extension = "pdf";
appaserver_link_file->session = session_key;
appaserver_link_file->process_id = 0;

char *document_root_filename =
    appaserver_link_get_output_filename(
        appaserver_link_file->
            output_file->
            document_root_directory,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );
```

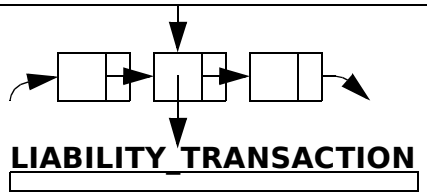


LIABILITY_CHECK

```
/* Usage */
LIABILITY_CHECK *liability_check_new(
      int check_number,
      char *transaction_memo(),
      LIABILITY_ENTITY *liability_entity );

/* Process */
LIABILITY_CHECK *liability_check_calloc( void );

char *liability_check_dollar_text(
      double liability_entity->amount_due );

char *liability_check_escape_payable_to(
      char *liability_entity->entity->full_name );

char *liability_check_move_down( void );

char *liability_check_date_display(
      char *pipe2string(
            char *LIABILITY_CHECK_DATE_COMMAND )
                  /* check_date */ );

char *liability_check_amount_due_display(
      char *place_commas_in_money(
            double liability_entity->amount_due )
                  /* amount_due_display */ );

char *liability_check_vendor_name_amount_due_display(
      char *liability_check_escape_payable_to(),
      char *liability_check_amount_due_display() );

char *liability_check_amount_due_stub_display(
      char *liability_check_amount_due_display() );

char *liability_check_dollar_text_display(
      char *liability_check_dollar_text() );

char *liability_check_memo_display(
      char *transaction_memo );

char *liability_check_number_display(
      int check_number );

char *liability_check_newpage( void );

char *liability_check_output_string(
      char *liability_check_move_down(),
      char *liability_check_date_display(),
      char *liability_check_vendor_name_amount_due_display(),
      char *liability_check_amount_due_stub_display(),
      char *liability_check_dollar_text_display(),
      char *liability_check_memo_display(),
      char *liability_check_number_display(),
      char *liability_check_newpage() );
```

LIABILITY_TRANSACTION_LIST

```
/* Usage */
LIABILITY_TRANSACTION_LIST *
   liability_transaction_list_new(
         double dialog_box_payment_amount,
         int starting_check_number,
         char *transaction_memo(),
         LIST *liability_entity_list,
         char *liability_payment_credit_account_name );

/* Process */
if ( dialog_box_payment_amount
&& list_length( liability_entity_list ) > 1 )
{
   return (LIABILITY_TRANSACTION_LIST *)0;
}

LIABILITY_TRANSACTION_LIST *
   liability_transaction_list_calloc(
         void );

LIST *list = list_new();

DATE *transaction_date_time =
   date_now_new(
         date_utc_offset() );

for LIABILITY_ENTITY *liability_entity in liability_entity_list
{
   list_set(
         list,
         liability_transaction_new(
               dialog_box_payment_amount,
               starting_check_number
                     /* check_number */,
               transaction_memo,
               liability_payment_credit_account_name,
               liability_entity,
               transaction_date_time ) );

   if ( starting_check_number )
   {
         starting_check_number++;
   }

   date_increment_seconds( transaction_date_time, 1 );
}

/* Usage */
/* May reset transaction_date_time */
void liability_transaction_list_insert(
   LIABILITY_TRANSACTION_LIST *
         liability_transaction_list,
   char *appaserver_error_filename );

/* Process */
LIST *liability_transaction_list_extract(
   liability_transaction_list->list );

char *transaction_list_insert(
   liability_transaction_list_extract(),
   1 /* insert_journal_list_boolean */ );

/* Usage */
void liability_transaction_list_html_display(
   LIABILITY_TRANSACTION_LIST *
         liability_transaction_list );

/* Process */
LIST *liability_transaction_list_extract(
   liability_transaction_list->list );

void transaction_list_html_display(
   liability_transaction_list_extract() );

/* Public */
LIST *liability_transaction_list_extract(
   LIST *liability_transaction_list->list );
```



LIABILITY_TRANSACTION

```
/* Usage */
LIABILITY_TRANSACTION *liability_transaction_new(
    double dialog_box_payment_amount,
    int check_number,
    char *transaction_memo(),
    char *liability_payment_credit_account_name,
    LIABILITY_ENTITY *liability_entity,
    DATE *transaction_date_time );

/* Process */
LIABILITY_TRANSACTION *liability_transaction_calloc(
    void );
```

```
LIABILITY_JOURNAL_LIST *liability_journal_list =
    liability_journal_list_new(
        dialog_box_payment_amount,
        liability_payment_credit_account_name,
        liability_entity );

if ( !liability_journal_list )
{
    return (LIABILITY_TRANSACTION *)0;
}
```

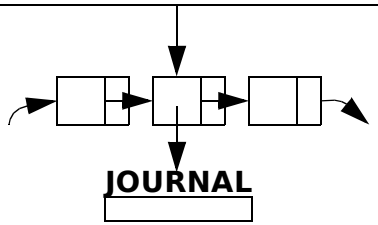
```
TRANSACTION *transaction =
    transaction_entity_new(
        liability_entity->entity,
        date_display_19( transaction_date_time ),
        liability_journal_list->transaction_amount,
        check_number,
        (*transaction_memo)
            ? transaction_memo
            : LIABILITY_TRANSACTION_MEMO,
        liability_journal_list->journal_list );
```

LIABILITY_JOURNAL_LIST

TRANSACTION

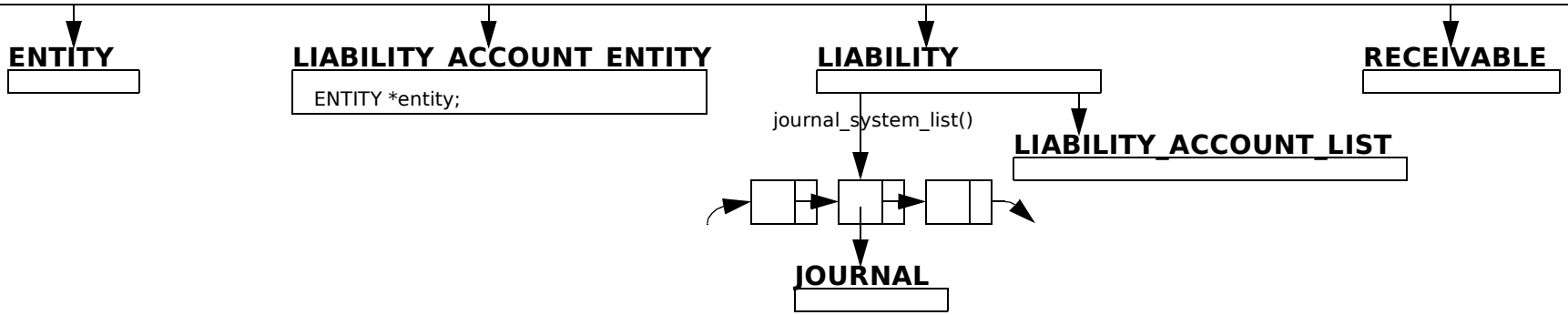
LIABILITY_JOURNAL_LIST

```
/* Usage */
LIABILITY_JOURNAL_LIST *liability_journal_list_new(
	double dialog_box_payment_amount,
	char *liability_payment_credit_account_name,
	LIABILITY_ENTITY *liability_entity );

/* Process */
LIABILITY_JOURNAL_LIST *liability_journal_list_calloc(
	void );

LIST *journal_list = list_new();

if ( dialog_box_payment_amount )
{
	liability_journal_list_transaction_amount(
		dialog_box_payment_amount );
}
else
{
	liability_journal_list_transaction_amount(
		liability_entity->amount_due );
}

list_set(
	journal_list,
	journal_credit_new(
		liability_payment_credit_account_name,
		liability_journal_list_transaction_amount() ) );

for LIABILITY_ACCOUNT *liability_account in
	liability_entity->liability->liability_account_list->list
{
	list_set(
		journal_list,
		journal_debit_new(
			liability_account->account_name,
			(dialog_box_payment_amount)
				? dialog_box_payment_amount
				: liability_account->credit_amount ) );
}

if ( liability_entity->receivable )
{
	for RECEIVABLE_ACCOUNT *receivable_account in
		liability_entity->receivable->receivable_account_list->list
	{
		list_set(
			journal_list,
			journal_credit_new(
				receivable_account->account_name,
				receivable_account->debit_amount ) );
	}
}

/* Usage */
double liability_journal_list_transaction_amount(
	double amount_due );

/* Process */
return amount_due;
```



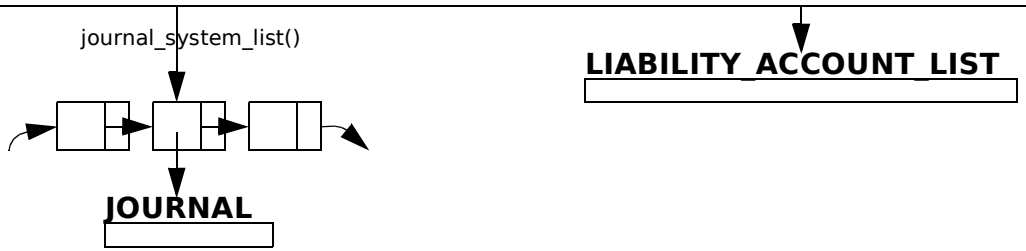
LIABILITY_ENTITY

```
/* Usage */
LIABILITY_ENTITY *liability_entity_new(
   double dialog_box_payment_amount,
   LIST *liability_account_entity_list(),
   LIST *account_current_liability_name_list(),
   LIST *account_receivable_name_list(),
   ENTITY *entity );

/* Process */
if ( !entity ) exit( 1 );

LIABILITY_ENTITY *liability_entity_calloc( void );

/* Attribute */
ENTITY *entity;

LIABILITY_ACCOUNT_ENTITY *
   liability_account_entity =
         liability_account_entity_seek_entity(
               entity->full_name,
               entity->street_address,
               liability_account_entity_list );

if ( liability_account_entity )
{
   ENTITY *entity = liability_account_entity->entity;

   LIABILITY *liability =
         liability_account_fetch(
               liability_account_entity->
                     account_name );
}
else
{
   LIABILITY *liability =
         liability_entity_fetch(
               entity->full_name,
               entity->street_address,
               account_current_liability_name_list );
}

if ( !liability )
{
   return (LIABILITY_ENTITY *)0;
}

if ( !liability_account_entity )
{
   RECEIVABLE *receivable =
         receivable_fetch(
               entity->full_name,
               entity->street_address,
               account_receivable_name_list );
}

double liability_entity_amount_due(
   double dialog_box_payment_amount,
   LIABILITY *liability,
   RECEIVABLE *receivable );

/* Driver */
char *liability_entity_display(
   LIABILITY_ENTITY *liability_entity );
```



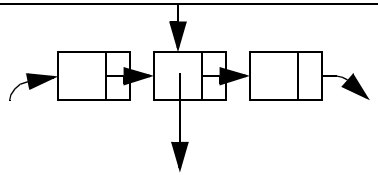
LIABILITY

```
/* Usage */
LIABILITY *liability_entity_fetch(
        char *full_name,
        char *street_address,
        LIST *account_current_liability_name_list() );

/* Process */
LIABILITY *liability_calloc( void );

char *timlib_in_clause(
        account_current_liability_name_list );

char *liability_entity_where(
        char *full_name,
        char *street_address,
        char *timlib_in_clause() );

LIST *journal_system_list(
        journal_system_string(
                JOURNAL_SELECT,
                JOURNAL_TABLE,
                liability_entity_where() ),
        0 /* not fetch_account */,
        0 /* not fetch_subclassification */,
        0 /* not fetch_element */,
        0 /* not fetch_transaction */ );

if ( !list_length( journal_system_list() )
{
        return (LIABILITY *)0;
}

double journal_credit_debit_difference_sum(
        journal_system_list() );

if ( !journal_credit_debit_difference_sum() )
{
        return (LIABILITY *)0;
}

LIABILITY_ACCOUNT_LIST *
        liability_account_list =
                liability_account_list_new(
                        journal_system_list() );

/* Usage */
LIABILITY *liability_account_fetch(
        char *liability_accout_name );

/* Process */
LIABILITY *liability_calloc();

char *liability_account_where(
        char *liability_account_name );

LIST *journal_system_list(
        journal_system_string(
                JOURNAL_SELECT,
                JOURNAL_TABLE,
                liability_account_where() ),
        0 /* not fetch_account */,
        0 /* not fetch_subclassification */,
        0 /* not fetch_element */,
        0 /* not fetch_transaction */ );

if ( !list_length( journal_system_list() )
{
        return (LIABILITY *)0;
}

double journal_credit_debit_difference_sum(
        journal_system_list() );

if ( !journal_credit_debit_difference_sum() )
{
        return (LIABILITY *)0;
}

LIABILITY_ACCOUNT_LIST *
        liability_account_list =
                liability_account_list_new(
                        journal_system_list() );
```



LIABILITY_ACCOUNT_LIST

```
/* Usage */
LIABILITY_ACCOUNT_LIST *
    liability_account_list_new(
        LIST *journal_system_list() );

/* Process */
LIABILITY_ACCOUNT_LIST *
    liability_account_list_calloc(
        void );

LIST *list = list_new();

for JOURNAL *journal in journal_system_list
{
    LIABILITY_ACCOUNT *
        liability_account =
            liability_account_getset(
                list,
                journal->account_name );

    if ( journal->credit_amount )
    {
        liability_account->credit_amount +=
            journal->credit_amount;
    }
    else
    {
        liability_account->credit_amount -=
            journal->debit_amount;
    }
}
```



LIABILITY_ACCOUNT

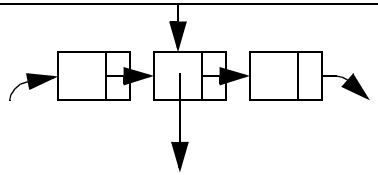
```
/* Usage */
LIABILITY_ACCOUNT *
    liability_account_getset(
        LIST *list,
        char *account_name );

/* Process */
for LIABILITY_ACCOUNT *liability_account in list
{
    if ( strcmp(
        account_name,
        liability_account->account_name ) == 0 )
    {
        return liability_account;
    }
}

LIABILITY_ACCOUNT *
    liability_account =
        liability_account_new(
            account_name );

LIABILITY_ACCOUNT *
    liability_account_calloc(
        void );

list_set(
    list,
    liability_account );

/* Attributes */
char *account_name;
double credit_amount;
```

RECEIVABLE

```
/* Usage */
RECEIVABLE *receivable_fetch(
        char *full_name,
        char *street_address,
        LIST *account_receivable_name_list() );

/* Process */
RECEIVABLE *receivable_calloc( void );

char *timlib_in_clause(
        account_receivable_name_list );

char *receivable_where(
        char *full_name,
        char *street_address,
        char *timlib_in_clause() );

LIST *journal_system_list(
        journal_system_string(
                JOURNAL_SELECT,
                JOURNAL_TABLE,
                receivable_where() ),
        0 /* not fetch_account */,
        0 /* not fetch_subclassification */,
        0 /* not fetch_element */,
        0 /* not fetch_transaction */ );

if ( !list_length( journal_system_list() )
{
        return (RECEIVABLE *)0;
}

double journal_debit_credit_difference_sum(
        journal_system_list() );

if ( !journal_debit_credit_difference_sum() )
{
        return (RECEIVABLE *)0;
}

RECEIVABLE_ACCOUNT_LIST *
        receivable_account_list_new(
                journal_system_list() );
```

RECEIVABLE_ACCOUNT_LIST

```
/* Usage */
RECEIVABLE_ACCOUNT_LIST *
    receivable_account_list_new(
        LIST *journal_system_list() );

/* Process */
if ( !list_rewind( journal_system_list ) )
{
    return (RECEIVABLE_ACCOUNT_LIST *)0;
}

RECEIVABLE_ACCOUNT_LIST *
    receivable_account_list_calloc(
        void );

LIST *list = list_new();

for JOURNAL *journal in journal_system_list
{
    RECEIVABLE_ACCOUNT *
        receivable_account =
            receivable_account_getset(
                list,
                journal->account_name );
    if ( journal->debit_amount )
    {
        receivable_account->debit_amount +=
            journal->debit_amount;
    }
    else
    {
        receivable_account->debit_amount -=
            journal->credit_amount;
    }
}
```



RECEIVABLE_ACCOUNT

```
/* Usage */
RECEIVABLE_ACCOUNT *
    receivable_account_getset(
        LIST *list,
        char *journal->account_name );

/* Process */
for RECEIVABLE_ACCOUNT *recievable_account in list
{
    if ( strcmp(
        account_name,
        receivable_account->account_name ) == 0 )
    {
        return receivable_account;
    }
}

RECEIVABLE_ACCOUNT *
    receivable_account =
        receivable_account_new(
            account_name );

RECEIVABLE_ACCOUNT *
    receivable_account_calloc(
        void );

list_set(
    list,
    receivable_account );

/* Attributes */
char *account_name;
double debit_amount;
```

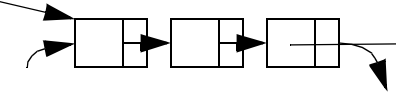
EMPLOYEE

char *full_name
char *street_address
double hourly_wage
double period_salary
double commission_sum_percent
double gross_pay_year_to_date
double net_pay_year_to_date
char * federal_marital_status
int federal_withholding_allowances
int federal_withholding_additional_period_amount
boolean state_withholding_exempt
char *state_marital_status
int state_withholding_allowances
int state_itemized_deduction_allowances
int retirement_contribution_plan_employee_period_amount
int retirement_contribution_plan_employer_period_amount
int health_insurance_employee_period_amount
int health_insurance_employer_period_amount
double union_dues_period_amount
char *terminated_date

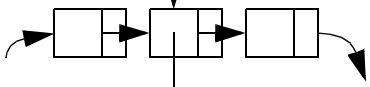
LIST *
employee_work_day_list

LIST *
employee_work_period_list

LIST *
customer_sale_list



CUSTOMER_SALE

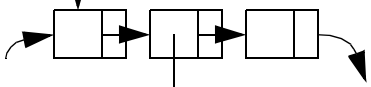


EMPLOYEE_WORK_PERIOD

char *full_name
char *street_address
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
char *transaction_date_time

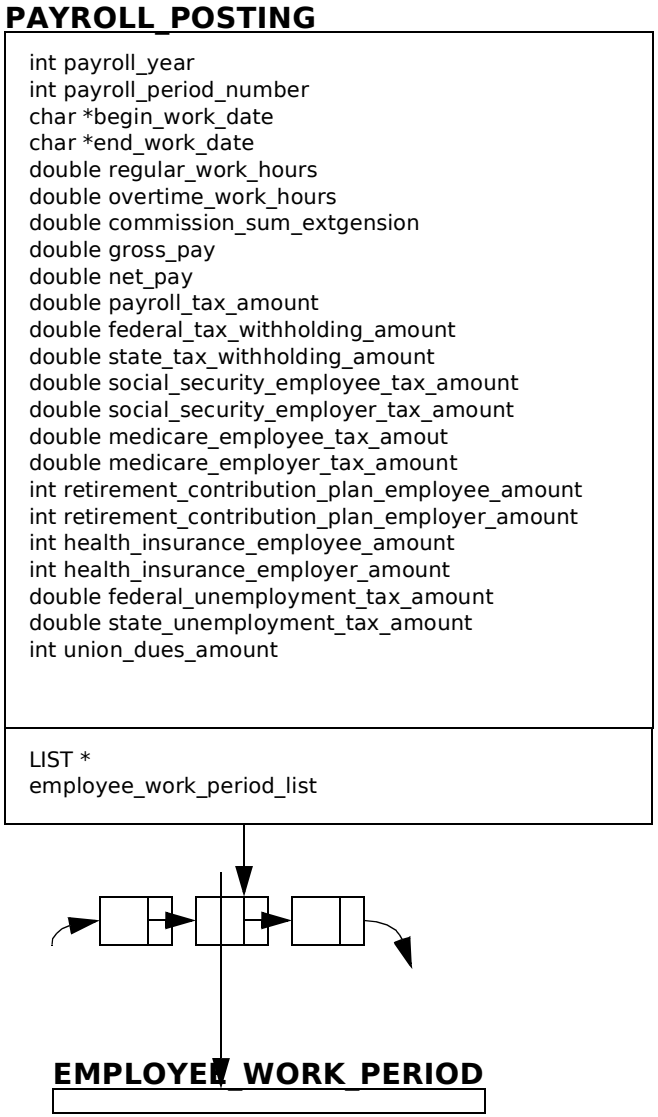
TRANSACTION *
transaction

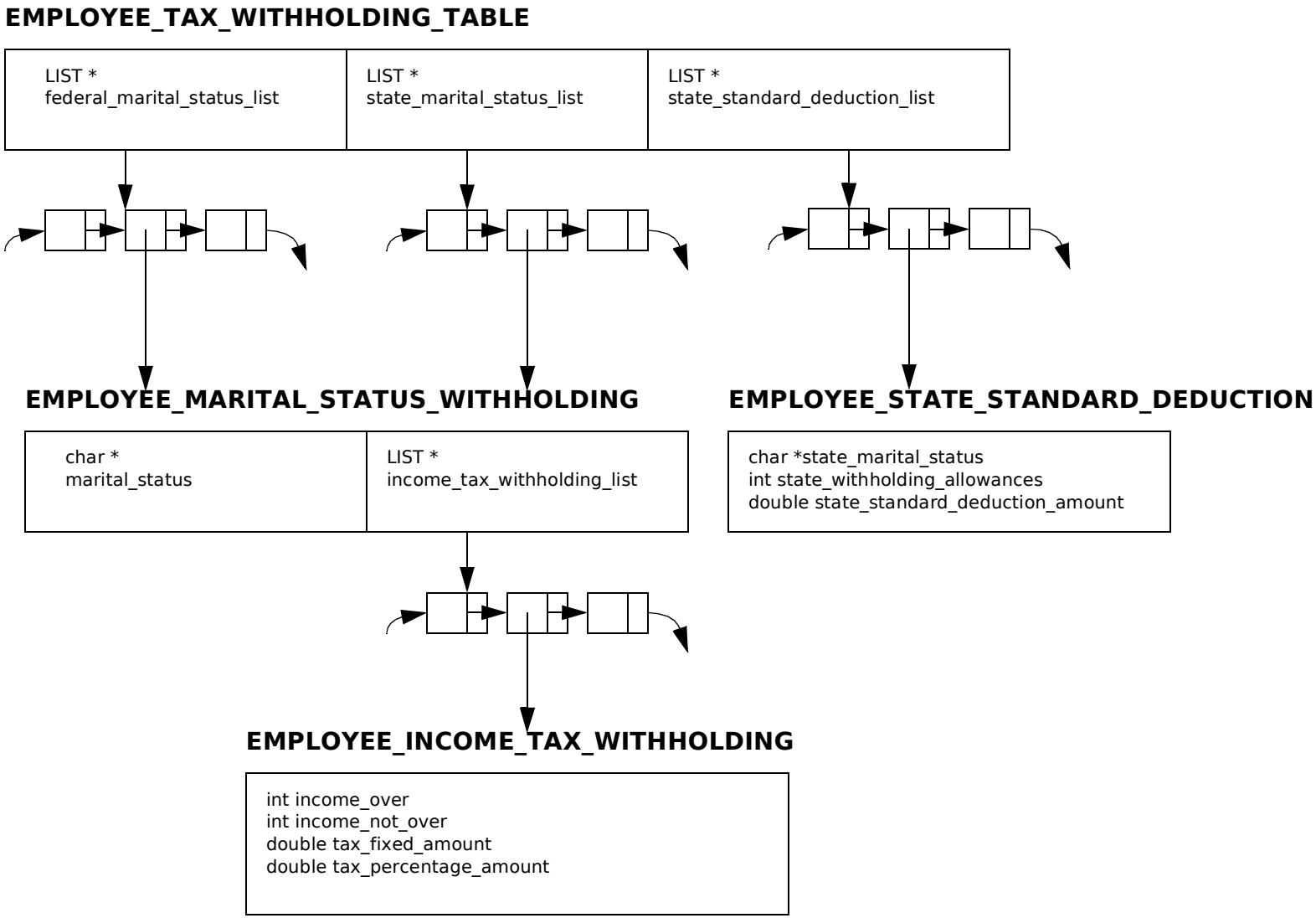
TRANSACTION



EMPLOYEE_WORK_DAY

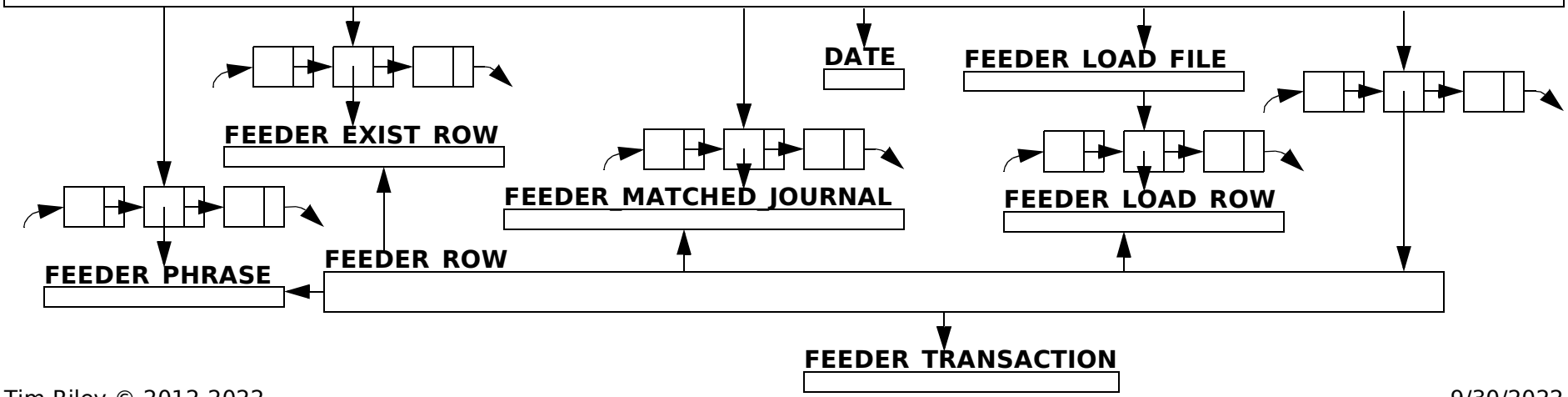
char *begin_work_date_time
char *end_work_date_time
double employee_work_hours
boolean overtime_work_day


PAYROLL_POSTING
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extgension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
LIST *
employee_work_period_list
EMPLOYEE_WORK_PERIOD


EMPLOYEE_TAX_WITHHOLDING_TABLE
LIST *
federal_marital_status_list
LIST *
state_marital_status_list
LIST *
state_standard_deduction_list
EMPLOYEE_MARITAL_STATUS_WITHHOLDING
char *
marital_status
LIST *
income_tax_withholding_list
EMPLOYEE_STATE_STANDARD_DEDUCTION
char *state_marital_status
int state_withholding_allowances
double state_standard_deduction_amount
EMPLOYEE_INCOME_TAX_WITHHOLDING
int income_over
int income_not_over
double tax_fixed_amount
double tax_percentage_amount

FEEDER (1)

```
/* Usage */
FEEDER *feeder_fetch(
   char *login_name,
   char *feeder_account,
   char *feeder_load_filename,
   int date_column /* one_based */,
   int description_column /* one_based */,
   int debit_column /* one_based */,
   int credit_column /* one_based */,
   int balance_column /* one_based */,
   int reference_column /* one_based */,
   boolean reverse_order_boolean,
   double parameter_account_end_balance );

/* Process */
FEEDER *feeder_calloc( void );

char *account_uncleared_checks(
   ACCOUNT_UNCLEARED_CHECKS_KEY );

LIST *feeder_phrase_list(
   char *FEEDER_PHRASE_SELECT,
   char *FEEDER_PHRASE_TABLE );

if ( !char *feeder_load_file_minimum_date(
         feeder_load_filename,
         date_column /* one_based */ ) )
{
   return (FEEDER *)0;
}

LIST *feeder_exist_row_list(
         feeder_account,
         feeder_load_file_minimum_date() );

LIST *feeder_matched_journal_list(
   feeder_account,
   FEEDER_ROW_TABLE,
   feeder_load_file_minimum_date(),
   account_uncleared_checks() );

FEEDER_LOAD_FILE *feeder_load_file =
   feeder_load_file_fetch(
         feeder_load_filename,
         date_column /* one_based */,
         description_column /* one_based */,
         debit_column /* one_based */,
         credit_column /* one_based */,
         balance_column /* one_based */,
         reference_column /* one_based */,
         reverse_order_boolean );

LIST *feeder_row_list(
   feeder_account,
   account_uncleared_checks(),
   feeder_phrase_list(),
   feeder_exist_row_list(),
   feeder_matched_journal_list(),
   feeder_load_file->feeder_load_row_list ) );

FEEDER_ROW *
   feeder_row_first_out_balance(
         feeder_row_list );

double feeder_account_end_balance(
   feeder_account,
   balance_column,
   parameter_account_end_balance,
   feeder_row_list(),
   feeder_row_first_out_balance() );

double feeder_prior_account_end_balance(
   feeder_account,
   feeder_load_file_minimum_date(),
   balance_column,
   feeder_account_end_balance(),
   feeder_row_list(),
   feeder_row_first_out_balance() );

void feeder_row_list_calculate_balance_set(
   feeder_row_list(),
   feeder_prior_account_end_balance(),
   feeder_row_first_out_balance() );
```



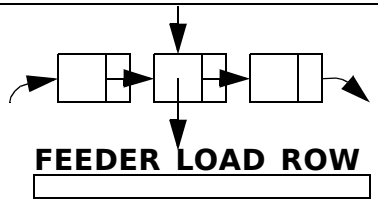
FEEDER (2)

```
/* Process (continued) */
char *feeder_account_end_date(
   LIST *feeder_row_list(),
   FEEDER_ROW *feeder_row_first_out_balance() );

if ( feeder_account_end_date() )
{
   double feeder_account_end_calculate_balance(
         LIST *feeder_row_list(),
         FEEDER_ROW *feeder_row_first_out_balance() );

   char *feeder_load_date_time =
         char *date_now16( date_utc_offset() );

   FEEDER_LOAD_EVENT *feeder_load_event =
         feeder_load_event_new(
               feeder_account,
               feeder_load_date_time,
               login_name,
               feeder_load_file_basename_filename(),
               feeder_account_end_date(),
               feeder_account_end_calculate_balance() );
}

/* Usage */
double feeder_account_end_balance(
      char *feeder_account,
      int balance_column,
      double parameter_account_end_balance,
      LIST *feeder_row_list,
      FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
FEEDER_LOAD_EVENT *
   feeder_load_event_latest_fetch(
         FEEDER_LOAD_EVENT_TABLE,
         feeder_account );

if ( feeder_load_event_latest_fetch() )
{
   double feeder_row_list_sum_amount(
         feeder_row_list,
         feeder_row_first_out_balance );

   double account_end_balance =
         feeder_load_event_latest_fetch()->
               account_end_balance +
         feeder_row_list_sum_amount();
}
else
if ( balance_column )
{
   FEEDER_ROW *last_feeder_row =
         list_last( feeder_row_list );

   double account_end_balance =
         double feeder_load_row_account_end_balance(
               (last_feeder_row)
                     ? feeder_row->feeder_load_row
                     : (FEEDER_LOAD_ROW *)0
                           /* last_feeder_load_row */ );
}
else
if ( parameter_account_end_balance )
{
      double account_end_balance =
            parameter_account_end_balance );
}

/*Usage */
double feeder_prior_account_end_balance(
   char *feeder_account,
   char *feeder_load_file_minimum_date(),
   boolean balance_column,
   double feeder_account_end_balance(),
   LIST *feeder_row_list,
   FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
FEEDER_LOAD_EVENT *
      feeder_load_event_latest_fetch(
            FEEDER_LOAD_EVENT_TABLE,
            feeder_account );

if ( feeder_load_event_latest_fetch() )
{
      prior_account_end_balance =
            feeder_load_event_latest_fetch()->
                  account_end_balance;
}
else
{
      prior_account_end_balance =
            journal_prior_account_end_balance(
                  feeder_load_file_minimum_date,
                  feeder_account /* account_name */ );
}

if ( !prior_account_end_balance && balance_column )
{
   FEEDER_ROW *first_feeder_row =
         list_first( feeder_row_list );

   double prior_account_end_balance =
         feeder_load_row_prior_account_end_balance(
               (first_feeder_row)
                     ? feeder_row->feeder_load_row
                     : (FEEDER_LOAD_ROW *)0
                           /* first_feeder_load_row */ );
}
else
if ( !prior_account_end_balance )
{
   double feeder_row_list_sum_amount(
         feeder_row_list,
         feeder_row_first_out_balance );

   prior_account_end_balance =
         feeder_account_end_balance -
         feeder_row_list_sum_amount();
}

/* Driver */
char *feeder_parameter_account_end_balance_error(
   double parameter_account_end_balance,
   FEEDER_ROW *feeder_row_first_out_balance(),
   double feeder_account_end_calculate_balance(),
   int feeder_row_seek_matched_count() );
```

FEEDER_LOAD_EVENT

FEEDER_LOAD_FILE

```
/* Usage */
FEEDER_LOAD_FILE *
    feeder_load_file_fetch(
        char *feeder_load_filename,
        int date_column /* one_based */,
        int description_column /* one_based */,
        int debit_column /* one_based */,
        int credit_column /* one_based */,
        int balance_column /* one_based */,
        int reference_column /* one_based */,
        boolean reverse_order_boolean );

/* Process */
FEEDER_LOAD_FILE *feeder_load_file_calloc(
    void );

char *feeder_load_file_basename_filename(
  char *feeder_load_filename );

FILE *feeder_load_file_input_open(
    feeder_load_filename );

LIST *feeder_load_row_list = list_new();

for char input[] in
    string_input(
        feeder_load_file_input_open() )
{
    if ( !*input )
    {
        ++row_number;
        continue;
    }

    remove_character( input, '\\' );

    FEEDER_LOAD_ROW *feeder_load_row =
        feeder_load_row_new(
            date_column /* one_based */,
            description_column /* one_based */,
            debit_column /* one_based */,
            credit_column /* one_based */,
            balance_column /* one_based */,
            reference_column  /* one_based */,
            remove_character() /* input */,
            ++row_number );

    if ( !feeder_load_row ) continue;

    if ( reverse_order_boolean )
    {
        list_set_first(
            feeder_load_row_list,
            feeder_load_row );
    }
    else
    {
        list_set(
            feeder_load_row_list,
            feeder_load_row );
    }
}

fclose( feeder_load_file_input_open() );

/* Public */
char *feeder_load_file_minimum_date(
    char *feeder_load_filename,
    int date_column /* one_based */ );
```



FEEDER_LOAD_ROW (1)

```
/* Usage */
FEEDER_LOAD_ROW *feeder_load_row_new(
   int date_column /* one_based */,
   int description_column /* one_based */,
   int debit_column /* one_based */,
   int credit_column /* one_based */,
   int balance_column /* one_based */,
   int reference_column  /* one_based */,
   char *input,
   int row_number );

/* Process */
FEEDER_LOAD_ROW *feeder_load_row_calloc(
   void );

/* Attribute */
int row_number;

char *american_date =
   string_strdup(
         piece_quote_comma(
               buffer[],
               input,
               date_column - 1 ) );

char *feeder_load_row_international_date(
   char *american_date );

if ( !feeder_load_row_international_date() )
{
   return (FEEDER_LOAD_ROW *)0;
}

char *file_row_description =
   string_strdup(
         piece_quote_comma(
               buffer[],
               input,
               description_column - 1 ) );

if ( !credit_column )
{
   double file_row_amount =
         string_atof(
               piece_quote_comma(
                     buffer[],
                     input,
                     debit_column - 1 ) );
}
else
{
   double debit =
         string_atof(
               piece_quote_comma(
                     buffer[],
                     input,
                     debit_column - 1 ) );

   double credit =
         string_atof(
               piece_quote_comma(
                     buffer[],
                     input,
                     credit_column - 1 ) );

   double file_row_amount =
         feeder_load_row_amount(
               double debit,
               double credit );
}

if ( !file_row_amount )
{
   return (FEEDER_LOAD_ROW *)0;
}

if ( balance_column )
{
   double file_row_balance =
         string_atof(
               piece_quote_comma(
                     buffer[],
                     input,
                     balance_column - 1 ) );
}

if ( reference_column )
{
   char *reference_string =
         string_strdup(
               piece_quote_comma(
                     buffer[],
                     input,
                     reference_column - 1 ) );
}

int feeder_load_row_check_number(
   char *file_row_description );

char *feeder_load_row_description_embedded(
   file_row_description,
   balance,
   reference_string,
   feeder_load_row_check_number() );
```

FEEDER_LOAD_ROW (2)

```
/* Usage */
char *feeder_load_row_description_embedded(
      char *file_row_description,
      double balance,
      char *reference_string,
      int feeder_load_row_check_number() );

/* Process */
if ( feeder_load_row_check_number() )
{
      return file_row_description;
}

char *feeder_load_row_trim_date(
      char *file_row_description );

char *sed_trim_double_spaces(
      feeder_load_row_trim_date() );

char *feeder_load_row_description_build(
      char *sed_trim_double_spaces(),
      double balance,
      char *reference_string );

char *feeder_load_row_description_crop(
      char *feeder_load_row_description_build(),
      int FEEDER_DESCRIPTION_SIZE );

/* Public */
double feeder_load_row_account_end_balance(
   FEEDER_LOAD_ROW *last_feeder_load_row );

double feeder_load_row_prior_account_end_balance(
   FEEDER_LOAD_ROW *first_feeder_load_row );
```

FEEDER_ROW (1)

```
/* Usage */
LIST *feeder_row_list(
   char *feeder_account,
   char *account_uncleared_checks(),
   LIST *feeder_phrase_list(),
   LIST *feeder_exist_row_list(),
   LIST *feeder_matched_journal_list(),
   LIST *feeder_load_row_list );

/* Process */
DATE *feeder_row_date( void );

for FEEDER_LOAD_ROW *feeder_load_row in
   feeder_load_row_list
{
   FEEDER_ROW *feeder_row_new(
         feeder_row_date() /* in/out */,
         feeder_account,
         account_uncleared_checks,
         feeder_phrase_list,
         feeder_exist_row_list,
         feeder_matched_journal_list,
         feeder_load_row,
         minimum_transaction_date_time );

   if (  feeder_row_new()->feeder_matched_journal
   ||    feeder_row_new()->feeder_phrase_seek() )
   {
         minimum_transaction_date_time =
               feeder_row_new()->
                     transaction_date_time );
   }

   list_set(
         list,
         feeder_row_new() );
}

/* Usage */
FEEDER_ROW *feeder_row_new(
   DATE *feeder_row_date /* in/out */,
   char *feeder_account,
   char *account_uncleared_checks,
   LIST *feeder_phrase_list(),
   LIST *feeder_exist_row_list(),
   LIST *feeder_matched_journal_list(),
   FEEDER_LOAD_ROW *feeder_load_row,
   char *minimum_transaction_date_time );

/* Process */
FEEDER_ROW *feeder_row_calloc(
   void );

/* Attribute */
FEEDER_LOAD_ROW *feeder_load_row;

FEEDER_EXIST_ROW *
   feeder_exist_row_seek(
         feeder_load_row->international_date,
         feeder_load_row->description_embedded,
         feeder_exist_row_list );

if ( feeder_exist_row_seek() )
{
   return feeder_row;
}

FEEDER_MATCHED_JOURNAL *feeder_matched_journal =
   feeder_matched_journal_check_seek(
         feeder_account,
         account_uncleared_checks,
         feeder_load_row->check_number,
         feeder_load_row->file_row_amount,
         feeder_matched_journal_list );

if ( !feeder_matched_journal )
{
   FEEDER_MATCHED_JOURNAL *
         feeder_matched_journal =
               feeder_matched_journal_amount_seek(
                     feeder_load_row->file_row_amount,
                     feeder_matched_journal_list );
}

if ( feeder_matched_journal )
{
   feeder_matched_journal->taken = 1;
}

if ( !feeder_matched_journal )
{
   FEEDER_PHRASE *feeder_phrase_seek(
         feeder_load_row->file_row_description,
         feeder_phrase_list );
}
```

feeder_exist_row_seek() feeder_matched_journal feeder_phrase_seek()

FEEDER_LOAD_ROW **FEEDER_EXIST_ROW** **FEEDER_MATCHED_JOURNAL** **FEEDER_PHRASE**

FEEDER_ROW (2)

```
/* Process (continued) */
if ( feeder_phrase_seek() )
{
   char *transaction_date_time =
         feeder_row_transaction_date_time(
               char *feeder_load_row->international_date,
               char *date_hms( feeder_row_date() ),
               char *minimum_transaction_date_time );

   date_increment_seconds( feeder_load_date(), 1 );

   char *memo =
         feeder_load_row->
               description_embedded;

   FEEDER_TRANSACTION *
         feeder_transaction =
               feeder_transaction_new(
                     feeder_account,
                     feeder_phrase_seek,
                     feeder_load_row->file_row_amount,
                     transaction_date_time,
                     memo );
}
else
if ( feeder_matched_journal )
{
      char *transaction_date_time =
            feeder_matched_journal->
                  transaction_date_time;
}

/* Usage */
void feeder_row_list_calculate_balance_set(
   LIST *feeder_row_list,
   double feeder_prior_account_end_balance(),
   FEEDER_ROW *feeder_row_first_out_balance() );
```

```
/* Process */
double prior_account_end_balance =
   feeder_prior_account_end_balance;

for FEEDER_ROW *feeder_row in
   feeder_row_list
{
   if (   feeder_row ==
          feeder_row_first_out_balance )
   {
          return;
   }

   if ( feeder_row->feeder_exist_row_seek )
   {
          feeder_row->
                calculate_balance =
                      feeder_row->
                            feeder_exist_row_seek->
                            file_row_balance;
   }
   else
   {
          feeder_row->
                calculate_balance =
                      prior_account_end_balance +
                      feeder_row->
                            feeder_load_row->
                            file_row_amount;

          prior_account_end_balance =
                feeder_row->
                      calculate_balance;
   }
}

/* Usage */
FEEDER_ROW *feeder_row_first_out_balance(
   LIST *feeder_row_list );

/* Process */
for FEEDER_ROW *feeder_row in
   feeder_row_list
{
   if (   !feeder_row->feeder_exist_row_seek
   &&  !feeder_row->feeder_matched_journal
   &&  !feeder_row->feeder_phrase_seek )
   {
          return feeder_row;
   }
}

return (FEEDER_ROW *)0;

/* Usage */
double feeder_row_list_sum_amount(
   LIST *feeder_row_list,
   FEEDER_ROW *feeder_row_first_out_balance )

/* Process */
for FEEDER_ROW *feeder_row in
   feeder_row_list
{
   if ( feeder_row == feeder_row_first_out_balance )
   {
          break;
   }

   if ( !feeder_row->feeder_exist_row_seek )
   {
          sum_amount +=
                feeder_row->
                      feeder_load_row->
                      file_row_amount;
   }
}
```

feeder_transaction

FEEDER_TRANSACTION

FEEDER_ROW (3)

```
/* Usage */
char *feeder_row_system_string(
      char *FEEDER_ROW_SELECT,
      char *FEEDER_ROW_TABLE,
      char *where );

/* Process */

/* Usage */
LIST *feeder_row_system_list(
      feeder_row_system_string() );

/* Process */
FILE *feeder_row_input_pipe(
      char *feeder_row_system_string );

for char input[] in
      string_input(
            feeder_row_input_pipe() )
{
      list_set(
            list,
            FEEDER_ROW *
                  feeder_row_parse(
                        input ) );
}

pclose( feeder_row_input_pipe() );

/* Usage */
FEEDER_ROW *feeder_row_parse(
   char *input );

/* Process */
FEEDER_ROW *feeder_row_calloc();

/* Public */
int feeder_row_seek_matched_count(
      LIST *feeder_row_list,
      FEEDER_ROW *feeder_row_first_out_balance() );

/* Driver */
boolean feeder_row_error_display(
      int balance_column,
      LIST *feeder_row_list );

/* Process */
LIST *feeder_row_error_extract_list(
      LIST *feeder_row_list );

boolean feeder_row_list_display(
      feeder_row_error_extract_list(),
      (FEEDER_ROW *)0
            /* feeder_row_first_out_balance */ );

/* Driver */
boolean feeder_row_list_display(
      int balance_column,
      LIST *feeder_row_list,
      FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
char *system_string =
      char *feeder_row_list_display_system_string(
            int balance_column );

FILE *display_pipe =
      FILE *feeder_row_list_display_pipe(
            system_string );

for FEEDER_ROW *feeder_row in
      feeder_row_list
{
      if (   feeder_row ==
            feeder_row_first_out_balance )
      {
            printf(
                  “%s\n”,
                  char *feeder_row_no_more_display() );
      }

      FILE *display_pipe =
            FILE *feeder_row_display_output(
                  balance_column,
                  display_pipe,
                  feeder_row );

      if ( !display_pipe
            && !list_at_end( feeder_row_list ) )
            {
                  FILE *display_pipe =
                        feeder_row_list_display_pipe(
                              system_string );
            }
}

if ( display_pipe ) pclose( display_pipe );

return (boolean)list_length( feeder_row_list );

/* Usage */
FILE *feeder_row_display_output(
      int balance_column,
      FILE *display_pipe,
      FEEDER_ROW *feeder_row );

/* Process */
char *feeder_row_display_results(
      FEEDER_EXIST_ROW *
            feeder_exist_row_seek,
      FEEDER_MATCHED_JOURNAL *
            feeder_matched_journal,
      FEEDER_PHRASE *
            feeder_phrase_seek );

if ( feeder_transaction )
{
      pclose( display_pipe );
      display_pipe = (FILE *)0;

      journal_list_html_display(
            feeder_transaction->
                  transaction->
                  journal_list,
            feeder_transaction->
                  transaction->
                  transaction_date_time,
            feeder_transaction->
                  transaction->
                  memo );
}
```

FEEDER_ROW (4)

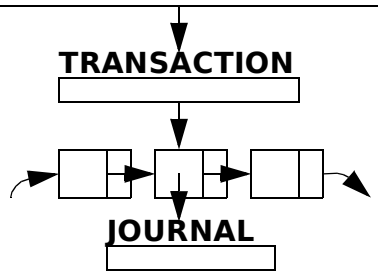
```
/* Driver */
void feeder_row_list_insert(
	char *feeder_account,
	char *feeder_load_event->feeder_load_date_time,
	LIST *feeder_row_list,
	FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
char *feeder_row_list_insert_system_string(
	char *FEEDER_ROW_INSERT_COLUMNS,
	char *FEEDER_ROW_TABLE,
	char SQL_DELIMITER );

FILE *feeder_row_list_insert_open(
	feeder_row_list_insert_system_string() );

for FEEDER_ROW *feeder_row in feeder_row_list
{
	if (	feeder_row == feeder_row_first_out_balance )
	{
		break;
	}

	if ( feeder_row->feeder_exist_row_seek )
	{
		continue;
	}

	JOURNAL *feeder_row_journal(
		FEEDER_PHRASE *
			feeder_row->feeder_phrase_seek,
		char *feeder_row->
			transaction_date_time,
		FEEDER_MATCHED_JOURNAL *
			feeder_row->feeder_matched_journal );

	char *feeder_row_phrase(
		FEEDER_PHRASE *
			feeder_row->feeder_phrase_seek );

	void feeder_row_insert_pipe(
		FILE *feeder_row_list_insert_open(),
		char *feeder_account,
		char *feeder_load_date_time,
		char *feeder_row->
			feeder_load_row->
			international_date,
		int feeder_row->
			feeder_load_row->
			row_number,
		char *feeder_row_journal()->
			full_name,
		char *feeder_row_journal()->
			street_address,
		char *feeder_row_journal()->
			transaction_date_time,
		char *feeder_row->
			feeder_load_row->
			description_embedded,
		double feeder_row->
			feeder_load_row->
			file_row_amount,
		double feeder_row->
			feeder_load_row->
			file_row_balance,
		double feeder_row->
			calculate_balance,
		int feeder_row->
			feeder_load_row->
			check_number,
		char *feeder_row_phrase(),
		char SQL_DELIMITER );
}

pclose( feeder_row_list_insert_open() );

/* Driver */
void feeder_row_transaction_insert(
	char *feeder_account,
	char *feeder->account_uncleared_checks,
	LIST *feeder->feeder_row_list,
	FEEDER_ROW *feeder->
		feeder_row_first_out_balance );

/* Process */
LIST *feeder_row_extract_transaction_list(
	LIST *feeder_row_list,
	FEEDER_ROW *feeder_row_first_out_balance ),

char *transaction_list_insert(
	feeder_row_extract_transaction_list(),
	0 /* not insert_journal_list_boolean */ );

void transaction_journal_list_insert(
   feeder_row_extract_transaction_list(),
   1 /* with_propagate */ );

char *first_transaction_date_time =
	feeder_row_check_journal_update(
		feeder_row_list,
		feeder_row_first_out_balance );

if ( first_transaction_date_time )
{
	journal_propagate(
		first_transaction_date_time,
		feeder_account );

	journal_propagate(
		first_transaction_date_time,
		account_uncleared_checks );
}

/* Usage */
LIST *feeder_row_extract_transaction_list(
	LIST *feeder_row_list,
	FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
for FEEDER_ROW *feeder_row in
	feeder_row_list
{
	if (	feeder_row ==
		feeder_row_first_out_balance )
	{
		break;
	}

	if ( feeder_row->feeder_transaction )
	{
		list_set(
			transaction_list,
			feeder_row->feeder_transaction );
	}
}

/* Usage */
/* Returns first_transaction_date_time */
char *feeder_row_check_journal_update(
	LIST *feeder_row_list,
	FEEDER_ROW *feeder_row_first_out_balance );

/* Process */
FILE *feeder_row_check_journal_update_pipe(
	void );
```

FEEDER_EXIST_ROW

```
/* Usage */
LIST *feeder_exist_row_list(
      char *feeder_account,
      char *feeder_load_file_minimum_date() );

/* Process */
LIST *list = list_new();

char *feeder_exist_row_where(
      char *feeder_account,
      char *feeder_load_file_minimum_date );

char *feeder_exist_row_system_string(
      char *FEEDER_EXIST_ROW_SELECT,
      char *FEEDER_ROW_TABLE,
      char *feeder_exist_row_where() );

FILE *feeder_exist_row_input_open(
      char *feeder_exist_row_system_string() );

while ( input[] = string_input( input_open ) )
{
      list_set(
            list,
            FEEDER_EXIST_ROW *
                  feeder_exist_row_parse(
                        input ) );
}

pclose( feeder_exist_row_input_open() );

/* Usage */
FEEDER_EXIST_ROW *
      feeder_exist_row_parse(
            char *input );

/* Process */
FEEDER_EXIST_ROW *feeder_exist_row_calloc(
      void );

/* Attributes */
char *feeder_date;
char *file_row_description;
char *transaction_date_time;
double file_row_balance;

/* Public */
FEEDER_EXIST_ROW *
      feeder_exist_row_seek(
            char *international_date,
            char *description_embedded,
            LIST *feeder_exist_row_list );
```

FEEDER_TRANSACTION

```
/* Usage */
FEEDER_TRANSACTION *
   feeder_transaction_new(
         char *feeder_account,
         FEEDER_PHRASE *feeder_phrase_seek(),
         double file_row_amount,
         char *transaction_date_time,
         char *memo );

/* Process */
FEEDER_TRANSATION *feeder_transaction_calloc(
   void );

if ( file_row_amount < 0.0 )
{
   char *debit_account_name =
               feeder_phrase_seek->
                     nominal_account;

   char *credit_account_name = feeder_account;

   file_row_amount = -file_row_amount;
}
else
{
   char *debit_account_name = feeder_account;

   char *credit_account_name =
               feeder_phrase_seek->
                     nominal_account;
}

TRANSACTION *transaction =
   transaction_binary(
         feeder_phrase_seek->
               full_name,
         feeder_phrase_seek->
               street_address,
         transaction_date_time,
         file_row_amount
               /* transaction_amount */,
         memo,
         debit_account_name,
         credit_account_name );
```



FEEDER_MATCHED_JOURNAL

```
/* Usage */
LIST *feeder_matched_journal_list(
   char *feeder_account,
   char *FEEDER_ROW_TABLE,
   char *feeder_load_file_minimum_date(),
   char *account_uncleared_checks(
         char *ACCOUNT_UNCLEARED_CHECKS_KEY ) );

/* Process */
char *feeder_matched_journal_subquery(
   char *feeder_account,
   char *account_uncleared_checks,
   char *JOURNAL_TABLE,
   char *FEEDER_ROW_TABLE );

int feeder_matched_journal_days_ago(
   char *FEEDER_LOAD_TRANSACTION_DAYS_AGO );

char *feeder_matched_journal_minimum_date(
   char *feeder_load_file_minimum_date,
   int feeder_matched_journal_days_ago() );

char *feeder_matched_journal_where(
   char *feeder_account,
   char *account_uncleared_checks,
   char *feeder_matched_journal_minimum_date() );

LIST *journal_system_list(
   char *journal_system_string(
         JOURNAL_SELECT,
         JOURNAL_TABLE,
         feeder_matched_journal_list_where() ),
   0 /* not fetch_account */,
   0 /* not fetch_subclassification */,
   0 /* not fetch_element */,
   1 /* fetch_transaction */ );

if ( !list_rewind( journal_system_list() )
{
   return (LIST *)0;
}

LIST *list = list_new();

for JOURNAL *journal in journal_system_list()
{
   list_set(
         list,
         FEEDER_MATCHED_JOURNAL *
               feeder_matched_journal_new(
                     journal ) );
}

/* Usage */
FEEDER_MATCHED_JOURNAL *
   feeder_matched_journal_new(
         JOURNAL *journal );

/* Process */
FEEDER_MATCHED_JOURNAL *
   feeder_matched_journal_calloc(
         void );

double feeder_matched_journal_amount(
   double journal->debit_amount,
   double journal->credit_amount );

/* Attributes */
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double debit_amount;
double credit_amount;
double amount;
int check_number;
boolean taken;
char *subquery;
char *where;
LIST *journal_system_list;
char *check_update_statement;

/* Usage */
FEEDER_MATCHED_JOURNAL *
   feeder_matched_journal_check_seek(
         char *feeder_account,
         char *account_uncleared_checks,
         int check_number,
         double amount,
         LIST *feeder_matched_journal_list() );

/* Process */
char *
   feeder_matched_journal_check_update_statement(
         char *JOURNAL_TABLE,
         char *feeder_account,
         char *account_uncleared_checks,
         char *full_name,
         char *street_address,
         char *transaction_date_time );

/* Public */
FEEDER_MATCHED_JOURNAL *
   feeder_matched_journal_amount_seek(
         double amount,
         LIST *feeder_matched_journal_list() );

/* Driver */
void feeder_matched_journal_not_taken_display(
   LIST *feeder_matched_journal_list() );

/* Process */
char *feeder_matched_journal_not_taken_system_string(
   void );
```

FEEDER_PHRASE

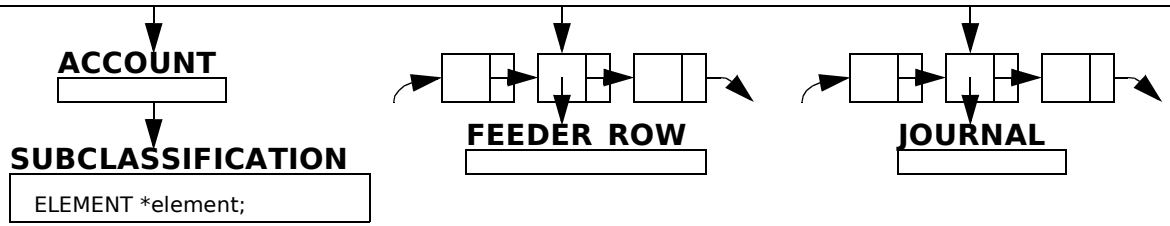
```
/* Usage */
LIST *feeder_phrase_list(
	char *FEEDER_PHRASE_SELECT,
	char *FEEDER_PHRASE_TABLE );

/* Process */
char *feeder_phrase_system_string(
	char *FEEDER_PHRASE_SELECT,
	char *FEEDER_PHRASE_TABLE );

FILE *feeder_phrase_pipe_open(
	char *feeder_phrase_system_string() );

LIST *list = list_new();

while ( input[] = string_input( feeder_phrase_pipe_open() ) )
{
	list_set(
		list,
		feeder_phrase_parse( input ) );
}

pclose( feeder_phrase_pipe_open() );

/* Usage */
FEEDER_PHRASE *feeder_phrase_parse(
	char *input );

/* Process */
FEEDER_PHRASE *feeder_phrase_new(
	char *phrase );

FEEDER_PHRASE *feeder_phrase_calloc( void );

/* Attributes */
char *phrase;
char *nominal_account;
char *full_name;
char *street_address;
boolean feeder_phrase_ignore;

/* Usage */
FEEDER_PHRASE *feeder_phrase =
	feeder_phrase_seek(
		char *file_row_description,
		LIST *feeder_phrase_list() );

/* Process */
char feeder_component[];

for FEEDER_PHRASE *feeder_phrase in feeder_phrase_list
{
	for ( int piece_number = 0;
		piece(	feeder_component,
			char feeder_phrase_delimiter( feeder_phrase->phrase ),
			feeder_phrase->phrase,
			piece_number );
		piece_number++ )
	{
		if ( string_exists_substr(
			file_row_description /* string */,
			feeder_component /* substring */ ) )
		{
			return feeder_phrase;
		}
	}
}
```

FEEDER_LOAD_EVENT

```
/* Usage */
FEEDER_LOAD_EVENT *feeder_load_event_new(
	char *feeder_account,
	char *feeder_load_date_time,
	char *login_name,
	char *feeder_load_file_basename_filename(),
	char *feeder_account_end_date(),
	double feeder_account_end_calculate_balance() );

/* Process */
FEEDER_LOAD_EVENT *feeder_load_event_calloc(
	void );

/* Usage */
FEEDER_LOAD_EVENT *feeder_load_event_fetch(
	char *feeder_account,
	char *feeder_load_date_time );

/* Process */
char *feeder_load_event_primary_where(
	char *feeder_account,
	char *feeder_load_date_time );

char *feeder_load_event_system_string(
	char *FEEDER_LOAD_EVENT_SELECT,
	char *FEEDER_LOAD_EVENT_TABLE,
	char *feeder_load_event_primary_where() );

FILE *feeder_load_event_input_open(
	char *feeder_load_event_system_string() );

FEEDER_LOAD_EVENT *feeder_load_event_parse(
	char *string_input(
		input[],
		feeder_load_event_input_open() )
		/* input */ );

pclose( feeder_load_event_input_open() );

/* Usage */
FEEDER_LOAD_EVENT *
	feeder_load_event_latest_fetch(
		char *FEEDER_LOAD_EVENT_TABLE,
		char *feeder_account );

/* Process */
char *feeder_load_event_account_where(
	char *feeder_account );

char *feeder_load_event_latest_system_string(
	char *FEEDER_LOAD_EVENT_TABLE,
	char *feeder_load_event_account_where() );

char *feeder_load_event_latest_date_time(
	char *feeder_load_event_latest_system_string() );

FEEDER_LOAD_EVENT *
	feeder_load_event_fetch(
		feeder_account,
		feeder_load_event_latest_date_time() );

/* Attributes */
char *feeder_account;
char *feeder_load_date_time;
char *login_name;
char *basename_filename;
char *account_end_date;
double account_end_calculate_balance;

/* Driver */
void feeder_load_event_insert(
	char *FEEDER_LOAD_EVENT_TABLE,
	char *FEEDER_LOAD_EVENT_INSERT,
	char *feeder_load_event->feeder_account,
	char *feeder_load_event->feeder_load_date_time,
	char *feeder_load_event->login_name,
	char *feeder_load_file->basename_filename,
	char *feeder_load_event->account_end_date,
	double feeder_load_event->
		account_end_calculate_balance );

/* Process */
char *feeder_load_event_insert_system_string(
	char *FEEDER_LOAD_EVENT_TABLE,
	char *FEEDER_LOAD_EVENT_INSERT,
	char SQL_DELIMITER );

FILE *feeder_load_event_insert_open(
	char *feeder_load_event_insert_system_string() );

void feeder_load_event_insert_pipe(
	FILE *feeder_load_event_insert_open(),
	char SQL_DELIMITER,
	char *feeder_account,
	char *feeder_load_date_time,
	char *login_name,
	char *basename_filename,
	char *account_end_date,
	double account_end_calculate_balance );

pclose( feeder_load_event_insert_open() );
```

FEEDER_AUDIT

```
/* Usage */
FEEDER_AUDIT *feeder_audit_fetch(
   char *feeder_account_name,
   char *feeder_load_date_time );

/* Process */
FEEDER_AUDIT *feeder_audit_calloc( void );

ACCOUNT *feeder_account =
   account_fetch(
         feeder_account_name,
         1 /* fetch_subclassification */,
         1 /* fetch_element */ );

char *feeder_load_event_primary_where(
   feeder_account_name,
   feeder_load_date_time );

char *feeder_row_system_string(
   FEEDER_ROW_SELECT,
   FEEDER_ROW_TABLE,
   feeder_load_event_primary_where()
         /* where */ );

LIST *feeder_row_list =
   LIST *feeder_row_system_list(
         feeder_row_system_string() );

if ( !list_length( feeder_row_list ) )
   return (FEEDER_AUDIT *)0;

FEEDER_ROW *first_feeder_row =
   list_first( feeder_row_list );

JOURNAL *journal_prior(
   first_feeder_row->transaction_date_time,
   feeder_account_name,
   0 /* not fetch_account */,
   0 /* not fetch_subclassification */,
   0 /* not fetch_element */ );

LIST *journal_list_prior(
   journal_prior(),
   feeder_account_name );

HTML_TABLE *html_table =
   html_table_new(
         FEEDER_AUDIT_HTML_TITLE,
         (char *)0 /* sub_title */,
         (char *)0 /* sub_sub_title */ );

html_table->heading_list =
   LIST *feeder_audit_html_heading_list(
         void );

if ( journal_prior )
{
   char *prior_transaction_date_time =
         journal_prior()->transaction_date_time;
}

for FEEDER_ROW *feeder_row in
   feeder_row_list
{
   JOURNAL *journal =
         journal_seek(
               feeder_row->transaction_date_time,
               feeder_account_name,
               journal_list_prior() );

   if ( !journal ) exit( 1 );

   double journal_balance(
         journal->balance,
         feeder_account->
               subclassification->
               element->
               accumulate_debit );

   if (   feeder_row->calculate_balance !=
         journal_balance()
   ||     list_at_last( feeder_row_list ) )
   {
         list_set(
               html_table->row_list,
               HTML_ROW *feeder_audit_html_row(
                     prior_transaction_date_time,
                     feeder_row,
                     journal_balance(),
                     list_at_last( feeder_row_list ) );
   }

   char *prior_transaction_date_time =
         feeder_row->transaction_date_time;
}

/* Usage */
HTML_ROW *feeder_audit_html_row(
   char prior_transaction_date_time,
   FEEDER_ROW *feeder_row,
   double journal_balance(),
   boolean list_at_last );

/* Process */
LIST *feeder_audit_html_cell_list(
   int feeder_row->row_number,
   char *feeder_row->full_name,
   char *feeder_row->file_row_description,
   char *prior_transaction_date_time,
   char *feeder_row->transaction_date_time,
   char *feeder_row->feeder_date,
   double feeder_row->file_row_amount,
   int feeder_row->check_number,
   double feeder_row->calculate_balance,
   double journal_balance(),
   boolean list_at_last );

/* Driver */
void html_table_output(
   feeder_audit->html_table,
   HTML_TABLE_ROWS_BETWEEN_HEADING );
```



ACCURAL (1)

```
/* Usage */
LIST *accrual_list_fetch(
   char *application_name );

/* Process */
char *accrual_select(
   ACCRUAL_SELECT_ATTRIBUTES,
   ACCRUAL_PROPERTY_ATTRIBUTE,
   accrual_property_attribute_exists() );

char *accrual_system_string(
   accrual_select(),
   ACCRUAL_TABLE,
   “1 = 1” /* where */ );

LIST *accrual_system_list(
   accrual_system_string(),
   accrual_property_attribute_exists );

void accrual_list_transaction_set(
   accrual_system_list() /* in/out */ );

/* Usage */
LIST *accrual_system_list(
   char *accrual_system_string(),
   boolean accrual_property_attribute_exists() );

/* Process */
FILE *accrual_input_pipe(
   char *accrual_system_string() );

for input[] in string_input( accrual_input_pipe() )
{
   list_set(
         accrual_list,
         ACCRUAL *
               accrual_parse(
                     accrual_property_attribute_exists,
                     input ) );
}

void pclose( accrual_input_pipe() );

/* Usage */
ACCRUAL *accrual_fetch(
   char *application_name,
   char *full_name,
   char *street_address,
   char *accrual_description );

/* Process */
boolean accrual_property_attribute_exists(
   application_name,
   ACCRUAL_TABLE,
   ACCRUAL_PROPERTY_ATTRIBUTE );

char *accrual_select(
   ACCRUAL_SELECT_ATTRIBUTES,
   ACCRUAL_PROPERTY_ATTRIBUTE,
   accrual_property_attribute_exists );

char *accrual_primary_where(
   full_name,
   street_address,
   accrual_description );

char *accrual_system_string(
   accrual_select(),
   ACCRUAL_TABLE,
   accrual_primary_where() );

ACCRUAL *accrual_parse(
   accrual_property_attribute_exists,
   string_pipe_fetch(
         accrual_system_string() ) );

TRANSACTION *accrual_parse()->transaction =
   TRANSACTION *accrual_transaction(
         accrual_parse()->full_name,
         accrual_parse()->street_address,
         accrual_parse()->accrual_description,
         accrual_parse()->
               transaction_increment_date_time,
         accrual_parse()->debit_account,
         accrual_parse()->credit_account,
         accrual_parse()->accrued_daily_amount,
         accrual_parse()->accrued_monthly_amount,
         accrual_parse()->
               rental_property_street_address );

/* Usage */
ACCRUAL *accrual_parse(
   boolean accrual_property_attribute_exists,
   char *input );

/* Process */
ACCRUAL *accrual_new(
   strdup( full_name ),
   strdup( street_address ),
   strdup( accrual_description ) );

/* Increments second each invocation. */
char *transaction_increment_date_time(
   (char *)0 /* transaction_date */ );

/* Usage */
ACCRUAL *accrual_new(
   char *full_name,
   char *street_address,
   char *accrual_description );

/* Process */
ACCRUAL *accrual_calloc( void );

/* Attributes */
char *full_name;
char *street_address;
char *accrual_description;
char *debit_account;
char *credit_account;
double accrued_daily_amount;
double accrued_monthly_amount;
boolean property_attribute_exists;
char *rental_property_street_address;
char *transaction_increment_date_time;
TRANSACTION *transaction;

/* Usage */
void accrual_list_transaction_set(
   LIST *accrual_list() /* in/out */ );

/* Process */
for ACCRUAL *accrual in accrual_list()
{
   TRANSACTION *accrual->transaction =
         TRANSACTION *accrual_transaction(
               accrual->full_name,
               accrual->street_address,
               accrual->accrual_description,
               accrual->
                     transaction_increment_date_time,
               accrual->debit_account,
               accrual->credit_account,
               accrual->accrued_daily_amount,
               accrual->accrued_monthly_amount,
               accrual->
                     rental_property_street_address );
}
```

ACCRUAL (2)

```
/* Usage */
TRANSACTION *accrual_transaction(
   char *full_name,
   char *street_address,
   char *accrual_description,
   char *transaction_increment_date_time,
   char *debit_account,
   char *credit_account,
   double accrued_daily_amount,
   double accrued_monthly_amount,
   char *rental_property_street_address );

/* Process */
if ( accrued_daily_amount )
{
   TRANSACTION *accrual_daily_transaction(
         full_name,
         street_address,
         accrual_description,
         transaction_increment_date_time,
         debit_account,
         credit_account,
         accrued_daily_amount );
}
else
if ( accrued_monthly_amount )
{
   TRANSACTION *accrual_monthly_transaction(
         full_name,
         street_address,
         accrual_description,
         transaction_increment_date_time,
         debit_account,
         credit_account,
         accrued_monthly_amount,
         rental_property_street_address );
}

/* Usage */
TRANSACTION *accrual_daily_transaction(
   char *full_name,
   char *street_address,
   char *transaction_description,
   char *transaction_increment_date_time,
   char *debit_account,
   char *credit_account,
   double accrued_daily_amount );

/* Process */
int accrual_last_transaction_days_between(
   TRANSACTION_TABLE,
   full_name,
   street_address,
   transaction_increment_date_time,
   debit_account,
   credit_account );

if ( !accrual_last_transaction_days_between() )
{
   return (TRANSACTION *)0;
}

double accrual_daily_accrued_amount(
   double accrued_daily_amount,
   int accrual_last_transaction_days_between() );

char *accrual_memo(
   accrual_description,
   credit_account );

TRANSACTION *transaction_binary(
   full_name,
   street_address,
   transaction_increment_date_time,
   accrual_daily_accrued_amount(),
   accrual_memo(),
   debit_account,
   credit_account );

/* Usage */
TRANSACTION *accrual_monthly_transaction(
   char *full_name,
   char *street_address,
   char *accrual_description,
   char *transaction_increment_date_time,
   char *debit_account,
   char *credit_account,
   double accrued_monthly_amount,
   char *rental_property_street_address );

/* Process */
char *accrual_max_transaction_date_time(
   TRANSACTION_TABLE,
   full_name,
   street_address,
   debit_account,
   credit_account );

double accrual_monthly_accrue(
   accrual_max_transaction_date_time(),
   transaction_increment_date_time,
   accrued_monthly_amount );

char *accrual_memo(
   accrual_description,
   credit_account );

TRANSACTION *transaction_binary(
   full_name,
   street_address,
   transaction_increment_date_time,
   accrual_monthly_accrue(),
   accrual_memo(),
   debit_account,
   credit_account );
```

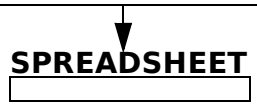
TRANSACTION

ACCRUAL (3)

```
/* Usage */
int accrual_last_transaction_days_between(
   char *TRANSACTION_TABLE,
   char *full_name,
   char *street_address,
   char *transaction_increment_date_time,
   char *debit_account,
   char *credit_account );

char *accrual_max_transaction_date_time(
   TRANSACTION_TABLE,
   full_name,
   street_address,
   debit_account,
   credit_account );

int date_days_between(
   accrual_max_transaction_date_time(),
   transaction_increment_date_time );

/* Usage */
char *accrual_max_transaction_date_time(
   char *TRANSACTION_TABLE,
   char *full_name,
   char *street_address,
   char *debit_account,
   char *credit_account );

/* Process */
char *accrual_journal_transaction_subquery(
   char *JOURNAL_TABLE,
   char *TRANSACTION_TABLE,
   char *debit_account,
   char *credit_account );

/* Usage */
char *accrual_primary_where(
   char *full_name,
   char *street_address,
   char *accrual_description );

/* Process */
char *entity_escape_full_name(
   full_name );

char *accrual_escape_transaction_description(
   char *transaction_description );

/* Public */
boolean accrual_property_attribute_exists(
   char *application_name,
   char *ACCRUAL_TABLE,
   char *ACCRUAL_PROPERTY_ATTRIBUTE );

char *accrual_select(
   char *ACCRUAL_SELECT_ATTRIBUTES,
   char *ACCRUAL_PROPERTY_ATTRIBUTE,
   boolean accrual_property_attribute_exists() );

char *accrual_system_string(
   char *accrual_select(),
   char *ACCRUAL_TABLE,
   char *where );

char *accrual_memo(
   char *accrual_description,
   char *credit_account );

LIST *accrual_transaction_list_extract(
   LIST *accrual_list );

/* Driver */
if ( !execute && with_html )
{
   if ( list_length( accrual_list ) )
   {
         void transaction_list_html_display(
               LIST *accrual_transaction_list_extract(
                     accrual_list ) );
   }
   else
   if ( accrual )
   {
         void transaction_html_display(
               accrual->transaction );
   }
}
else
if ( execute )
{
   if ( list_length( accrual_list ) )
   {
         void transaction_list_insert(
               LIST *accrual_transaction_list_extract(
                     accrual_list ),
               1 /* insert_journal_list_boolean */ );
   }
   else
   if ( accrual )
   {
         char *transaction_date_time =
               transaction_stamp_insert(
                     accrual->transaction,
                     1 /* insert_journal_list_boolean */ );
   }
}
```

PAYPAL

```
/* Usage */
PAYPAL *paypal_fetch(
      spreadsheet_filename,
      date_label );

/* Process */
SPREADSHEET *spreadsheet =
      spreadsheet_fetch(
            spreadsheet_filename,
            date_label );
```

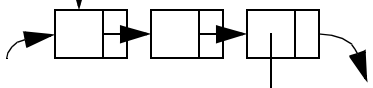


PAYPAL_DATASET

```
/* Input */
char *date_A;
char *time_B;
char *timezone_C;
char *full_name_D;
char *transaction_type_E;
char *transaction_status_F;
char *currency_G;
char *gross_revenue_H;
char *transaction_fee_I;
char *net_revenue_J;
char *from_email_address_K;
char *to_email_address_L;
char *transaction_ID_M;
char *shipping_address_N;
char *address_status_O;
char *item_title_P;
char *item_ID_Q;
char *shipping_handling_amount_R;
char *insurance_amount_S;
char *sales_tax_T;
char *option_1_name_U;
char *option_1_value_V;
char *option_2_name_W;
char *option_2_value_X;
char *reference_txn_ID_Y;
char *invoice_number_Z;
char *custom_number_AA;
char *quantity_AB;
char *receipt_ID_AC;
char *balance_AD;
char *address_line_1_AE;
char *address_2_district_neighborhood_AF;
char *town_city_AG;
char *state_province_AH;
char *zip_code_AI;
char *country_AJ;
char *contact_phone_number_AK;
char *subject_AL;
char *note_AM;
char *country_code_AN;
char *balance_impact_AO;
```

SPREADSHEET

```
/* Input */
char *spreadsheet_filename;
char *date_label;

/* Process */
LIST *spreadsheet_column_list(
      spreadsheet_filename,
      date_label );

char *maximum_date;

char *spreadsheet_minimum_date(
      &maximum_date,
      spreadsheet_filename );

char *spreadsheet_heading_data(
      spreadsheet_column_list(),
      input_row,
      heading );
```



SPREADSHEET_COLUMN

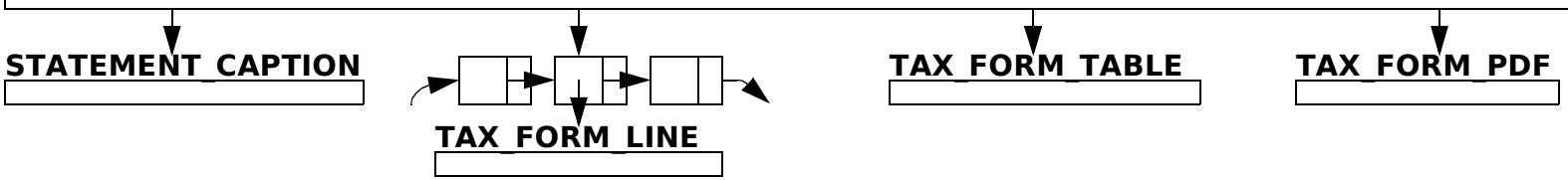
```
/* Usage */
SPREADSHEET_COLUMN *spreadsheet_column_new(
      heading,
      piece_offset );

/* Process */
SPREADSHEET_COLUMN *spreadsheet_column_heading_seek(
      spreadsheet_column_list,
      heading );
```

SPREADSHEET_UPLOAD_EVENT

```
/* Input */
char *spreadsheet_filename;

/* Process */
char *spreadsheet_upload_date_time =
      date_time19_now();

char *spreadsheet_upload_file_sha256sum(
      spreadsheet_filename );
```

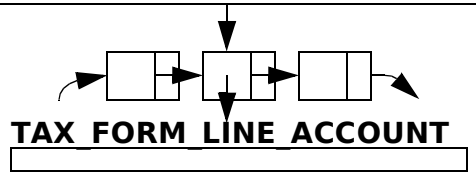
TAX_FORM

```
/* Usage */
TAX_FORM *tax_form_fetch(
      char *application_name,
      char *process_name,
      char *document_root_directory,
      char *tax_form_name,
      int tax_year,
      int fiscal_begin_month /* zero or one based */,
      char *output_medium_string );

/* Process */
TAX_FORM *tax_form_calloc( void );

enum statement_output_medium
      statement_output_medium =
            statement_resolve_output_medium(
                  char *output_medium_string );

char *tax_form_fiscal_begin_date(
      int tax_year,
      int fiscal_begin_month );

char *tax_form_fiscal_end_date(
      int tax_year,
      int fiscal_begin_month );

STATEMENT_CAPTION *statement_caption =
      statement_caption_new(
            application_name,
            process_name,
            tax_form_fiscal_begin_date,
            tax_form_fiscal_end_date_string );

LIST *tax_form_line_list(
      tax_form_name,
      tax_form_fiscal_begin_date(),
      tax_form_fiscal_end_date() );

if (  statement_output_medium ==
      statement_output_table )
{
      TAX_FORM_TABLE *tax_form_table =
            tax_form_table_new(
                  tax_form_name,
                  statement_caption,
                  tax_form_line_list() );
}
else
if (  statement_output_medium ==
      statement_output_PDF )
{
      TAX_FORM_PDF *tax_form_pdf =
            tax_form_pdf_new(
                  application_name,
                  process_name,
                  document_root_directory,
                  tax_form_name,
                  statement_caption,
                  tax_form_line_list() );
}

/* Attributes */
enum statement_output_medium
      statement_output_medium;
char *fiscal_begin_date;
char *fiscal_end_date;
LIST *tax_form_line_list;
TAX_FORM_TABLE *tax_form_table;
TAX_FORM_PDF *tax_form_pdf;

/* Public */
char *tax_form_primary_where(
      char *tax_form_name );

/* Driver */
if ( tax_form->tax_form_table )
{
      void html_table_list_output(
            tax_form->
                  tax_form_table->
                  html_table_list,
            HTML_TABLE_ROWS_BETWEEN_HEADING );
}
else
if ( tax_form->tax_form_pdf )
{
      void statement_latex_output(
            tax_form->
                  tax_form_pdf->
                  latex,
            tax_form->
                  tax_form_pdf->
                  statement_link->
                  ftp_output_filename,
            statement_pdf_prompt( process_name ),
            process_name,
            tax_form->
                  statement_caption->
                  date_time_string );
}
```



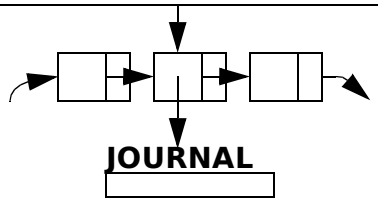
TAX_FORM_LINE

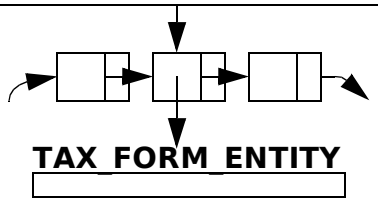
```
/* Usage */
LIST *tax_form_line_list(
	char *tax_form_name,
	char *tax_form_fiscal_begin_date(),
	char *tax_form_fiscal_end_date() );

/* Process */
char *tax_form_primary_where(
	tax_form_name );

char *tax_form_line_system_string(
	char *TAX_FORM_LINE_SELECT,
	char *TAX_FORM_LINE_TABLE,
	char *tax_form_primary_where() );

LIST *tax_form_line_system_list(
	tax_form_name,
	tax_form_fiscal_begin_date,
	tax_form_fiscal_end_date,
	tax_form_line_system_string() );

/* Usage */
LIST *tax_form_line_system_list(
	tax_form_name,
	char *tax_form_fiscal_begin_date(),
	char *tax_form_fiscal_end_date(),
	char *tax_form_line_system_string() );

/* Process */
FILE *tax_form_line_input_pipe(
	char *tax_form_line_system_string );

for input[] in string_input( tax_form_line_input_pipe() )
{
	list_set(
		list,
		tax_form_line_parse(
			tax_form_name,
			tax_form_fiscal_begin_date,
			tax_form_fiscal_end_date,
			input ) );
}

pclose( tax_form_line_input_pipe() );

/* Usage */
TAX_FORM_LINE *tax_form_line_parse(
	char *tax_form_name,
	char *tax_form_fiscal_begin_date,
	char *tax_form_fiscal_end_date,
	char *input );

/* Process */
TAX_FORM_LINE *tax_form_line_calloc( void );

LIST *tax_form_line_account_list(
	tax_form_name,
	tax_form_fiscal_begin_date,
	tax_form_fiscal_end_date,
	tax_form_line->string );

double tax_form_line_total(
	LIST *tax_form_line_account_list() );

/* Attributes */
char *tax_form_name;
char *string;
char *description;
LIST *tax_form_line_account_list;
double total;

/* Public */
char *tax_form_line_where(
	char *tax_form_name,
	char *tax_form_line_string );
```



TAX_FORM_LINE_ACCOUNT

```
/* Usage */
LIST *tax_form_line_account_list(
      char *tax_form_name,
      char *tax_form_fiscal_begin_date(),
      char *tax_form_fiscal_end_date(),
      char *tax_form_line_string );

/* Process */
char *tax_form_line_where(
      tax_form_name,
      tax_form_line_string );

char *tax_form_line_account_system_string(
      char *TAX_FORM_LINE_ACCOUNT_SELECT,
      char *TAX_FORM_LINE_ACCOUNT_TABLE,
      char *tax_form_line_where() );

LIST *tax_form_line_account_system_list(
      tax_form_name,
      tax_form_fiscal_begin_date,
      tax_form_fiscal_end_date,
      tax_form_line_string,
      tax_form_line_account_system_string() );

/* Usage
LIST *tax_form_line_account_system_list(
      char *tax_form_name,
      char *tax_form_fiscal_begin_date,
      char *tax_form_fiscal_end_date,
      char *tax_form_line_string,
      char *tax_form_line_account_system_string() );

/* Process */
FILE *tax_form_line_account_input_pipe(
      char *tax_form_line_account_system_string );

for input[] in string_input(
tax_form_line_account_input_pipe() )
{
      list_set(
            list,
            tax_form_line_account_parse(
                  tax_form_name,
                  tax_form_fiscal_begin_date,
                  tax_form_fiscal_end_date,
                  tax_form_line_string,
                  input ) );
}

pclose( tax_form_line_account_input_pipe() );

/* Usage */
TAX_FORM_LINE_ACCOUNT *
      tax_form_line_account_parse(
            char *tax_form_name,
            char *tax_form_fiscal_begin_date,
            char *tax_form_fiscal_end_date,
            char *tax_form_line_string,
            char *input );

/* Process */
TAX_FORM_LINE_ACCOUNT *
      tax_form_line_account_calloc(
            void );

LIST *journal_tax_form_list(
      tax_form_fiscal_begin_date,
      tax_form_fiscal_end_date,
      TRANSACTION_PRECLOSE_TIME,
      account_name );

double tax_form_line_account_total(
      journal_tax_form_list() );

LIST *tax_form_entity_list(
      journal_tax_form_list() );

/* Usage */
double tax_form_line_account_total(
      LIST *journal_tax_form_list() );

/* Process */
JOURNAL *last_journal =
      list_last( journal_tax_form_list );

return last_journal->balance;

/* Attributes */
char *tax_form_name;
char *tax_form_line_string;
char *account_name;
LIST *journal_tax_form_list;
double total;
LIST *tax_form_entity_list;
```





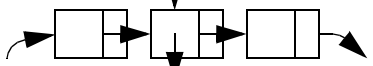
TAX_FORM_ENTITY

```
/* Usage */
LIST *tax_form_entity_list(
	LIST *journal_tax_form_list() );

/* Process */
LIST *list = list_new();

for JOURNAL *journal in journal_tax_form_list
{
	double journal_amount(
		journal->debit_amount,
		journal->credit_amount,
		journal->account->subclassification->element->accumulate_debit );

	void tax_form_entity_getset(
		list,
		journal->full_name,
		journal_amount() ) );
}

/* Usage */
void tax_form_entity_getset(
	LIST *list,
	char *full_name,
	double journal_amount );
```

```
/* Process */
for TAX_FORM_ENTITY *tax_form_entity in list
{
	if ( strcmp(
		tax_form_entity->entity->full_name,
		full_name ) == 0 )
	{
		tax_form_entity->total += journal_amount;
		return;
	}
}

TAX_FORM_ENTITY *tax_form_entity_calloc( void );

ENTITY *entity =
	entity_new(
		full_name,
		(char *)0 /* street_address */ );

total = journal_amount;

list_set( list, tax_form_entity_calloc() );

/* Attributes */
ENTITY *entity;
double total;
```

TAX_FORM_TABLE

```
/* Usage */
TAX_FORM_TABLE *tax_form_table_new(
      char *tax_form_name,
      STATEMENT_CAPTION *statement_caption,
      LIST *tax_form_line_list() );

/* Process */
TAX_FORM_TABLE *tax_form_table_calloc( void );

LIST *html_table_list = list_new();

TAX_FORM_LINE_HTML_TABLE *
   tax_form_line_html_table =
         tax_form_line_html_table_new(
               tax_form_name,
               statement_caption->subtitle,
               tax_form_line_list );

list_set(
   html_table_list,
   tax_form_line_html_table->html_table );

TAX_FORM_ACCOUNT_HTML_LIST *
      tax_form_account_html_list =
            tax_form_account_html_list_new(
                  tax_form_line_list );

list_set_list(
   html_table_list,
   tax_form_account_html_table_list->
         html_table_list );

TAX_FORM_ENTITY_HTML_LIST *
      tax_form_entity_html_list =
            tax_form_entity_html_list_new(
                  tax_form_line_list );

list_set_list(
   html_table_list,
   tax_form_entity_html_table_list->
         html_table_list );
```



HTML_TABLE

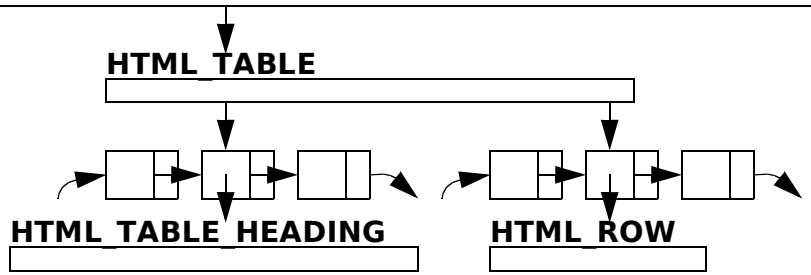
TAX_FORM_LINE_HTML_TABLE

TAX_FORM_ACCOUNT_HTML_TABLE_LIST

TAX_FORM_ENTITY_HTML_TABLE_LIST

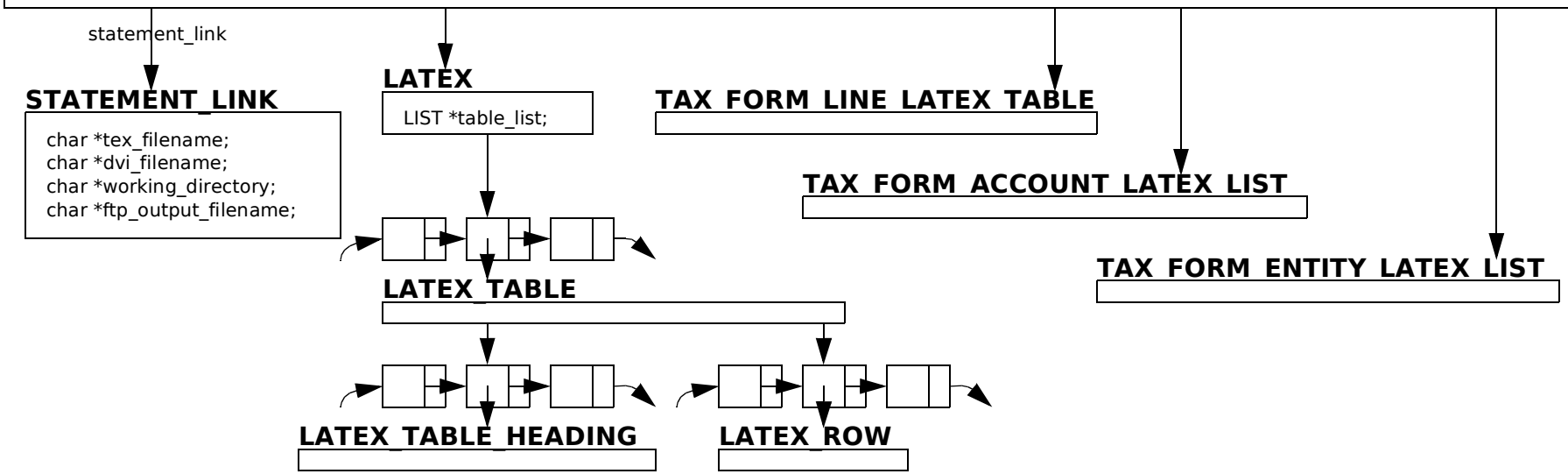
TAX_FORM_LINE_HTML_TABLE

```
/* Usage */
TAX_FORM_LINE_HTML_TABLE *
   tax_form_line_html_table_new(
         char *tax_form_name,
         char *statement_caption_subtitle,
         LIST *tax_form_line_list );

/* Process */
TAX_FORM_LINE_HTML_TABLE *
   tax_form_line_html_table_calloc(
         void );

char *tax_form_line_html_table_caption(
   char *tax_form_name,
   char *statement_caption_subtitle );

HTML_TABLE *html_table =
      html_table_new(
            (char *)0 /* title */,
            tax_form_line_html_table_caption(),
            (char *)0 /* subsub_title */ );

html_table->heading_list =
      LIST *tax_form_line_html_table_heading_list();

html_table->row_list =
      LIST *tax_form_line_html_table_row_list(
            tax_form_line_list );

/* Usage */
LIST *tax_form_line_html_table_heading_list( void );

/* Process */
list_set(
   heading_list,
   html_table_heading_new(
         “tax_form_line“,
         0 /* right_justify_flag */ ) );

list_set(
   heading_list,
   html_table_heading_new(
         “tax_form_description”,
         0 /* not right_Justified_flag */ ) );

list_set(
   heading_list,
   html_table_heading_new(
         “balance”,
         1 /* right_justified_flag */ ) );

/* Usage */
LIST *tax_form_line_html_table_row_list(
      LIST *tax_form_line_list );

/* Process */
for TAX_FORM_LINE *tax_form_line in tax_form_line_list
{
   if ( !money_virtually_same(
         tax_form_line->total, 0.0 ) )
   {
         list_set(
               list,
               HTML_ROW *tax_form_line_html_table_row(
                     tax_form_line ) );
   }
}

/* Usage */
HTML_ROW *tax_form_line_html_table_row(
   TAX_FORM_LINE *tax_form_line );

/* Process */
HTML_ROW *html_row = html_row_new();

html_cell_set(
   html_row->cell_list,
   tax_form_line->tax_form_line_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_set(
   html_row->cell_list,
   tax_form_line->tax_form_description,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

html_cell_set(
   html_row->cell_list,
   strdup(
         timlib_dollar_round_string(
               tax_form_line->total ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```



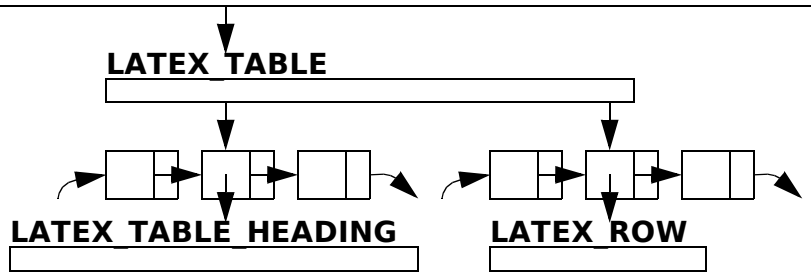
TAX_FORM_PDF

```
/* Usage */
TAX_FORM_PDF *tax_form_pdf_new(
      char *application_name,
      char *process_name,
      char *document_root_directory,
      char *tax_form_name,
      STATEMENT_CAPTION *statement_caption,
      LIST *tax_form_line_list() );

/* Process */
TAX_FORM_PDF *tax_form_pdf_calloc( void );

STATEMENT_LINK *statement_link =
   statement_link_new(
         application_name,
         process_name,
         document_root_directory,
         statement_caption->begin_date_string,
         statement_caption->end_date_string,
         (char *)0 /* statement_pdf_preclose_key */,
         get_pid() /* process_id */ );

LATEX *latex =
   latex_new(
         statement_link->tex_filename,
         statement_link->dvi_filename,
         statement_link->working_directory,
         0 /* not statement_pdf_landscape_boolean */,
         (char *)0 /* statement_logo_filename */ );

TAX_FORM_LINE_LATEX_TABLE *
   tax_form_line_latex_table =
         tax_form_line_latex_table_new(
               tax_form_name,
               statement_caption->subtitle,
               tax_form_line_list );

list_set(
   latex->table_list,
   tax_form_line_latex_table->latex_table );

TAX_FORM_ACCOUNT_LATEX_LIST *
      tax_form_account_latex_list =
            tax_form_account_latex_list_new(
                  tax_form_line_list );

list_set_list(
   latex->table_list,
   tax_form_account_latex_table_list->
         latex_table_list );

TAX_FORM_ENTITY_LATEX_LIST *
      tax_form_entity_latex_list =
            tax_form_entity_latex_list_new(
                  tax_form_line_list );

list_set_list(
   latex->table_list,
   tax_form_entity_latex_table_list->
         latex_table_list );
```



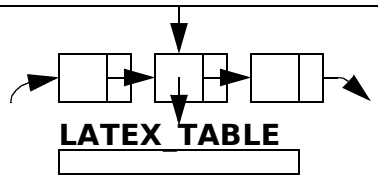
TAX_FORM_LINE_LATEX_TABLE

```
/* Usage */
TAX_FORM_LINE_LATEX_TABLE *
   tax_form_line_latex_table_new(
         char *tax_form_name,
         char *statement_caption_subtitle,
         LIST *tax_form_line_list );

/* Process */
TAX_FORM_LINE_LATEX_TABLE *
   tax_form_line_latex_table_calloc(
         void );

char *tax_form_line_latex_table_caption(
   char *tax_form_name,
   char *statement_caption_subtitle );

LATEX_TABLE *latex_table =
      latex_table_new(
            tax_form_line_latex_table_caption() );

latex_table->heading_list =
      LIST *tax_form_line_latex_table_heading_list();

latex_table->row_list =
      LIST *tax_form_line_latex_table_row_list(
            tax_form_line_list );

/* Usage */
LIST *tax_form_line_latex_table_heading_list( void );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         “tax_form_line“,
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “tax_form_description”,
         0 /* not right_Justified_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         “balance”,
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

/* Usage */
LIST *tax_form_line_latex_table_row_list(
      LIST *tax_form_line_list );

/* Process */
for TAX_FORM_LINE *tax_form_line in tax_form_line_list
{
   if ( !money_virtually_same(
         tax_form_line->total, 0.0 ) )
   {
         list_set(
               list,
               LATEX_ROW *tax_form_line_latex_table_row(
                     tax_form_line ) );
   }
}

/* Usage */
LATEX_ROW *tax_form_line_latex_table_row(
   TAX_FORM_LINE *tax_form_line );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   tax_form_line->tax_form_line_string,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   tax_form_line->description,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup(
         timlib_dollar_round_string(
               tax_form_line->total ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```



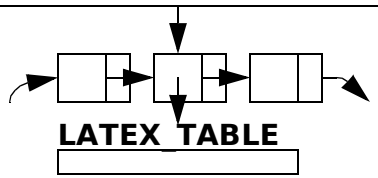
TAX_FORM_ACCOUNT_LATEX_LIST

```
/* Usage */
TAX_FORM_ACCOUNT_LATEX_LIST *
   tax_form_account_latex_list_new(
         LIST *tax_form_line_list );

/* Process */
TAX_FORM_ACCOUNT_LATEX_LIST *
   tax_form_account_latex_list_calloc(
         void );

LIST *list = list_new();

for TAX_FORM_LINE *tax_form_line in tax_form_line_list
{
   if ( !money_virtually_same(
         tax_form_line->total, 0.0 ) )
   {
         list_set(
               list,
               tax_form_account_latex_new(
                     tax_form_line->string,
                     tax_form_line->
                           tax_form_line_account_list,
                     tax_form_line->description,
                     tax_form_line->total ) );
   }
}

/* Usage */
LATEX_TABLE *tax_form_account_latex_new(
   char *tax_form_line_string,
   LIST *tax_form_line_account_list,
   char *tax_form_line_description,
   double tax_form_line_total );

/* Process */
char *tax_form_account_latex_caption(
   char *tax_form_line_sting,
   char *tax_form_line_description,
   double tax_form_line_total );

LATEX_TABLE *latex_table =
      latex_table_new(
            tax_form_account_latex_caption() );

latex_table->heading_list =
      LIST *tax_form_account_latex_heading_list();

latex_table->row_list =
      LIST *tax_form_account_latex_row_list(
            tax_form_line_account_list );

/* Usage */
LIST *tax_form_account_latex_heading_list( void );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         "account_name",
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         "total",
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

/* Usage */
LIST *tax_form_account_latex_row_list(
      LIST *tax_form_line_account_list );

/* Process */
for TAX_FORM_LINE_ACCOUNT *
         tax_form_line_account in
               tax_form_line_account_list
{
   if ( !money_virtually_same(
         tax_form_line_account->total, 0.0 ) )
   {
         list_set(
               list,
               tax_form_line_account_latex_row(
                     tax_form_line_account ) );
   }
}

/* Usage */
LATEX_ROW *tax_form_line_account_latex_row(
   TAX_FORM_LINE_ACCOUNT *tax_form_line_account );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   tax_form_line_account->account_name,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup(
         timlib_dollar_round_string(
               tax_form_line_account->total ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```



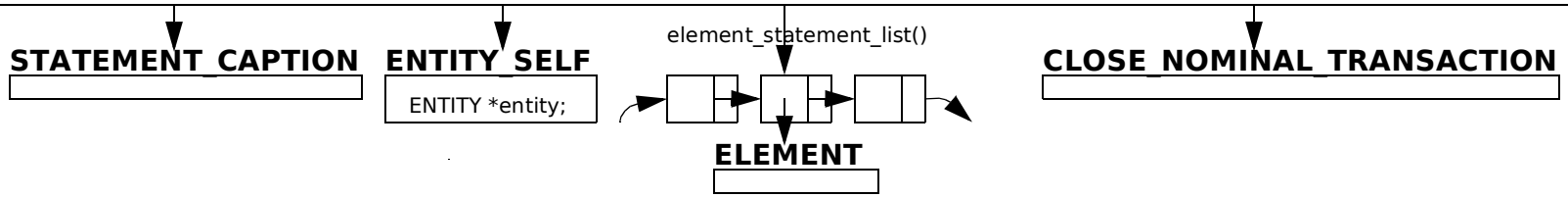
TAX_FORM_ENTITY_LATEX_LIST

```
/* Usage */
TAX_FORM_ENTITY_LATEX_LIST *
   tax_form_entity_latex_list_new(
         LIST *tax_form_line_list );

/* Process */
TAX_FORM_ENTITY_LATEX_LIST *
   tax_form_entity_latex_list_calloc(
         void );

LIST *list = list_new();

for TAX_FORM_LINE *tax_form_line in tax_form_line_list
{
   if ( money_virtually_same(
         tax_form_line->total, 0.0 ) )
   {
         continue;
   }

   for TAX_FORM_LINE_ACCOUNT *
         tax_form_line_account in
               tax_form_line->
                     tax_form_line_account_list
   {
         list_set(
               list,
               tax_form_account_entity_latex_new(
                     tax_form_line->string,
                     tax_form_line_account->
                           account_name,
                     tax_form_line_account->total,
                     tax_form_line_account->
                           tax_form_entity_list ) );
   }
}

/* Usage */
LATEX_TABLE *tax_form_account_entity_latex_new(
   char *tax_form_line_string,
   char *account_name,
   double tax_form_line_account_total,
   LIST *tax_form_entity_list );

/* Process */
char *tax_form_entity_latex_caption(
   char *tax_form_line_string,
   char *account_name,
   double tax_form_line_account_total );

LATEX_TABLE *latex_table =
      latex_table_new(
            tax_form_entity_latex_caption() );

latex_table->heading_list =
      LIST *tax_form_entity_latex_heading_list();

latex_table->row_list =
      LIST *tax_form_entity_latex_row_list(
            tax_form_entity_list );

/* Usage */
LIST *tax_form_entity_latex_heading_list( void );

/* Process */
list_set(
   heading_list,
   latex_table_heading_new(
         "full_name",
         0 /* right_justify_flag */,
         (char *)0 /* paragraph_size */ ) );

list_set(
   heading_list,
   latex_table_heading_new(
         "total",
         1 /* right_justified_flag */,
         (char *)0 /* paragraph_size */ ) );

/* Usage */
LIST *tax_form_entity_latex_row_list(
      LIST *tax_form_entity_list );

/* Process */
for TAX_FORM_ENTITY *
         tax_form_entity in
               tax_form_entity_list
{
   if ( !money_virtually_same(
         tax_form_entity->total, 0.0 ) )
   {
         list_set(
               row_list,
               tax_form_entity_latex_row(
                     tax_form_entity ) );
   }
}

/* Usage */
LATEX_ROW *tax_form_entity_latex_row(
   TAX_FORM_ENTITY *tax_form_entity );

/* Process */
LATEX_ROW *latex_row = latex_row_new();

latex_column_data_set(
   latex_row->column_data_list,
   tax_form_entity->entity->full_name,
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );

latex_column_data_set(
   latex_row->column_data_list,
   strdup(
         timlib_dollar_round_string(
               tax_form_entity->total ) ),
   0 /* not large_boolean */,
   0 /* not bold_boolean */ );
```



CLOSE_NOMINAL (1)

```
/* Usage */
CLOSE_NOMINAL *close_nominal_fetch(
   char *application_name,
   char *process_name,
   char *as_of_date );

/* Process */
CLOSE_NOMINAL *close_nominal_calloc( void );

STATEMENT_CAPTION *statement_caption =
      statement_caption_new(
            application_name,
            process_name,
            (char *)0 /* begin_date_string */,
            as_of_date /* end_date_string */ );

char *transaction_date_time_closing(
   TRANSACTION_PRECLOSE_TIME,
   TRANSACTION_CLOSE_TIME,
   as_of_date,
   0 /* not preclose_time_boolean */ );

boolean transaction_date_time_exists(
   TRANSACTION_TABLE,
   TRANSACTION_DATE_TIME_COLUMN,
   transaction_date_time_closing() );

if ( transaction_date_time_exists() ) return this;

LIST *close_nominal_element_name_list(
   char *ELEMENT_REVENUE,
   char *ELEMENT_EXPENSE,
   char *ELEMENT_GAIN,
   char *ELEMENT_LOSS );

LIST *element_statement_list(
   close_nominal_element_name_list(),
   transaction_date_time_closing(),
   1 /* fetch_subclassification_list */,
   1 /* fetch_account_list */,
   1 /* fetch_journal_latest */,
   0 /* not fetch_transaction */ );

void element_list_sum(
   element_statement_list() );

ELEMENT *revenue_element =
   element_seek(
         ELEMENT_REVENUE,
         element_statement_list() );

double revenue_sum =
   element_sum(
         revenue_element );

ELEMENT *gain_element =
   element_seek(
         ELEMENT_GAIN,
         element_statement_list() );

double gain_sum =
      element_sum(
            gain_element );

double close_nominal_debit_sum(
   double revenue_sum,
   double gain_sum );

ELEMENT *expense_element =
   element_seek(
         ELEMENT_EXPENSE,
         element_statement_list() );

double expense_sum =
      element_sum(
            expense_element );

ELEMENT *loss_element =
   element_seek(
         ELEMENT_LOSS,
         element_statement_list() );

double loss_sum = element_sum( loss_element );

char *account_drawing( ACCOUNT_DRAWING_KEY );

double close_nominal_drawing_sum(
   char *transaction_date_time_closing(),
   char *account_drawing() );

double close_nominal_credit_sum(
   double expense_sum,
   double loss_sum,
   double close_nominal_credit_sum() );

double close_nominal_retained_earnings(
   double close_nominal_debit_sum(),
   double close_nominal_credit_sum() );

char *account_closing_entry( ACCOUNT_CLOSING_KEY );

ENTITY_SELF *entity_self = entity_self_fetch();

CLOSE_NOMINALTRANSACTION *
   close_nominal_transaction =
         close_nominal_transaction_new(
               element_statement_list() /* in/out */,
               transaction_date_time_closing(),
               account_drawing(),
               close_nominal_drawing_sum(),
               close_nominal_retained_earnings(),
               account_closing_entry(),
               entity_self->entity );

double journal_debit_sum(
   close_nominal_transaction->
         transaction->
         journal_list );

double journal_credit_sum(
   close_nominal_transaction->
         transaction->
         journal_list );
```

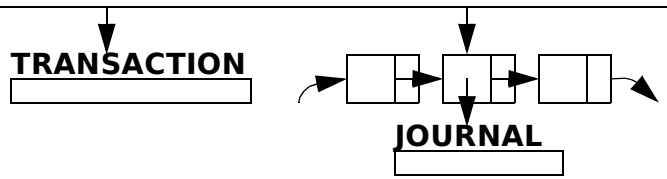


CLOSE_NOMINAL (2)

```
/* Driver */
if ( !transaction_date_time_exists() )
{
   if ( execute )
   {
         void close_nominal_transaction_insert(
               TRANSACTION *close_nominal_transaction->
                     transaction );
   }
   else
   {
         void journal_list_sum_html_display(
               close_nominal_transaction->
                     transaction->
                     journal_list,
               close_nominal_transaction->
                     transaction->
                     transaction_date_time,
               close_nominal_transaction->
                     transaction->
                     transaction_memo
               journal_debit_sum(),
               journal_credit_sum() );
   }
}
```

CLOSE_NOMINAL_TRANSACTION (1)

```
/* Usage */
CLOSE_NOMINAL_TRANSACTION *
   close_nominal_transaction_new(
         LIST *element_statement_list() /* in/out */,
         char *transaction_date_time_closing(),
         char *account_drawing(),
         double close_nominal_drawing_sum(),
         double close_nominal_retained_earnings(),
         char *account_closing_entry(),
         ENITTY *entity );

/* Process */
CLOSE_NOMINAL_TRANSACTION *
   close_nominal_transaction_calloc(
         void );

TRANSACTION *transaction =
   transaction_new(
         entity->full_name,
         entity->street_address,
         transaction_date_time_closing );

transaction->transaction_amount =
   float_abs( close_nominal_retained_earnings );

transaction->memo =
   TRANSACTION_CLOSING_ENTRY_MEMO;

transaction->journal_list =
   LIST *close_nominal_transaction_journal_list(
         element_statement_list /* in/out */,
         transaction_date_time_closing,
         account_drawing,
         close_nominal_drawing_sum,
         close_nominal_retained_earnings,
         account_closing_entry,
         entity );

/* Usage */
LIST *close_nominal_transaction_journal_list(
   LIST *element_statement_list /* in/out */,
   char *transaction_date_time_closing,
   char *account_drawing,
   double close_nominal_drawing_sum,
   double close_nominal_retained_earnings,
   char *account_closing_entry,
   ENTITY *entity );

/* Process */
void element_list_account_statement_list_set(
   element_statement_list );

LIST *journal_list = list_new();

for ELEMENT *element in element_statement_list
{
   for ACCOUNT *account in
         element->account_statement_list
   {
         if ( !account->account_journal_latest->balance )
               continue;

         JOURNAL *journal =
               journal_new(
                     entity->full_name,
                     entity->street_address,
                     transaction_date_time_closing(),
                     account->account_name );

         ACCOUNT *journal_account =
               ACCOUNT *account_fetch(
                     journal->account_name,
                     1 /* fetch_subclassification */,
                     1 /* fetch_element */ );

         journal->debit_amount =
               close_nominal_transaction_debit_amount(
                     element->accumulate_debit,
                     account->
                           account_journal_latest->
                           balance );

         journal->credit_amount =
               close_nominal_transaction_credit_amount(
                     element->accumulate_debit,
                     account->
                           account_journal_latest->
                           balance );

         list_set(
               journal_list,
               journal );
   }
}

if ( close_nominal_drawing_sum )
{
   list_set(
         journal_list,
         JOURNAL *
               close_nominal_transaction_drawing_journal(
                     transaction_date_time_closing,
                     account_drawing,
                     close_nominal_drawing_sum,
                     entity ) );
}

if ( close_nominal_retained_earnings )
{
   list_set(
         journal_list,
         JOURNAL *
               close_nominal_transaction_close_journal(
                     transaction_date_time_closing,
                     retained_earnings,
                     account_closing_entry,
                     entity ) );
}
```

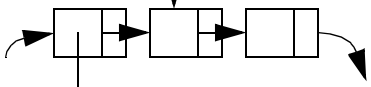


CLOSE_NOMINAL_TRANSACTION (2)

```
/* Usage */
double close_nominal_transaction_debit_amount(
   boolean element_accumulate_debit,
   double balance );

/* Process */
if ( !element_accumulate_debit )
   debit_amount = balance;
else
   debit_amount = 0.0;

/* Usage */
double close_nominal_transaction_credit_amount(
   boolean element_accumulate_debit,
   double balance );

/* Process */
if ( element_accumulate_debit )
   credit_amount = balance;
else
   credit_amount = 0.0;

/* Usage */
JOURNAL *close_nominal_transaction_drawing_journal(
   char *transaction_date_time_closing,
   char *account_drawing,
   double close_nominal_drawing_sum,
   ENTITY *entity );

/* Process */
JOURNAL *journal =
   JOURNAL *journal_new(
         entity->full_name,
         entity->street_address,
         transaction_date_time_closing,
         account_drawing );

ACCOUNT *journal->account =
   ACCOUNT *account_fetch(
         journal->account_name,
         1 /* fetch_subclassification */,
         1 /* fetch_element */ );

journal->credit_amount =
   0.0 - close_nominal_drawing_sum;

/* Usage */
JOURNAL *close_nominal_transaction_close_journal(
   char *transaction_date_time_closing,
   double retained_earnings,
   char *account_closing_entry,
   ENTITY *entity );

/* Process */
JOURNAL *journal =
   JOURNAL *journal_new(
         entity->full_name,
         entity->street_address,
         transaction_date_time_closing,
         account_closing_entry );

ACCOUNT *journal->account =
   ACCOUNT *account_fetch(
         journal->account_name,
         1 /* fetch_subclassification */,
         1 /* fetch_element */ );

if ( retained_earnings > 0.0 )
{
   journal->credit_amount = retained_earnings;
}
else
{
   journal->debit_amount = 0.0 - retained_earnings;
}
```

TAX_RENTAL_PROPERTY

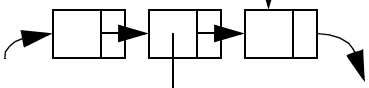
```
/* Usage */
LIST *tax_rental_property_list_fetch(
      char *tax_form_string,
      int tax_year );

/* Attributes */
char  *property_street_address;

/* Process */
LIST *tax_rental_property_string_list(
      tax_rental_property_list );

LIST *tax_form_line_rental_list(
      tax_form_line_list(),
      tax_rental_property_string_list );

double recovery_amount;
double revenue_total [23a];
double mortgage_interest_total [23c];
double depreciation_total [23d];
double expense_total [23e];
double positive_net_income [24];
double loss_total [25];
double tax_rental_property_total();
```



TAX_FORM_LINE_RENTAL

```
char *tax_form_line;
char *tax_form_description;

enum tax_form_line_category;
LIST *tax_form_line_account_list();

LIST *rental_property_list;
double tax_form_line_rental_property_total();
```

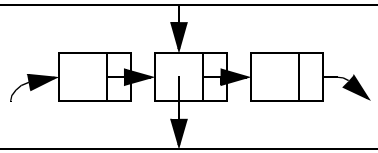
revenue
mortgage_interest_paid
depreciation_expense
operating_expense
loss



TAX_FORM_LINE_ACCOUNT

INVESTMENT_ACCOUNT

```
ENTITY *financial_entity =
      entity_new(
            full_name,
            street_address );

char *account_number;
char *investment_classification;
char *investment_purpose;

LIST *investment_account_balance_list(
      full_name,
      street_address,
      account_number );
```



ACCOUNT_BALANCE

```
char *date
double balance;
double balance_change;
int balance_change_percent;
```

ACCOUNT_BALANCE_EQUITY

char *date_time char *investment_operation double share_price doubel share_quantity_change double share_quantity_balance double cash_in double market_value double book_value_change double book_value_balance	double moving_share_price double unrealized_gain_balance double unrealized_gain_change double reaqlized_gain boolean table_first_row char *transaction_date_time	TRANSACTION * transaction

TRANSACTION